THE UNIVERSITY OF SYDNEY

School of Information Technologies
Faculty of Engineering & IT

ASSIGNMENT/PROJECT COVERSHEET - GROUP ASSESSMENT

Unit of Study: SOFT2412 Agile Software Development Practices

Assignment name: Group Project Assignment 2 - Sprint 1 Report

Tutorial time: Thursday 8am **Tutor name:** Stephanie Dunn

DECLARATION
We the undersigned declare that we have read and understood the *University of Sydney Student Plagiarism: Coursework Policy and Procedure*, and except where specifically acknowledged, the work contained in this assignment/project is our own work, and has not been copied from other sources or been previously submitted for award or assessment.

We understand that understand that failure to comply with the *Student Plagiarism: Coursework Policy and Procedure* can lead to severe penalties as outlined under Chapter 8 of the *University of Sydney By-Law 1999* (as amended). These penalties may be imposed in cases where any significant portion of my submitted work has been copied without proper acknowledgement from other sources, including published works, the internet, existing programs, the work of other students, or work previously submitted for other awards or assessments.

We realise that we may be asked to identify those portions of the work contributed by each of us and required to demonstrate our individual knowledge of the relevant material by answering oral questions or by undertaking supplementary work, either written or in the laboratory, in order to arrive at the final assessment mark.

Project team members				
Student name	**Student ID**	**Participated**	**Agree to share**	**Signature**
1. Zhiyuan Na	530002077	Yes / No	Yes / No	NA, ZHIYUAN
2. Huiying Jiao	530002664	Yes / No	Yes / No	Huiying Jiao
3. XINCHEN HUANG	510005801	Yes / No	Yes / No	XINCHEN HUANG
4. Yilin Li	530536402	Yes / No	Yes / No	
5.		Yes / No	Yes / No	Junrui Kang
6.		Yes / No	Yes / No	
7.		Yes / No	Yes / No	
8.		Yes / No	Yes / No	
9.		Yes / No	Yes / No	
10.		Yes / No	Yes / No	

Level 2, SIT Building, J12
The University of Sydney
NSW 2006 Australia

T +61 2 9351 3423
F +61 2 9351 3838
E sit.info@sydney.edu.au
sydney.edu.au/it

ABN 15 211 513 464
CRICOS 00026A

SOFT2412 Project Report - Sprint 2

GitHub Repository

Access the project repository on GitHub: https://github.sydney.edu.au/SOFT2412-COMP9412-2024Sem2/Steph_Lab11_Group01_Assignment2

Individual Contribution

A detailed account of my contributions and self-evaluation is available in the appendix of this report.

Meeting Records

Information about all meetings, including recording links, can be found in the appendix of this report.

1 Sprint Goal

In the second sprint of developing the Virtual Scroll Access System (referred to as VSAS throughout this report), our focus is on addressing bugs, enhancing features, and implementing new stakeholder requirements. Below is a detailed breakdown of our key sprint activities:

1.1 Sensibility

Fixing Bugs Identified in Sprint 1 Review Session:

- **Validating User Input**: Ensuring that user inputs, such as email addresses and phone numbers, conform to the expected formats to improve data integrity and user experience.
- **Validating File Formats**: Verifying the format of files when users upload or edit scroll files to prevent compatibility issues and maintain consistent data structure.
- **UI Bug Fixes**: Addressing other minor bugs found in the user interface to optimise the overall functionality and usability of the system.

1.2 Relevance

Enhancing Current Features and User Experience:

- **Preview Scroll Content**: Implementing a feature that allows users to preview scroll content before downloading, offering more flexibility and ensuring they select the correct files.
- **Search Functionality for Scrolls**: Enhancing the search feature by allowing users to search for scrolls based on attributes such as scroll name, uploader, and upload time (with a range filter). This will help users locate specific scrolls more efficiently.
- **Scroll Statistics**: Adding functionality for users to view statistics related to a particular scroll, including the number of times it has been uploaded and downloaded, providing useful insights into scroll popularity.

1.3 Clarity

Implementing New Feature Requested by the Stakeholder:

- **Daily Featured Scroll**: Introducing a new feature that displays a randomly selected scroll as the "Lucky Scroll of the Day." This randomly featured scroll is updated every day, encouraging users to engage with diverse content and increasing overall user interaction with the application.

These tasks form the core objectives of Sprint 2, driving us towards a more robust, user-friendly, and engaging version of VSAS.

2 Scrum Events and Artefacts

Scrum events and artifacts are essential components of the Scrum framework, providing structure, transparency, and continuous improvement throughout the development process. In this project, we adhere to these principles by implementing the following key Scrum events and artifacts:

- **Scrum Team Formation**: Earlier in the project, we defined clear roles and responsibilities within the team to establish a foundation for effective collaboration and accountability.
- **Meetings**:
  - **Sprint Planning Meetings**: At the start of each sprint, we outline the sprint goals and define the tasks necessary to achieve those goals.
  - **Daily Scrum Meetings**: These brief stand-up meetings occur daily, allowing the team to synchronise their activities, discuss progress, identify any obstacles, and plan their work for the day.
  - **Sprint Review Meetings**: At the end of each sprint, we review the work completed with stakeholders, gather feedback, and validate outcomes.
  - **Sprint Retrospective Meetings**: Conducted after each sprint, the team reflects on what went well, what didn't, and how we can improve.
- **Writing User Stories and Sprint Backlogs**: User stories are created to document functional requirements and the desired outcomes.
- **Estimate Workload Using Story Points and Burndown Chart**: We estimate the complexity and effort of tasks using story points, allowing us to track progress with a burndown chart.
- **Performing Acceptance Tests and Code Reviews**: We perform acceptance tests and code reviews to verify that features meet the specified criteria and are functioning as intended.
- **Maintaining Product and Sprint Backlogs**: We maintain an up-to-date product backlog, prioritising user stories based on stakeholder needs and project goals.
- **Tracking Progress with Velocity Metrics**: To visualise the flow of tasks and monitor the team's progress, we track team velocity to measure the amount of work completed in each sprint.

2.1 Scrum Team Revisited

> **Reference**
>
> Please refer to Sprint 1 report (section 2.1) to see the team formation process.

In this project, to establish an effective Scrum team, the following roles and responsibilities are defined:

- **Product Owner**: The Product Owner is responsible for defining the features of the product and ensuring that the team delivers value to the stakeholders.
  - Contribution to VSAS Sprint 2:
    - Communication with the client: This is a important role for the product owner through fact-to-face way to talk with the client, and catch out the expected target (e.g, UI/feature changes) they want, and then "translate" into the way that group developers are able to understand.
    - New User story creation: The product owner should create the user stories after discussing with the client.
    - New tasks sending: With the previous process, a good division of tasks to developers is also an essential role for the product owner to lead with them to finish the sprint.
- **Scrum Master**: The Scrum Master facilitates the Scrum process and ensures that the team adheres to Agile principles.
  - Contribution to VSAS Sprint 2:
    - Task follow-up: As a scrum master, following with the progress of the task is really important. It influences the whole process and makes sure each developer doing on the right track without missing understanding.
    - Team Meeting Starter: Another role of a scrum master needs to conduct every daily meeting and assists with the product owner that checks the progress of each developer's work and whether the direction of understanding is consistent with the general picture.
    - Team cohesion: The scrum master sometimes may be a character who promotes team cohesion by inspiring every developer. The scrum project focuses on teamwork, so this is also important.
- **Development Team**: This is a group of professionals with coding skills necessary to deliver the product increment.
  - Understanding: A good developer should have to brainstorm the methodologies how they solve assigned task.
  - Communicating: As the group work, the communication in the team cannot be missed which ensures the coding coverage each taken and the whole complexity

of the project. Also, some small points like the coding style should be talked between developers tha can be modified at the first time.

- Testing: This is one of the important points working on the group that could be easily missing. Each developer should write the basic JUnit testcase for their implemented methods. This can minimise the risks happening during the code generating and quickly find out the issue parts. Otherwise, it could generate the massive working to fix a tiny coding bug.

Name (Unikey)	**Scrum Role**
Yilin Li (yili2677)	Product Owner, Development Team
Junrui Kang (jkan3790)	Scrum Master, Development Team
Xinchen Huang (xhua2497)	Development Team
Huiying Jiao (hjia7380)	Development Team
Zhiyuan Na (zhna4143)	Development Team

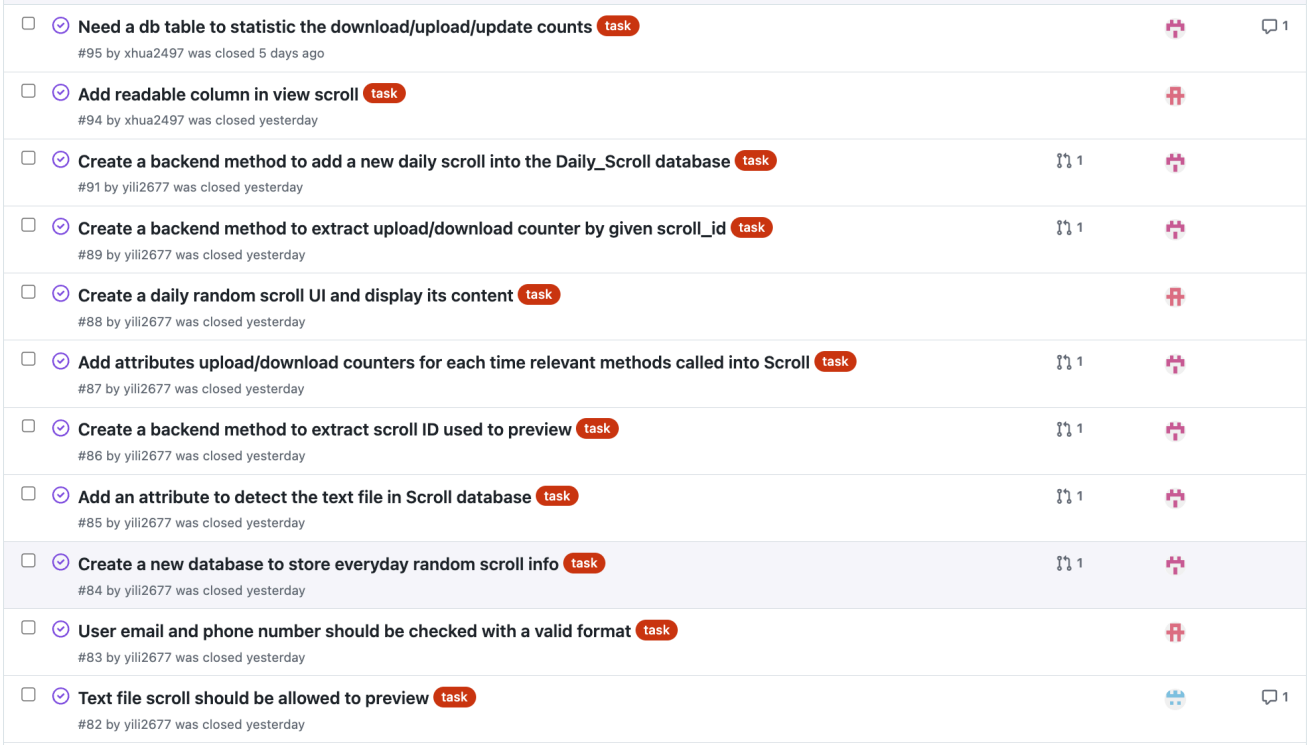
In Sprint 2, these roles remain unchanged, providing continuity and allowing team members to build on their expertise. The Product Owner and Scrum Master continue to contribute as part of the Development Team, effectively balancing their primary responsibilities with hands-on development work. This dual-role approach enhances the team's cohesion and ensures that critical tasks receive attention without sacrificing the agile processes crucial to the project's success.

2.1.1 Sprint 2 Group Member Contribution

Product Owner: Yilin Li (yili2677)

As the product owner, after sprint 1 finished, I communicated with the client during a tutorial (Thursday 10/10/2024 around 8:40 a.m). During the demo, the client has provided the new feature that needs to be updated and points out some errors that need to be fixed. As the sprint 1 demo, the client points out there should be the user information format checking (e.g., a phone number can be strictly an Australian phone number type). Also, the preview and filter features mentioned in sprint 0 should be quickly implemented. After that, the client wants a new feature that will allow the program to generate a random daily scroll with content that cannot be changed during an actual date.

In order to reach the target that the client wanted, firstly I conducted into the user story in such format: “As a/an xx, I want to xx, so I can xx” for developers easily to understand and clarify how to split the tasks for backends and frontends. Therefore, the user story #81 is published into the scrum project and ready for the sprint 2 task build. And previously created user stories #17, #15 are moved into this sprint to continue implementing.

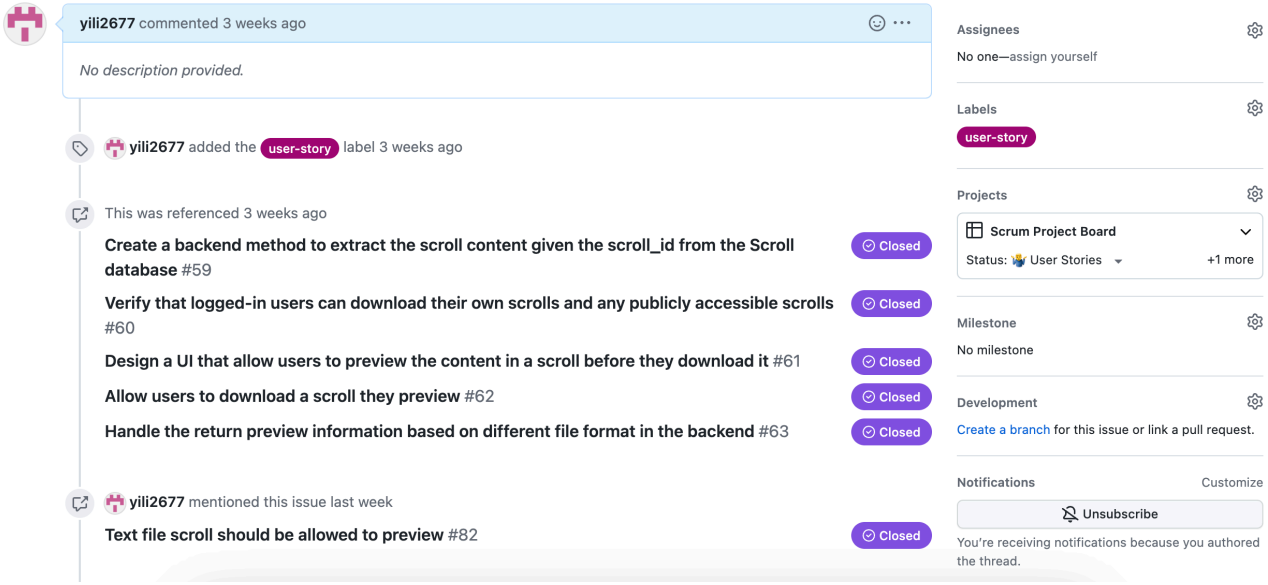


Based on considering how the developers understand what features the client expected and how to use the code language to implement them. I have sent a range of tasks based on the user stories that need to be worked on in sprint 2 with assignees and approximate method logic and method name. At this stage, the developers can quickly get started on the project construction and then communicate with other developers to facilitate collaborative decision-making. Therefore, several tasks are set up.

As a user, I want to preview scrolls, so I can view first before downloading. #17

Edit | New issue

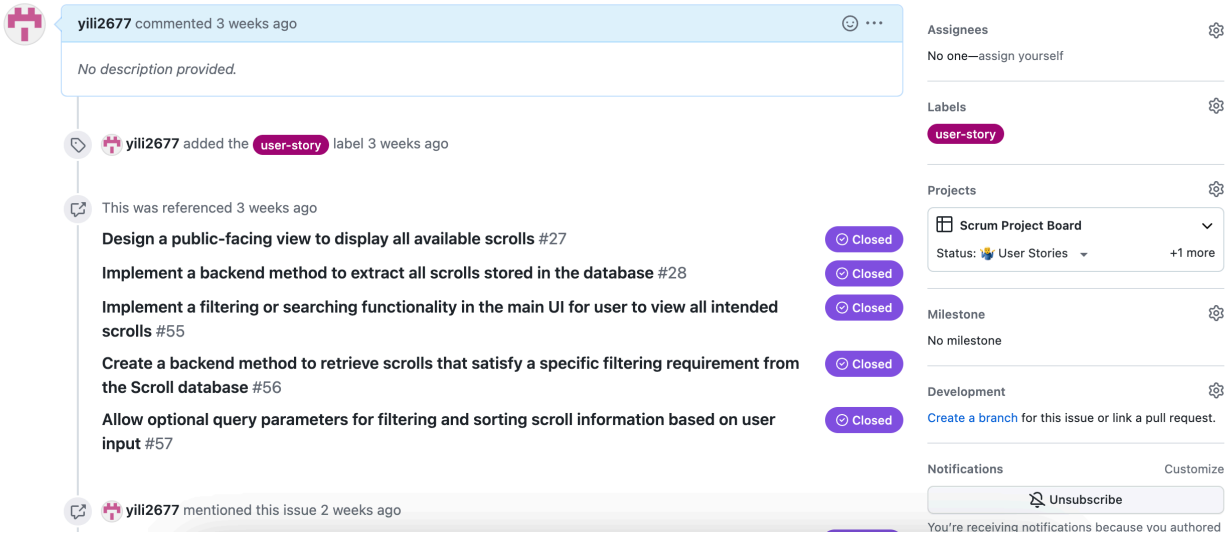
Open | yili2677 opened this issue 3 weeks ago · 0 comments

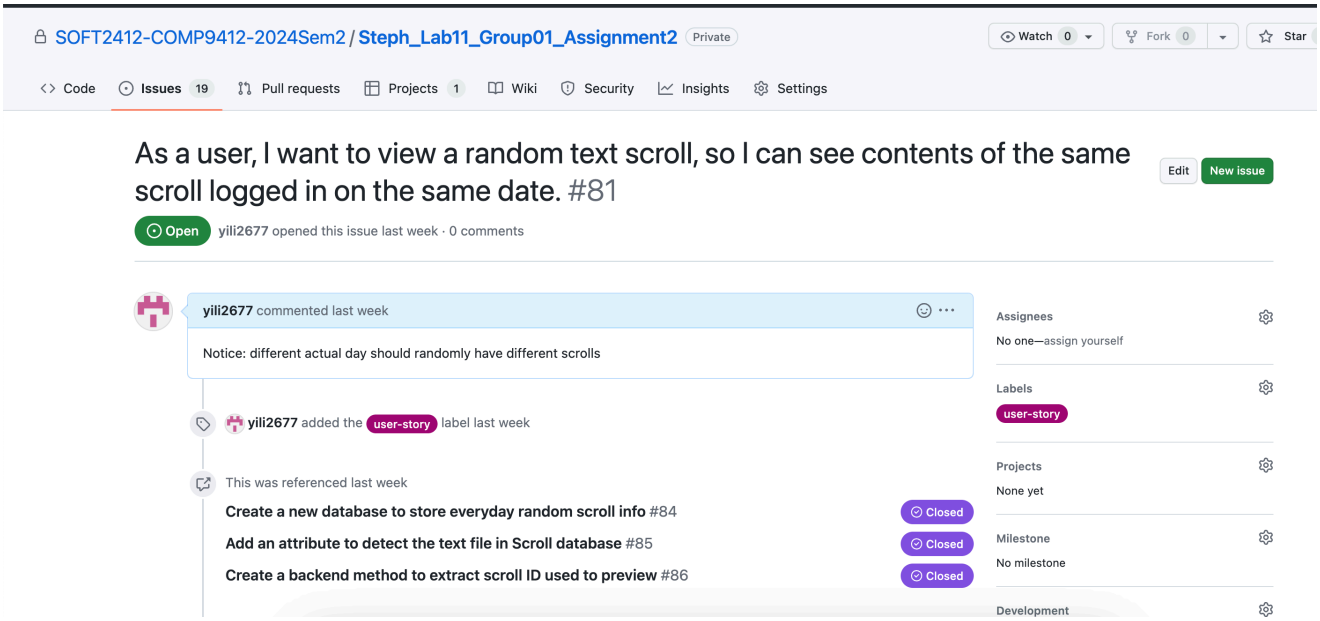


As a user, I want to view all available scrolls, so I can implement this in any possible ways (list of title names, timestamp, uploader). #15

Edit | New issue

Open | yili2677 opened this issue 3 weeks ago · 0 comments



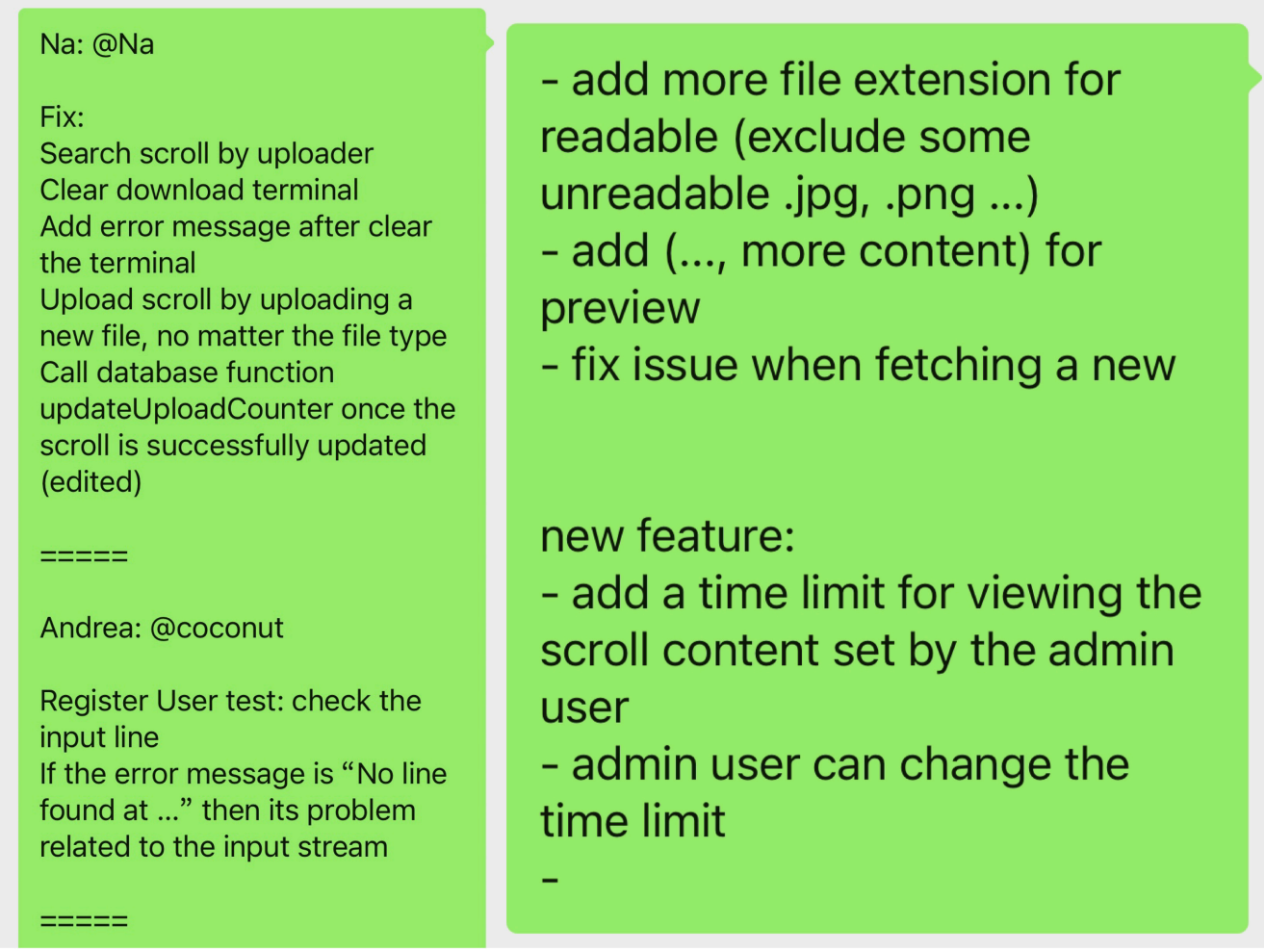


As the backend developer, I have implemented a Daily_Scroll database [`createDailyScrollTable` on `Setup.java`, `addDailyScroll`, `getScrollIDOfTheDay`], which is utilised on storing the date and scroll ID related to the Scroll database for finding the valid text scroll for content viewing [`searchScrollInfoByUploader`, `searchScrollInfoByTimeRange`, `searchScrollInfoByName`]. Also, the getting queries for filter methods based on given scroll ID, date range and user ID or name for the front ends to call and return the scroll information. And I have inserted the new attributes on the Scroll database and also for statistic retrieval [`getAllTextScrolls`, `getScrollStatistics`, `updateDownloadCounter`, `updateUploadCounter`].

Scrum Master: Junrui Kang (jkan3790)

As the scrum master, after the product owner published the tasks for this sprint 2, I have developed the new scrum meetings schedule with the team. I assisted with the product owner to build up the new sprint backlogs and double-check the validation of each task description as long as it becomes reasonable. Furthermore, I have independently communicated with every developer to ensure they have fully understood the goal of their assigned tasks.

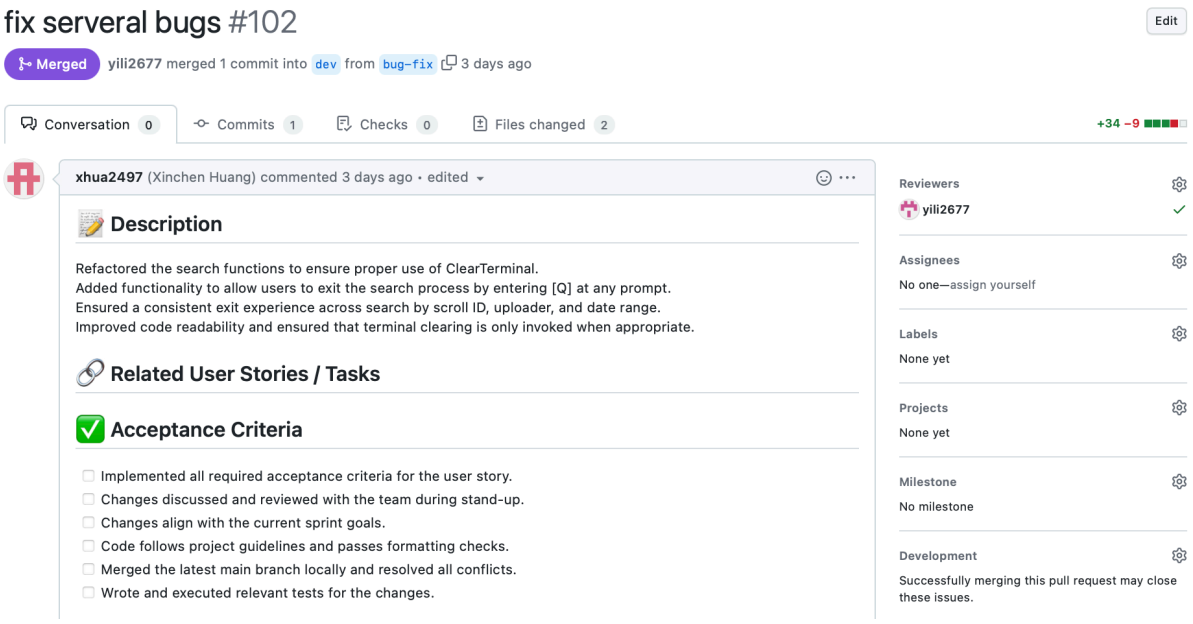
For the team meeting, I have come up with totally three types: sprint planning meetings, daily scrum meetings, and sprint review meetings. Different meeting types have different roles, which ensures that under the scrum project the risks and the percentage of redo become lowest. Before each meeting starts, I would remind everyone at the beginning of the date and also take notes for the bugs that needed to be fixed coming up with the team meeting. In addition, at each meeting, I have recorded to ensure some unpredicted situations happen or any developers forget their extensions developed on the team meeting.

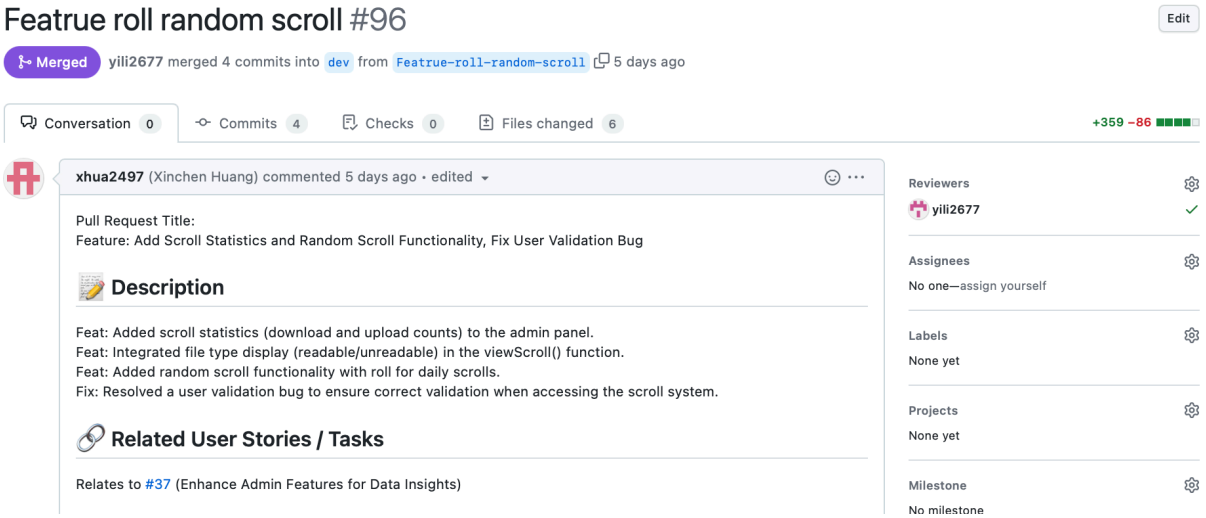


Finally, sometimes I take the role of team cohesion. This commonly happens when two or more developers come with different brainstorming and start to argue. Then I need to moderate the atmosphere to ensure that emotions are minimised.
As the developer, I implement the backend role on the scroll preview methods. To consider that the preview needs to display at least 20 characters on the screen, then the methodology is performed to return the text content [`getAllTextScrolls`, `getScrollPreview`].

Development Team: Xinchen Huang (xhua2497)

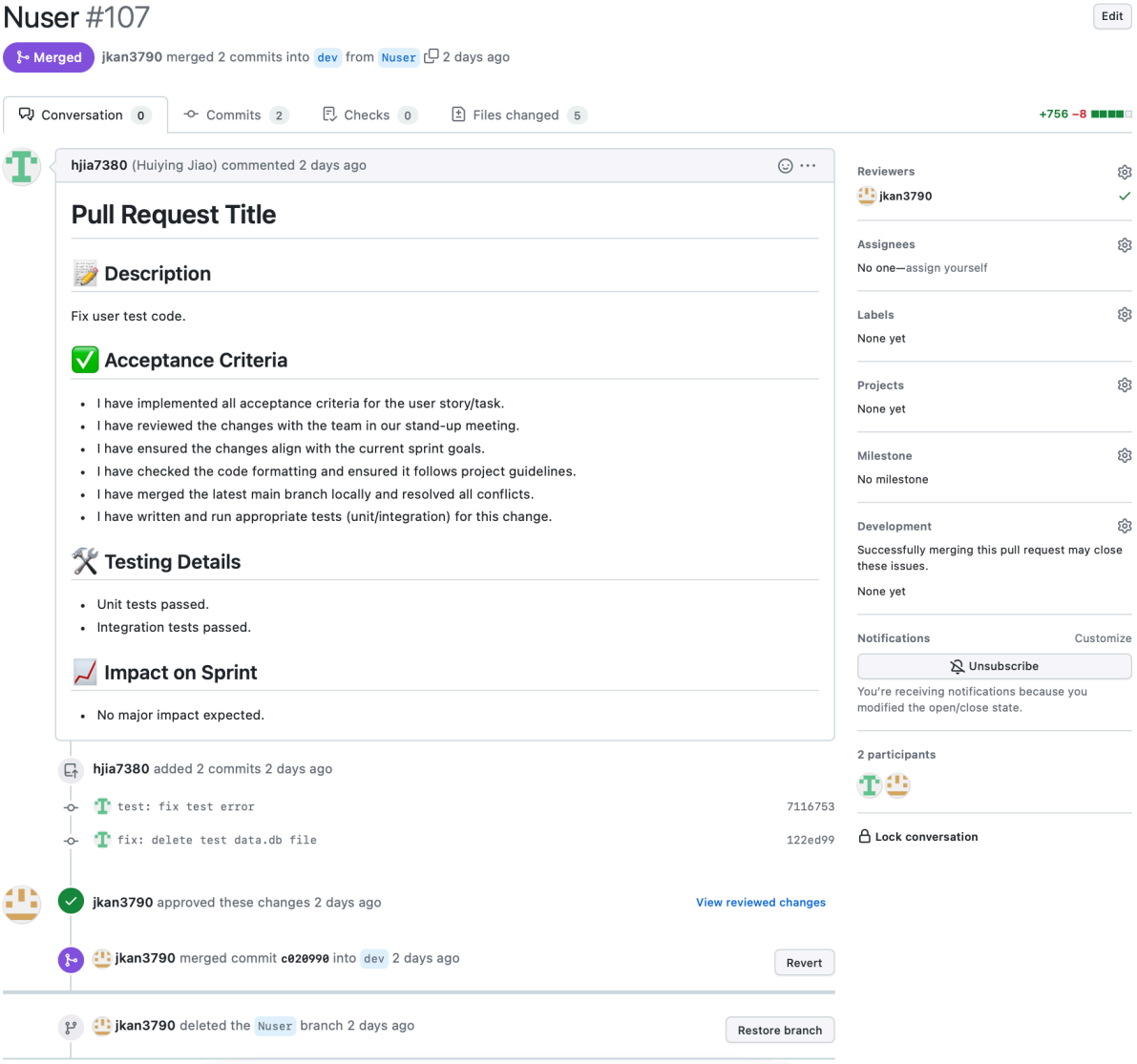
During Sprint 2, I was responsible for implementing the front-end of the "PickLuckyScrollForDay" method, as required by the client. Additionally, I developed the front-end for the "Preview Scroll" feature, enabling users to have a clearer view of the details within their selected scroll. Besides, I also fix some bugs for validation, registration and Search Scroll Prompt.





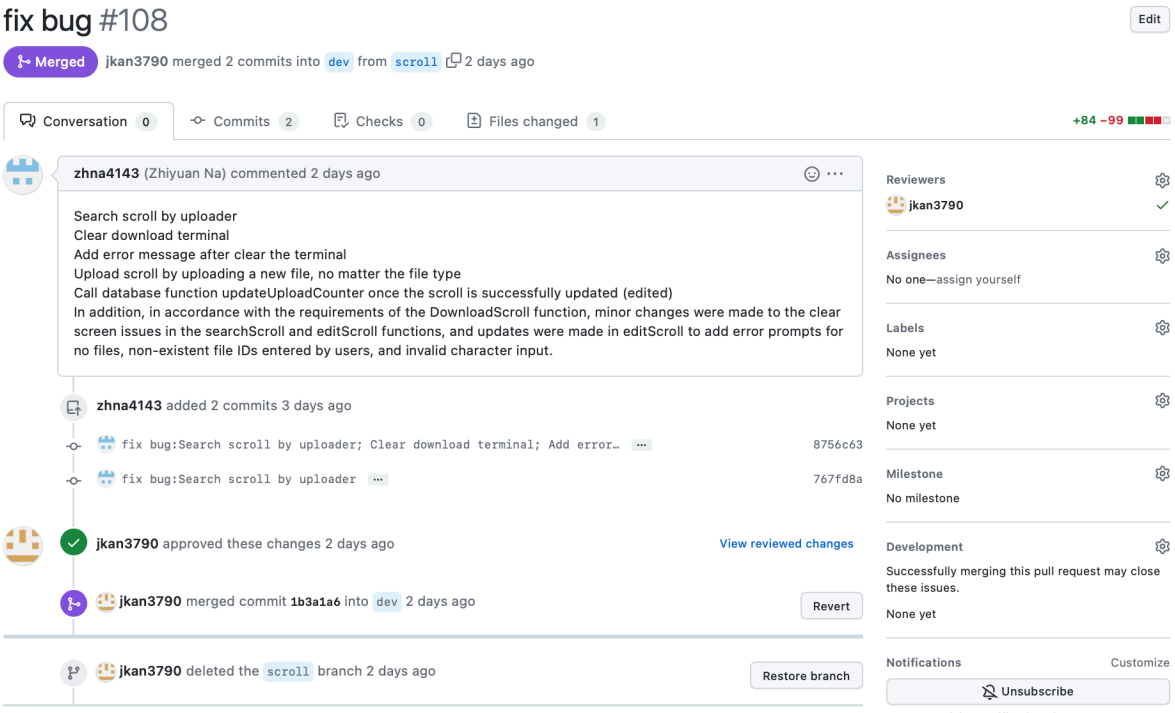
Development Team: Huiying Jiao (hjia7380)

In sprint 2, I focused on updating registered users, refining registration limitations and format. I also tested profile updates and the registered user files, enhancing test coverage to ensure all key functionalities were properly validated.



Development Team: Zhiyuan Na (zhna4143)

In Sprint 2, I implemented the searchScroll method, which includes three search options based on scroll ID, uploader, and date range, along with its UI. Additionally, I optimized the UI of the downloadScroll and editScroll methods, which I was previously responsible for. The main improvements include clearing the screen before each navigation and minimizing the use of Y/N questions, instead using options like 1, 2, 3. Furthermore, to enhance the user experience, I added a preview function before downloading. This allows users to preview a file before deciding whether to download it .Changed the method for 'Update' by replacing the original file content modification with changing the file, i.e., uploading a new file.



2.2 Meetings

Effective communication and coordination are critical to the success of our project, and various Scrum meetings are conducted to ensure alignment, progress tracking, and continuous improvement.

2.2.1 Sprint Retrospective Meetings (for Sprint 1)

> **Important**
>
> This meeting can be found in the meeting record in the appendix.

The Sprint Retrospective for Sprint 1 was held right after the review meeting with the stakeholder (tutorial), focusing on identifying what went well, what didn't, and areas for improvement in the Sprint 1.

Several potential issues of the application were detected during the review session, and they have been categorised as follows:

- It was observed that user inputs, such as email addresses and phone numbers, did not always conform to the expected formats. This inconsistency poses a risk to data integrity and can negatively impact the user experience. To address this, input validation mechanisms are being enhanced to ensure that all data entered meets predefined standards, reducing the likelihood of incorrect data entries.
- Another issue identified was the need for verifying file formats when users upload or edit scroll files. Incorrect file formats can lead to compatibility issues on preview functionality. To prevent this, we are implementing a file format validation system that checks for compatibility and consistency before any file is accepted or modified.
- Minor bugs and inconsistencies were also detected in the user interface, affecting usability and the overall user experience. These issues range from misaligned elements to inconsistent behaviours across different screens. Addressing these bugs is a priority to optimise the system's functionality and provide a smooth and intuitive interface for users.

The team also reflected on the individual collaboration, discussed the efficiency of their processes, and shared insights on challenges faced. Key action items from the retrospective included refining the bug tracking process and improving communication between team members.

These insights were carried forward into Sprint 2 to enhance team performance.

2.2.2 Sprint Planning Meetings

> **Important**
>
> This meeting can be found in the meeting record in the appendix.

2.2.2.1 Correctness

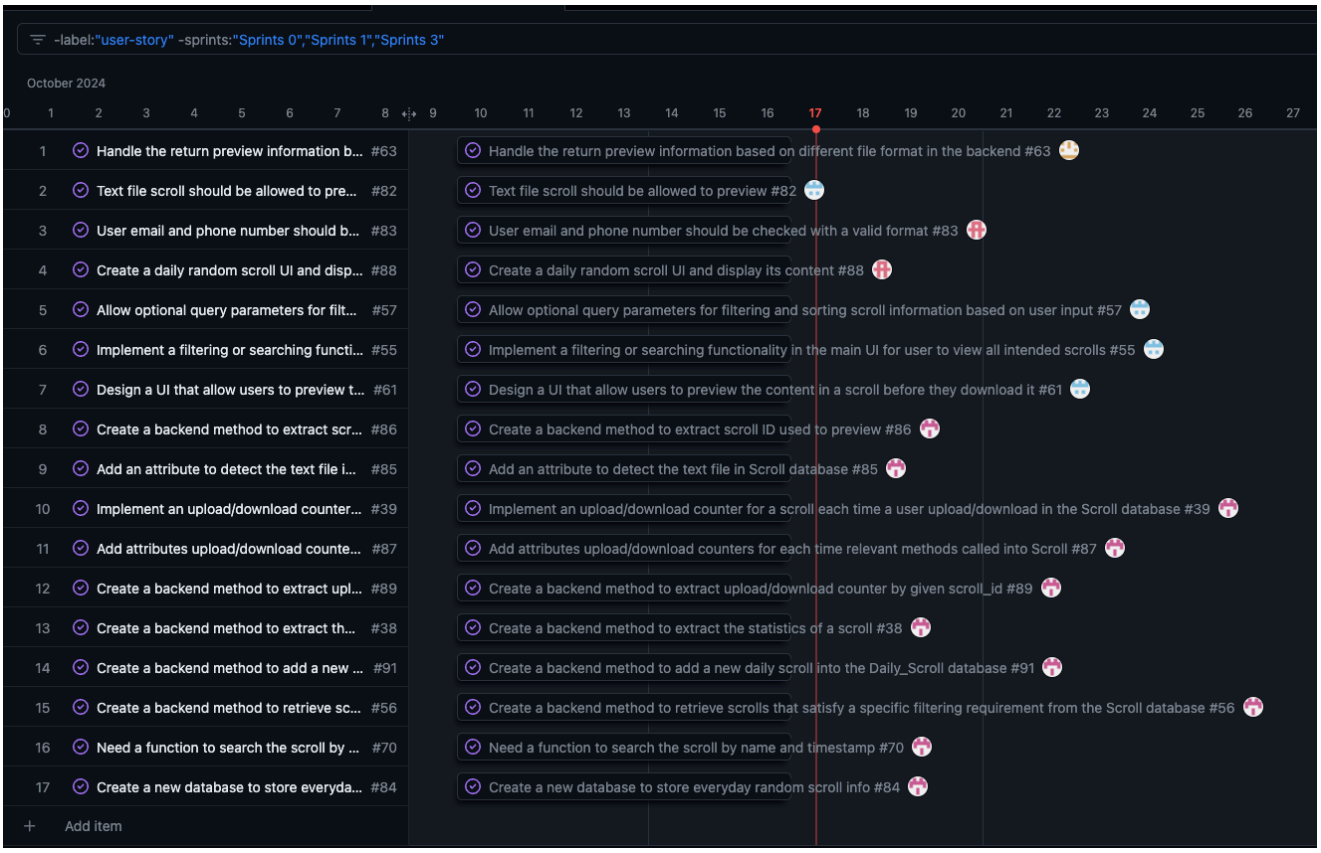
Sprint Planning Meeting is conducted at the beginning of the Sprint 2 to define the sprint goal and establish a clear plan for achieving it. In this sprint, the Sprint Planning meeting happened on Friday (the day after the tutorial).

The team reviews the product backlog, selects high-priority user stories, and breaks them down into actionable tasks in the scrum project board. During the sprint planning meeting, the team also estimates story points for each task and discusses potential risks or obstacles that might impact the sprint.

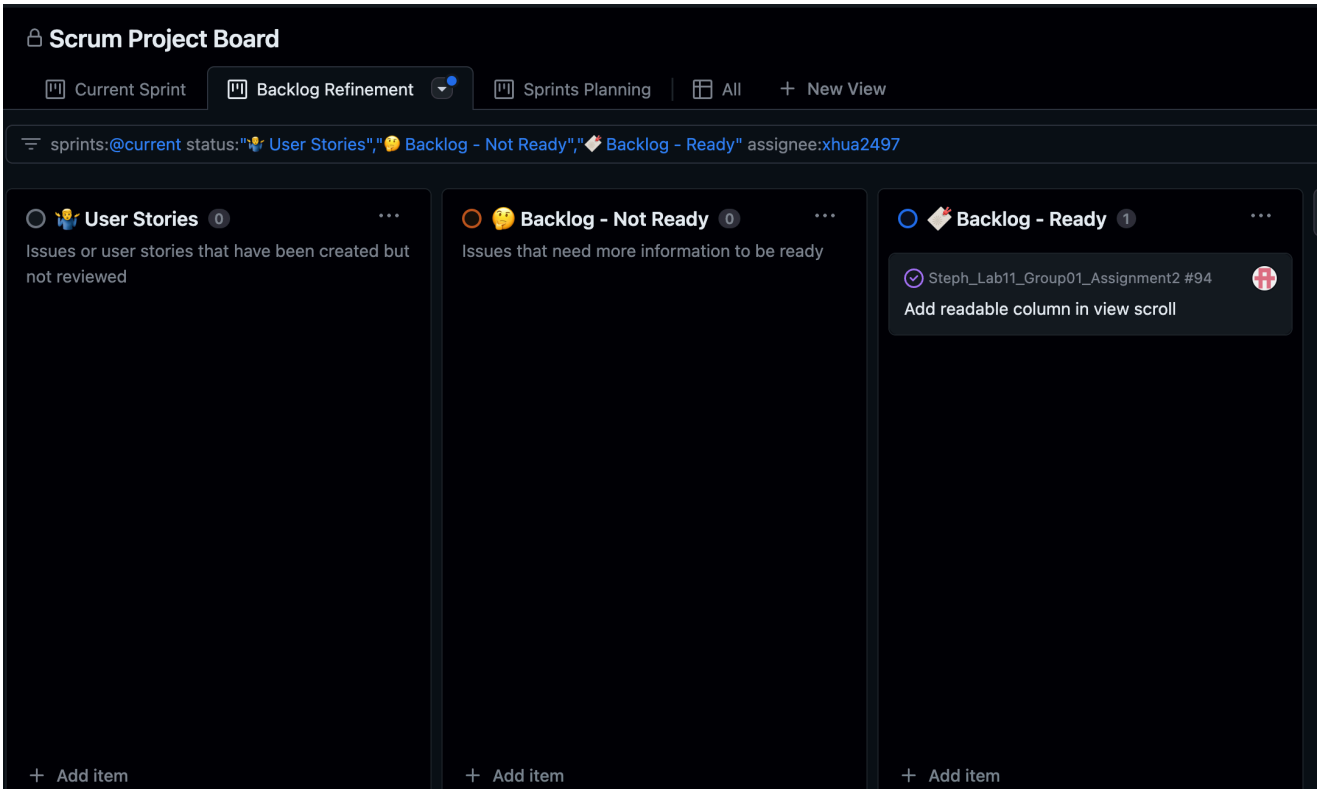
For Sprint 2, the team focused on fixing bugs identified in Sprint 1 (as detected in the Sprint Retrospective meeting), enhancing existing features (previewing, searching and viewing scroll statistics), and implementing new functionalities as requested by stakeholders (the lucky daily scroll).

2.2.2.2 Completeness

As the following figure shown, the correctness tasks discussed through this meeting for sprint 2 are labelled with the assignees and waits to be solved in the well-defined `Backlog – Not Ready` section. Before the group deadline finished, each developer must finish their tasks and ensure its correctness as required.

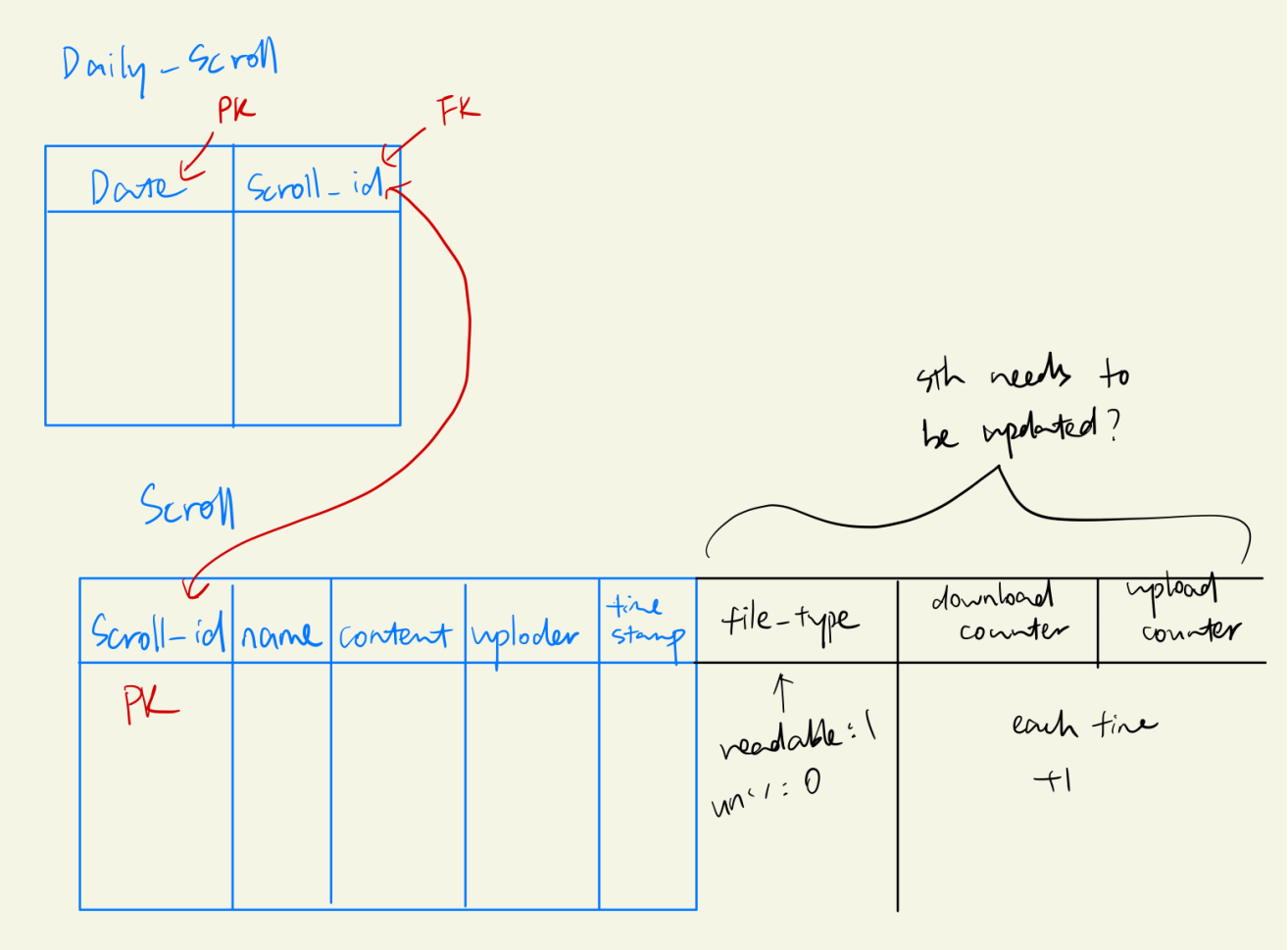


Therefore, with no doubt, those tasks can be moved into `Backlog – Ready` by the product owner and wait to begin with as the second image displayed. The `#94` task has been assigned to `xhua2497` developer, and then he can be able to manage his own progress for this task during sprint 2.



2.2.2.3 Challenges/Issues

In this sprint, the challenges and issues are solved during the start planning meeting. One of the challenges that is the construction of the new daily scroll database, which is discussed by two backend developers (`jkan3790` and `yili2677`). Like the below shown, a lofi-prototype for the new database structure are discussed during the meeting.



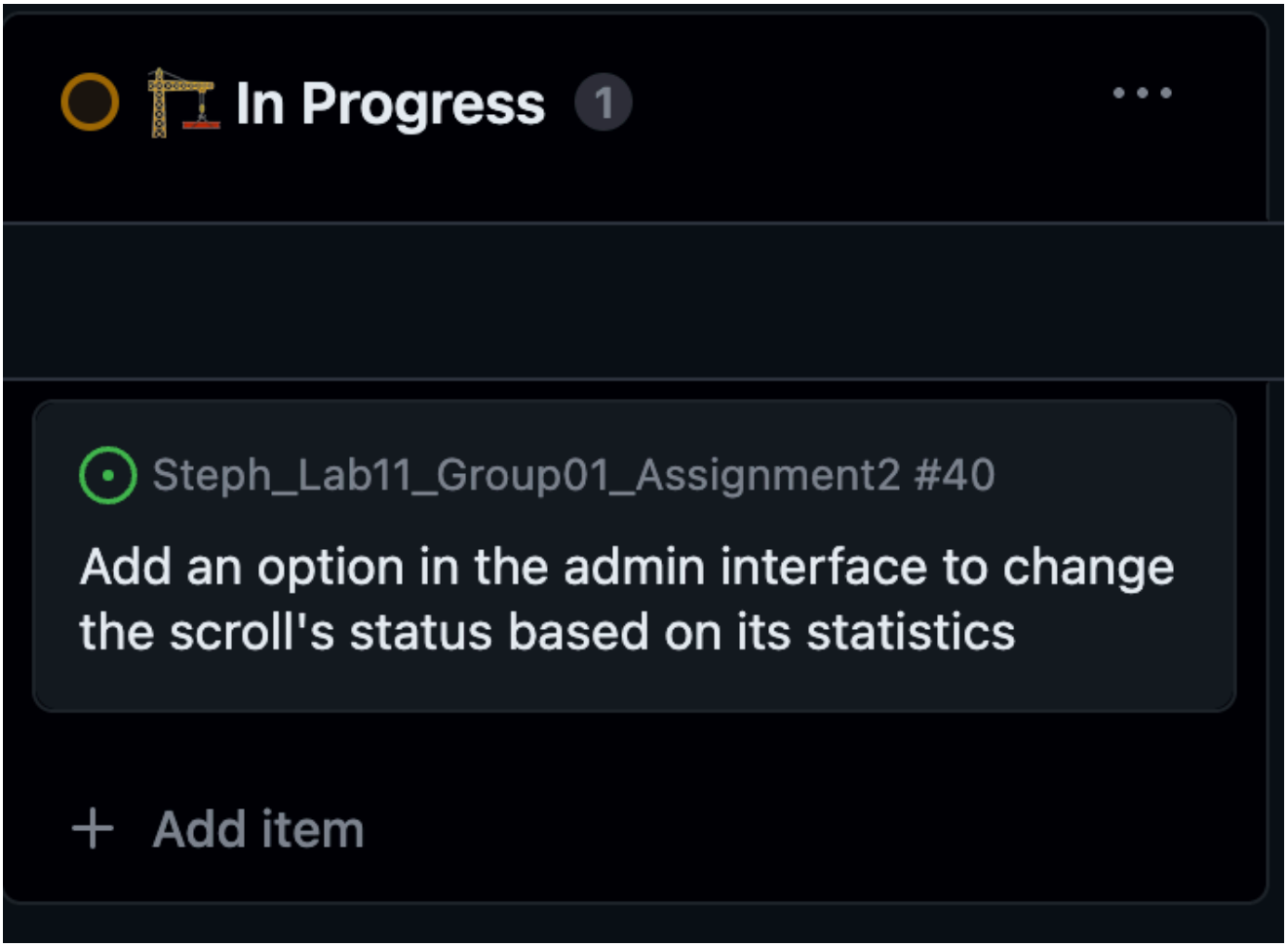
2.2.3 Daily Scrum Meetings

> **Reference**
>
> Please refer to Sprint 1 report (section 2.3.2) to see the detail of daily scrum meeting.

> **Important**
>
> These meetings can be found in the meeting record in the appendix.

2.2.3.1 Relevance

Upto this section, every developer's task should be put inside the `In Progress` section of the sprint 2 (an example task shown below). This clearly demonstrates that developers start to implement the code methodologies to focus on their tasks. After that, the product owner and scrum master should examine on those `In Progress` tasks during the daily scrum meetings.



2.2.3.2 Clarity

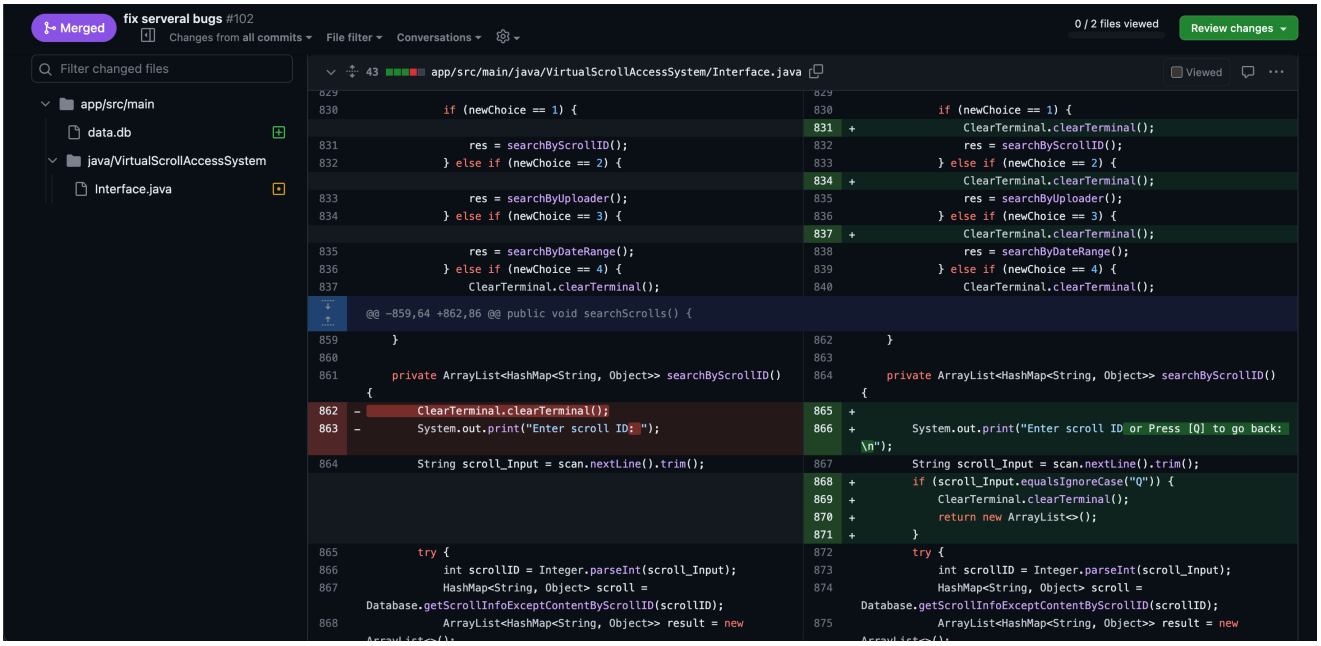
Daily Scrum Meetings are held to maintain alignment and transparency among team members. Similar to the Sprint 1, all team members have agreed to hold three formal online meetings each week, taking place on Zoom at the following times:

- Every Monday night 8:30pm
- Every Tuesday night 8:30pm
- Every Sunday night 8:30pm

For these meetings, the scrum master needs to inspect on:

- Runnable Demo or explanation on developer methodologies
- Questions faced (any developer could answer if they know how to solve)
- Developer idea inspiration (check with all group members to consider the reasonable)
- Suggestions (e.g., error fix way & refactoring)

2.2.3.3 Challenges/Issues



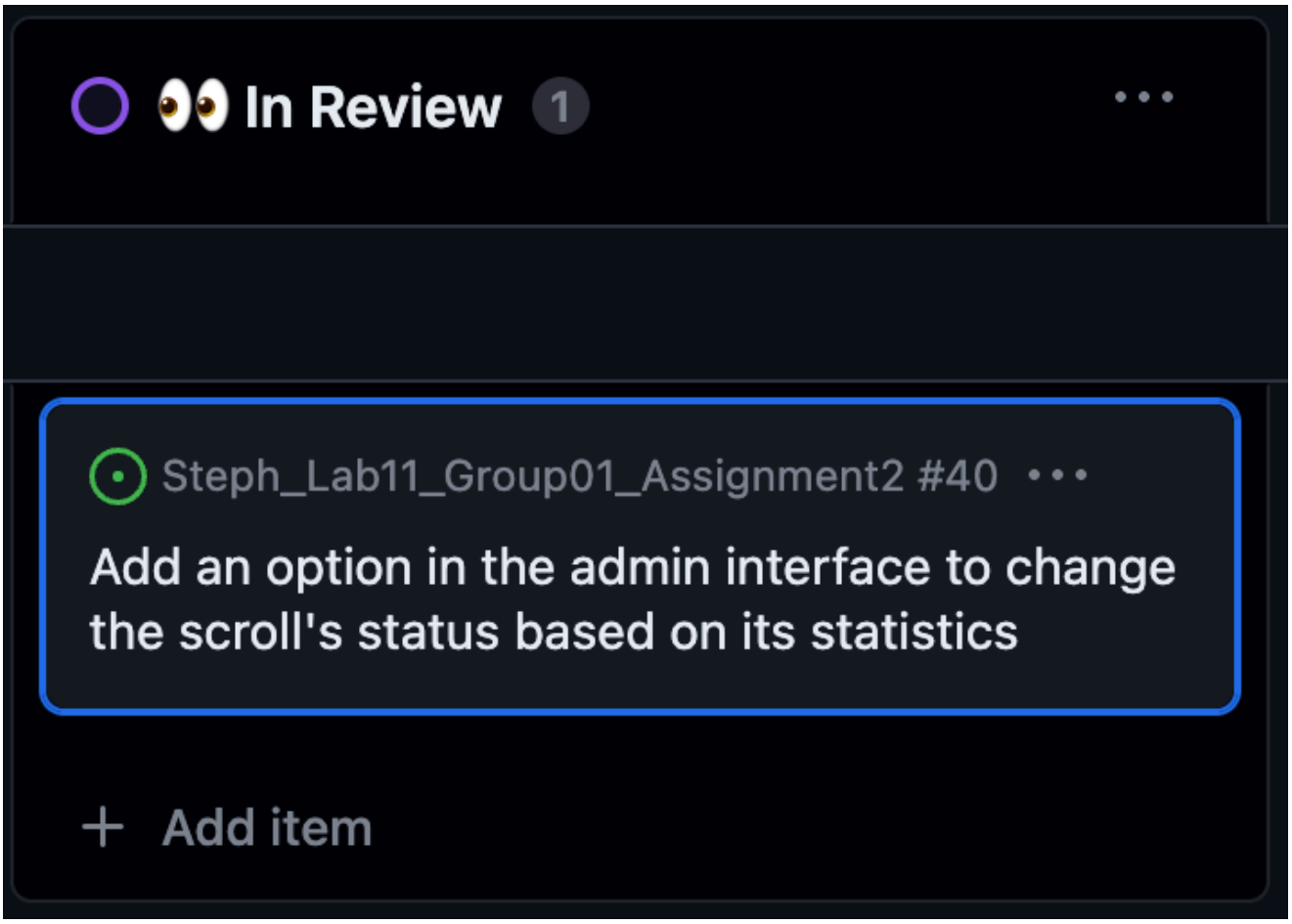
2.2.4 Sprint Review Meetings

2.2.4.1 Sensibility

The Sprint review meeting happens at the end of each sprint (Thursday in the tutorial) and provide an opportunity for the team to showcase completed work to stakeholders. During this meeting, the team presents the increment and demonstrates its functionality. The team demonstrates the product increment, including new features and bug fixes, and gathers feedback.

In our recent Sprint Review Meeting, the tutor, acting as our stakeholder, provided several key pieces of feedback. These insights are crucial for refining our application and enhancing the overall user experience. The feedback received includes:

- **Bug Detected about Lucky Scroll of the Day**: A bug was identified when accessing a deleted "Lucky Scroll of the Day." Currently, if the daily scroll has been deleted, the application does not automatically fetch a new scroll; instead, it throws an exception. This issue needs to be resolved so that the application can dynamically select a new scroll if the previously featured one is unavailable.
- **Support for More File Types in Previewing**: At present, the application only supports previewing `.txt` files. However, the stakeholder noted that in real-world scenarios, users may want to preview various file types, such as `.py`, `.java`, `.c`, and others. To meet this requirement, the application should be extended to support a broader range of file extensions for previewing, making it more versatile and user-friendly.
- **User Experience When Previewing Scroll**: The application currently displays a maximum of 30 words when previewing scroll content. If the content exceeds this limit, it is cut off without any indication, which can confuse users. To improve the user experience, the application should clearly indicate whether the previewed content is complete or provide an ellipsis ("...") or "continued" message when the content is truncated.
- **Scroll Previewing Time Limit**: The stakeholder also requested a new feature that allows the admin user to set a time limit for scroll previews. This time restriction would limit how long a user can preview the scroll content before access is restricted each time. The admin user should have the ability to modify this time limit within the application settings, offering flexibility and control over the previewing feature.



The task should be moved into the `In Review` in this section, when the assigned tasks are solved and add the pull request waiting for the reviewers checking (the product owner and scrum master) no issues, then merged into the `dev` branch. To ensure the correctness of the coding and avoids high risks, a pull request format has been inspired by the scrum master (`jkan3790`) for the developer self checking before merging shown below.

This format significantly deserves in this scrum project, and points a clearly self-checking way for developer instead of directly pull a request without protecting coding correctness. As well as the linked user stories can easily for other developers to quickly catch what features implemented on this commit.

Merged **Enhance searching and previewing features to access scroll database and a daily scroll support access** #92
jkan3790 merged 2 commits into `dev` from `feature-scroll-first-en…` last week

Pull Request Title

📝 Description

This pull request adds backend capabilities for the Scroll and Daily Scroll systems, such as searching scrolls by name and timestamp, retrieving filtered scrolls, extracting scroll statistics, and managing upload/download counters. In addition, a new database for random scroll data is built, and a text file detection property is added to the Scroll database. These enhancements enhance the general functioning and administration of scroll-related data.

🔗 Related User Stories / Tasks

- Closes Need a function to search the scroll by name and timestamp #70 Create a backend method to add a new daily scroll into the Daily_Scroll database #91 Create a backend method to retrieve scrolls that satisfy a specific filtering requirement from the Scroll database #56 Create a backend method to extract scroll ID used to preview #86 Create a backend method to extract the statistics of a scroll #38 Create a backend method to extract upload/download counter by given scroll_id #89 Create a new database to store everyday random scroll info #84 Add attributes upload/download counters for each time relevant methods called into Scroll #87 Implement an upload/download counter for a scroll each time a user upload/download in the Scroll database #39 Add an attribute to detect the text file in Scroll database #85
- Relates to User Stories- As a user, I want to view a random text scroll, so I can see contents of the same scroll logged in on the same date. #81 As a user, I want to view all available scrolls, so I can implement this in any possible ways (list of title names, timestamp, uploader). #15 As an admin, I want to view the number of downloads/uploads in each scroll, so I can manage the scrolls status. #7

✅ Acceptance Criteria

- [x] I have implemented all acceptance criteria for the user story/task.
- [x] I have reviewed the changes with the team in our stand-up meeting.
- [x] I have ensured the changes align with the current sprint goals.
- [x] I have checked the code formatting and ensured it follows project guidelines.
- [x] I have merged the latest main branch locally and resolved all conflicts.
- [x] I have written and run appropriate tests (unit/integration) for this change.

🛠️ Testing Details

- [x] Unit tests passed.
- [] Integration tests passed.
- [x] Manual testing:
  - Test scenario 1
  - Test scenario 2

📈 Impact on Sprint

- Allow front end developers to access the Scroll and Daily_Scroll database.

📸 Screenshots (Optional)

- None required.

⚠️ Potential Risks / Focus Areas

- Potential performance impacts in handling large datasets.

💡 Additional Context

- Relevant documentation updates have been included.

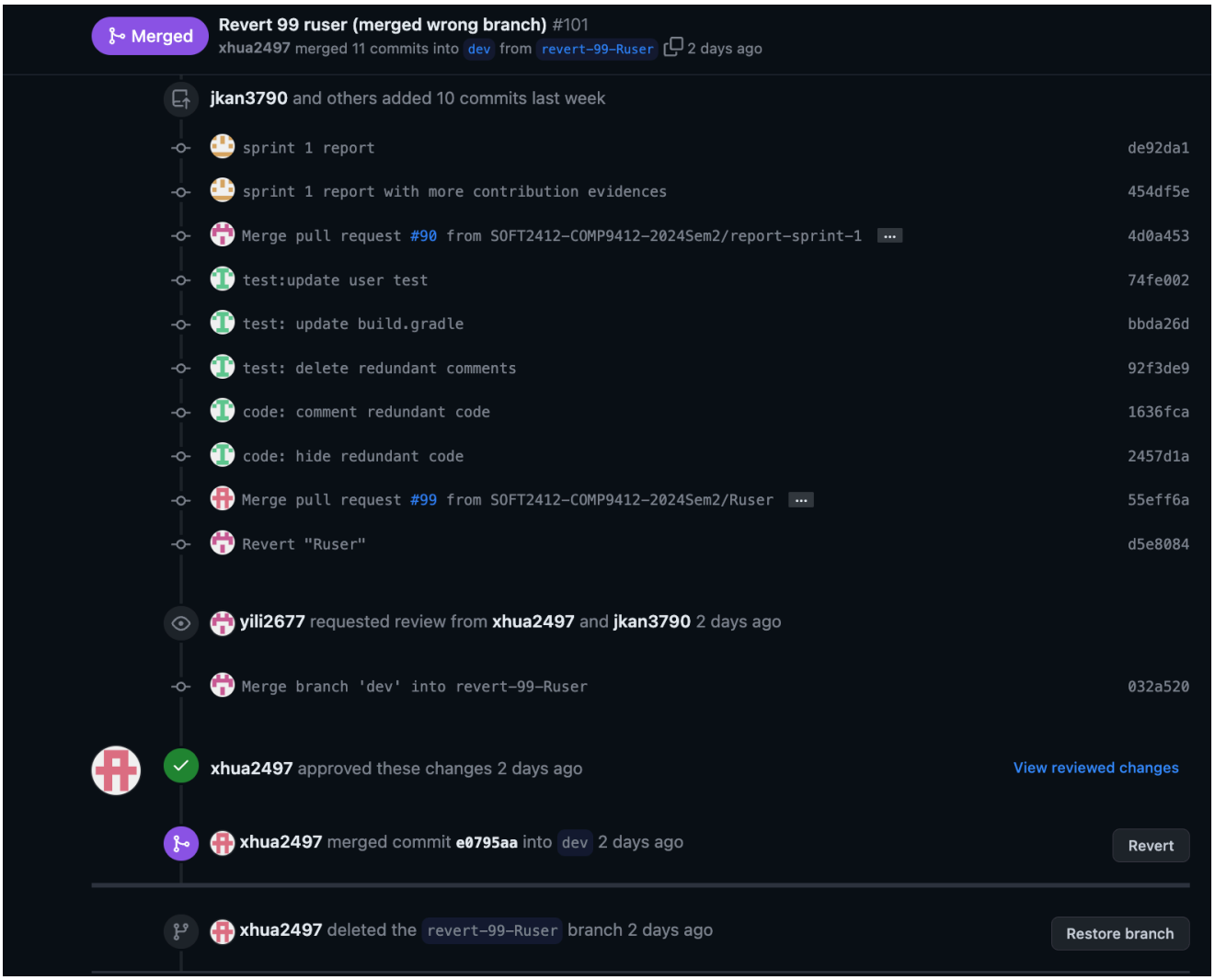
2.2.4.2 Challenges/Issues

Under this section is the high point of the issues happened. As code is pushed in from different developers, the overall code complexity increases. Thus, the scrum master and product owner are more focused on the coding correctness and conflicts solving during this sprint review meetings.

2.2.5 Challenges/Issues and Conflict Resolution

There exists a few issues and conflict in this sprint 2 during merging branches. Excepting the range of small bugs fixing, three significant issues are listed as the following:

1. Merged into the wrong branch



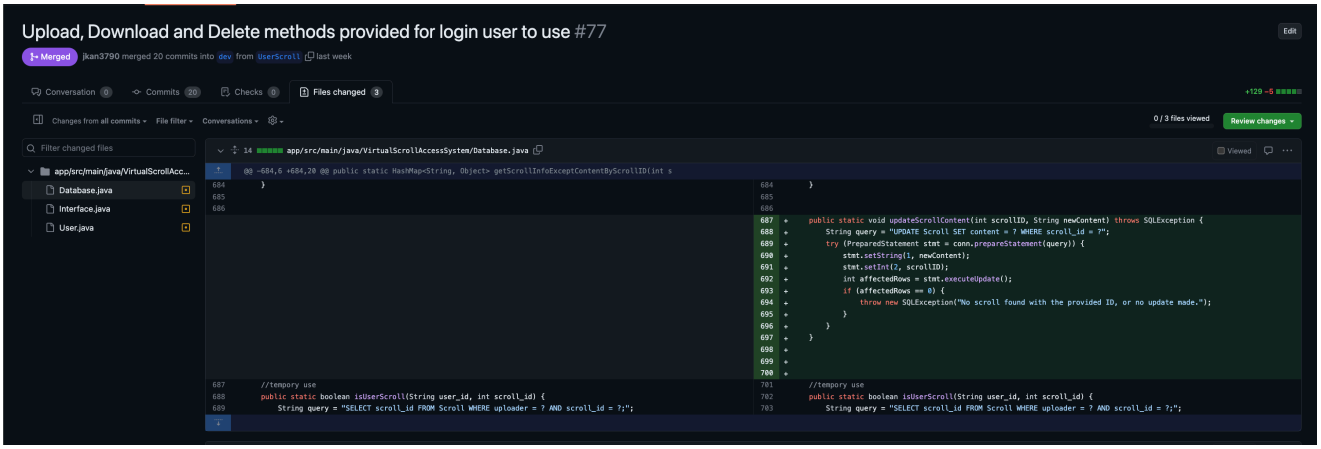
The first issue is merged into the wrong branch. As the scrum project, the target goal is every single developer created their own branches under the `dev` branch. Until all assigned tasks during current sprint has been solved then should merge their `feature-sth` branches into `dev` branch.

However, as above `hjia7380` made a pull request to merge into the `main` branch and approved. This is an issue happened on both requester and reviewer. Therefore, a possible improvement would be to require a double review by both the product owner and the scrum master for a successful merge in future sprints.

2. Code duplication

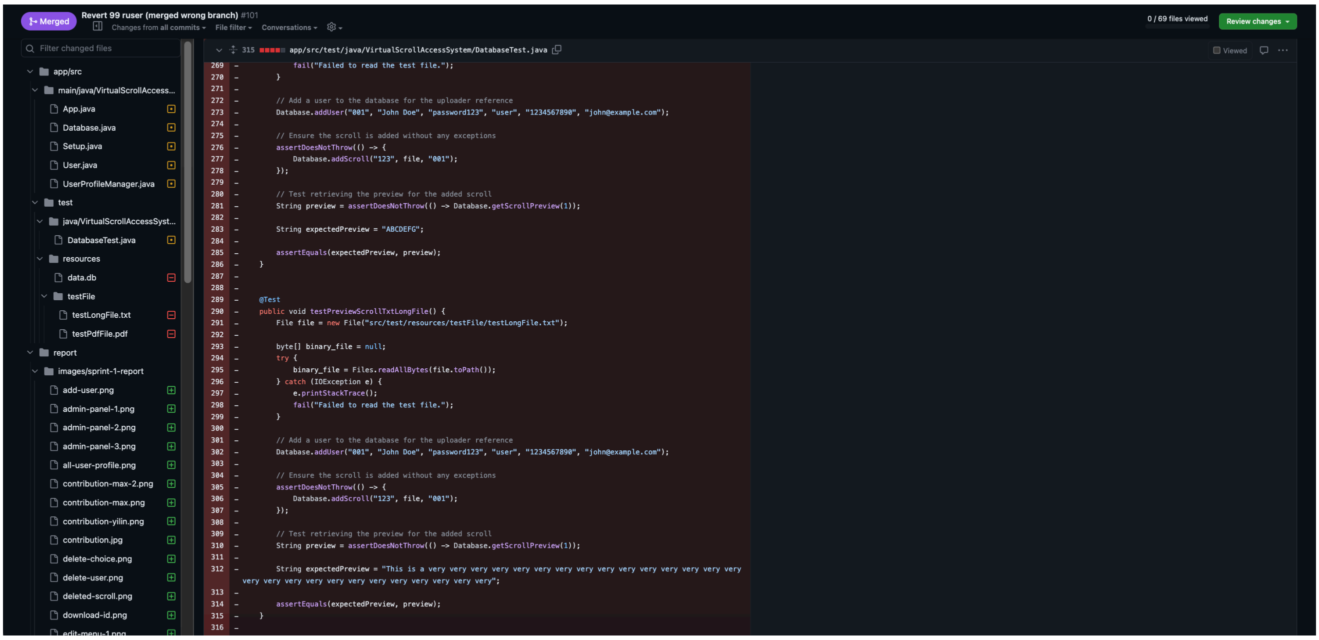
```java
    /**
     * Update scroll content based on the scroll id
     * @param file The path for the scroll content
     * @param scroll_id The id for the scroll used to update content
     * @throws IllegalArgumentException If the {@code scroll_id} is null, empty, or if no scroll is found with the provided {@code scroll_id}.
     * @throws IllegalStateException If there is an error accessing the database or an unexpected error occurs.
     */
    public static void updateScroll(File file, int scroll_id) {
        String query = "UPDATE Scroll SET content = ? WHERE scroll_id = ?;";

        // convert file to binary file format byte[]
        byte[] binary_file = null;
        try {
            binary_file = Files.readAllBytes(file.toPath());
        } catch (IOException e) {
            e.printStackTrace();
        }

        try {
            PreparedStatement stmt = conn.prepareStatement(query);
            stmt.setBytes(1, binary_file);
            stmt.setInt(2, scroll_id);
            int rowsAffected = db.update(conn, stmt);

            if (rowsAffected == 0) {
                throw new IllegalArgumentException("No scroll found with the provided scroll_id.");
            }

        } catch (IllegalArgumentException e) {
            throw e;

        } catch (SQLException e) {
            throw new IllegalStateException("Database access error occurred.", e);

        } catch (Exception e) {
            throw new IllegalStateException("Unexpected error occurred.", e);
        }
    }

```

The second issue is the code duplication. As the first image shown that the `updateScroll` methodology is implemented by `yili2677` already merged into `dev` branch. However, as the second image indicated the pull request on `zhna4143` branch has implemented a duplicated methodology called `updateScrollContent`.



Among the two developers, one belongs to the frontend (`zhna4143`) and the other to the backend (`yili2677`). This problem was attributed to the lack of timely communication between the developers and the lack of clarity in their individual tasks, which led to duplication of code functionality. One of the ways to solve this problem for future sprints is to improve communication between developers in daily scrum meetings and to give some motivation to the scrum master in meetings.

3. Code contribution confusion



The last issue is the code contribution confusion. This happened when reverted the first issue. As the pushed code by `hjia7380` is a couple of dates behind, then previous pushed code were deleting during the code conflict by the reviewer as shown above. Therefore, the code contribution becomes confusion as the data is complemented by the scrum master.

This also an issue due to both developer and reviewer without checking properly. To solve those issues, the reviewer numbers should increase into at least 2 approved to double sure the correctness.

2.3 Estimation and Progress Tracking

2.3.1 Story Point Voting

The story points for backlogs in this sprint 2, totalling 21 points, are voted on by all group members. The following descriptions can be used to explain the tale points based on their range and complexity level.

No	User Story	Backlogs	Story points
1	As a user, I want to preview scrolls, so I can view first before downloading	Design a UI that allow users to preview the content in a scroll before they download it	1
		Handle the return preview information based on different file format in the backend	2
2	As a user, I want to view all available scrolls based on specific search keywords, so I can find the scroll I want easily.	Allow optional query parameters for filtering and sorting scroll information based on user input	2
		Create a backend method to retrieve scrolls that satisfy a specific filtering requirement from the Scroll database	3
		Implement a filtering or searching functionality in the main UI for user to view all intended scrolls	2
		Need a function to search the scroll by name and timestamp	2
3	As an admin, I want to view the number of downloads/uploads in each scroll, so I can manage the scrolls status.	Add attributes upload/download counters for each time relevant methods called into Scroll	0.5
		Create a backend method to extract the statistics of a scroll	0.5
		Create a backend method to update upload/download counter by given scroll_id	0.5
		Implement an upload/download counter for a scroll each time a user upload/download in the Scroll database	1
4	As a user, I want to view a random text scroll, so I can see contents of the same scroll logged in on the same date.	Create a backend method to add a new daily scroll into the Daily_Scroll database	2
		Create a daily random scroll UI and display its content	2
		Create a new database to store everyday random scroll info	1
5	As a user, I want to categorise my uploaded scrolls, so that I can sort my scroll effectively.		
6	As a user, I wish to preview the content before I download it, and I can choose whether or not to download it, so that I can download the scroll I want.	Add an attribute to detect the text file in Scroll database	0.5
7	N/A	User email and phone number should be checked with a valid format	1

Each user story has been segmented into several components for detailed utilisation in sprint 2, facilitating a division of labour between frontend and backend developers. Consequently, the total story point range covers a minimal scope without compromising code complexity (Noticed that the not covered user story will be moved into the next sprint if still in need).

SOFT2412 Sprint 2 stories

Backlogs		
Story Title	Average	Estimate
Add an attribute to detect the text file in Scroll database	0.8	½
Add attributes upload/download counters for each time relevant methods called into Scroll	0.5	½
Allow optional query parameters for filtering and sorting scroll information based on user input	2.3	2
Create a backend method to add a new daily scroll into the Daily_Scroll database	2.1	2
Create a backend method to extract the statistics of a scroll	0.5	½
Create a backend method to update upload/download counter by given scroll_id	0.56	½
Create a backend method to retrieve scrolls that satisfy a specific filtering requirement from the Scroll database	3.7	3
Create a daily random scroll UI and display its content	1.9	2
Create a new database to store everyday random scroll info	0.8	1
Design a UI that allow users to preview the content in a scroll before they download it	1	1
Handle the return preview information based on different file format in the backend	2.2	2
Implement a filtering or searching functionality in the main UI for user to view all intended scrolls	1.8	2
Implement an upload/download counter for a scroll each time a user upload/download in the Scroll database	1	1
Need a function to search the scroll by name and timestamp	2.3	2
User email and phone number should be checked with a valid format	1	1

2.3.1.1 Preview Related

> **Backlog**
>
> Add an attribute to detect the text file in Scroll database (Estimated: 0.5 pts)

This story involves implementing a minor update that checks whether the file extensions in the Scroll database are.txt files. The code complexity is low since it simply includes a method for checking file ending types and determining whether they are readable (in this example, text files with the.txt file).

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 0.5 points.

> **Backlog**
>
> Design a UI that allow users to preview the content in a scroll before they download it (Estimated: 1 pt)

This story involves implementing a user previewing UI with the first 20 characters of content for all readable files (related to backlog 1) with the given scroll ID. The code complexity is low since it generally requires a new feature on user option extensions.

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 1 point.

> **Backlog**
>
> Handle the return preview information based on different file format in the backend (Estimated: 2 pts)

This story involves implementing the obtaining query to retrieve information and create a watching scroll table for visitors to select previewing content (related to backlog 10). The code complexity is moderate since it generally requires a new feature on SQLite database extensions.

Given the efforts straightforward and manageable risk/effort involved, the estimated story points for this backlog item is 2 points.

2.3.1.2 Scroll Statistics Related

Backlog

Add attributes upload/download counters for each time relevant methods called into Scroll (Estimated: 0.5 pts)

This story involves implementing two attributes for recording the number of upload/download scrolls into the Scroll database. The code complexity is low since it simply inserts attributes based on SQLite usage that are similar to the database creation (implemented in Sprint 1).

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 0.5 points.

Backlog

Create a backend method to extract the statistics of a scroll (Estimated: 0.5 pts)

This story involves implementing the getting query for the number of upload/download scrolls into the Scroll database. The code complexity is low since it simply inserts attributes based on SQLite usage that are similar to the database query (implemented in Sprint 1).

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 0.5 points.

Backlog

Create a backend method to update upload/download counter by given scroll_id (Estimated: 0.5 pts)

This story involves implementing the updating query for the number of upload/download scrolls into the Scroll database. The code complexity is low since it simply updates attributes based on SQLite usage that are similar to the database query (implemented in Sprint 1).

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 0.5 points.

Backlog

Create a backend method to retrieve scrolls that satisfy a specific filtering requirement from the Scroll database (Estimated: 3 pts)

This story involves implementing the searching query based on the user inputs (related to backlog 3) with preferred filtering type selections (date range, uploader, scroll ID) to query scroll information employing for UI display. The code complexity is moderate since it generally requires a new feature on database querying extensions.

Given the efforts straightforward and manageable risk/effort involved, the estimated story points for this backlog item is 3 points.

Backlog

Implement an upload/download counter for a scroll each time a user upload/download in the Scroll database (Estimated: 1 pt)

This story involves implementing at the frontend UI to illustrate counters by called method provided (related to backlog 5). The code complexity is low since it generally requires a new feature on user statistics data extensions.

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 1 point.

2.3.1.3 Filter (Search) Related

Backlog

Allow optional query parameters for filtering and sorting scroll information based on user input (Estimated: 2 pts)

This story involves implementing the part of the interface with three options (date range, uploader, scroll ID) for users to search relevant scrolls based on their inputs. As the searching methods provided by backlog 7, queried sorted scroll table displays on UI. The code complexity is moderate since it generally requires a new feature on user option extensions.

Given the efforts straightforward and manageable risk/effort involved, the estimated story points for this backlog item is 2 points.

Backlog

Implement a filtering or searching functionality in the main UI for user to view all intended scrolls (Estimated: 2 pts)

This story involves implementing a filtering UI with three scroll fields (uploader, scroll ID and date range). This feature calls query methods (related to backlog 3) and generates a sorted scroll table for users to view. The code complexity is moderate since it generally requires a new feature on user option extensions.

Given the efforts straightforward and manageable risk/effort involved, the estimated story points for this backlog item is 2 points.

Backlog

Need a function to search the scroll by name and timestamp (Estimated: 2 pts)

This story involves implementing the searching query based on the user inputs (related to backlog 3) with preferred filtering type selections (date range and username) to query scroll information employing for UI display. The code complexity is moderate since it generally requires a new feature on database querying extensions.

Given the efforts straightforward and manageable risk/effort involved, the estimated story points for this backlog item is 2 points.

2.3.1.4 Daily Featured Scroll (Random Scroll) Related

Backlog

Create a backend method to add a new daily scroll into the Daily_Scroll database (Estimated: 2 pts)

Explanation

This story involves implementing the updating query to pick up for a daily random scroll demonstration (related to backlog 8). The code complexity is moderate since it generally requires a new feature on SQLite database extensions.
Given the efforts straightforward and manageable risk/effort involved, the estimated story points for this backlog item is 2 points.

Backlog

Create a daily random scroll UI and display its content (Estimated: 2 pts)

This story involves implementing a random scroll UI and displaying the first 20 characters (connected to backlog 4, and only readable files). Whereas this daily scroll cannot be changed in the actual date of login/logout unless it is deleted from the database, and a new randomly readable file is selected as the daily scroll. The code complexity is moderate since it generally requires a new feature on user option extensions.

Given the efforts straightforward and manageable risk/effort involved, the estimated story points for this backlog item is 2 points.

Backlog

Create a new database to store everyday random scroll info (Estimated: 1 pt)

This story involves implementing a database called Daily_Scroll creation with two attributes (date and scroll ID referenced to the Scroll database) for a daily random scroll demonstration (related to backlog 8). The code complexity is low since it generally requires a new feature on SQLite database extensions.

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 1 point.

2.3.1.5 Other Bug Fixes

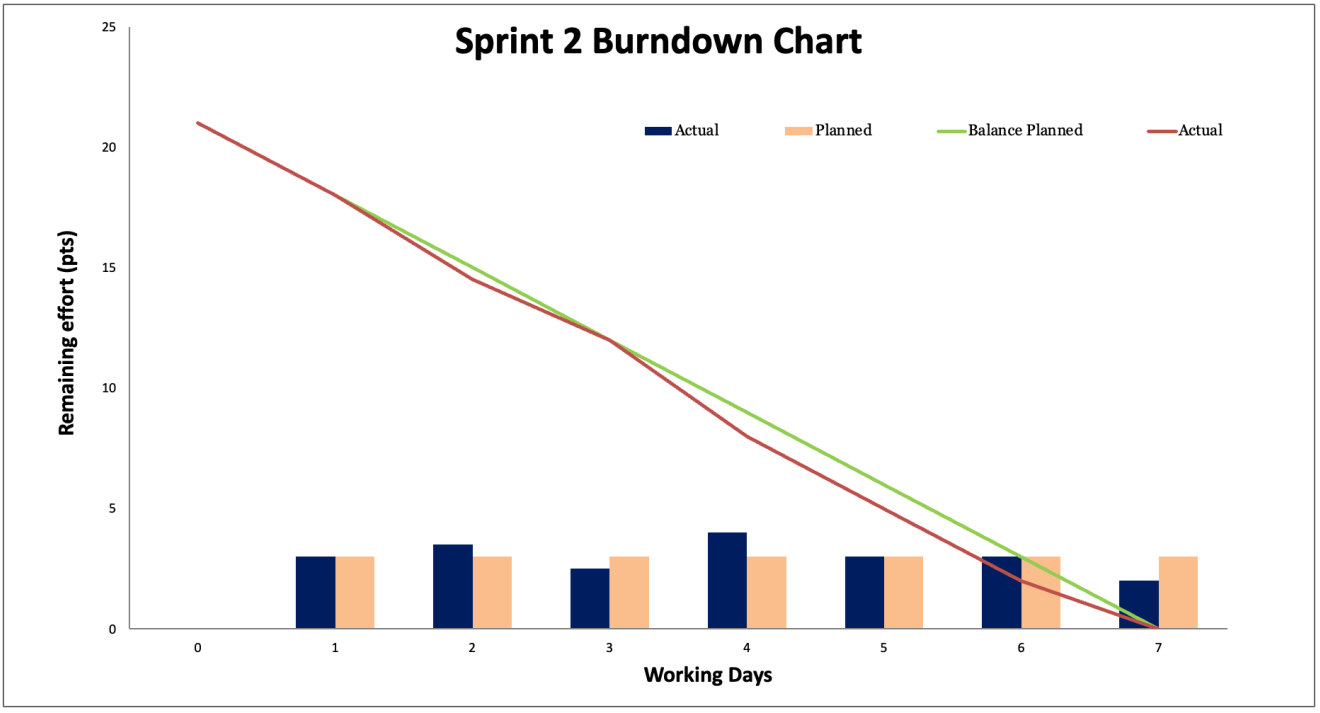
Backlog

User email and phone number should be checked with a valid format (Estimated: 1 pt)

This story involves implementing small modifications to the user email and phone number format (like containing the `@` sign and in 10 digits for Australian phone number format). The code complexity is low since it generally requires a new feature on user information extensions.

Given the simplicity and minimal risk/effort involved, the estimated story points for this backlog item is 1 point.

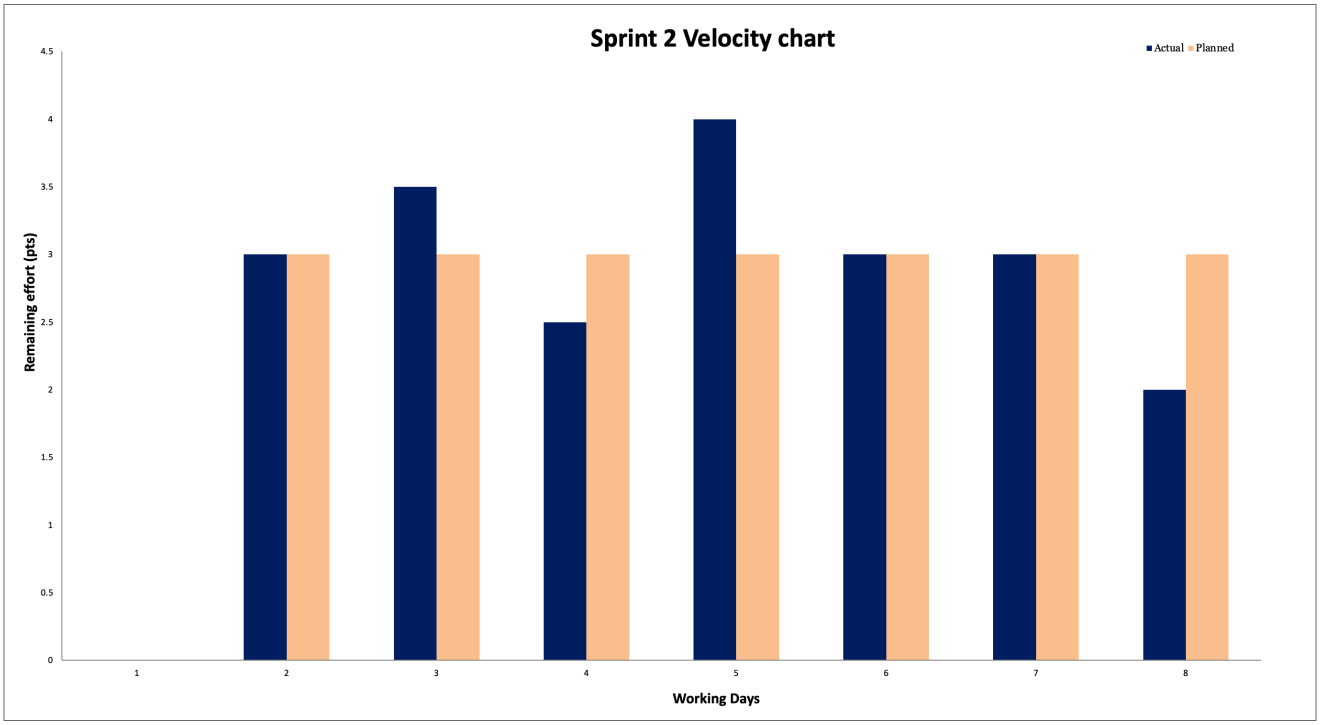
2.3.2 Burndown Chart



Day	Burned down		Balance		Done Today
	Planned	Actual	Planned	Actual	
0			21	21	N/A
1	3	3	18	18	3
2	3	3.5	15	14.5	3.5
3	3	2.5	12	12	2.5
4	3	4	9	8	4
5	3	3	6	5	3
6	3	3	3	2	3
7	3	2	0	0	2

The sprint burndown chart indicates the estimated versus actual working days and its efforts in the graphical represented way. As the burndown chart significantly demonstrates the first four days were relatively intensive (compared to the expected workload). As a result, the trend of work in the first half of the period declined more rapidly.

2.3.3 Velocity Chart



And in the velocity chart, you can clearly see that the actual work is more efficient than the expected completion (the actual workload is less than expected at a later stage). This is a good example of how well a scrum project can be accomplished with effective communication within the group.

2.4 User Stories, Backlogs and Review

In the Sprint 2, the team successfully implemented a series of user stories designed to enhance functionality within the Virtual Scroll Access System (VSAS).

2.4.1 Scroll Related

User Story

As a user, I wish to preview the content before I download it, and I can choose whether or not to download it, so that I can download the scroll I want.

Sprint Backlog

To achieve user story, these backlogs are created and assigned to developers:

- Add an attribute to detect the text file in Scroll database.
- Design a UI that allow users to preview the content in a scroll before they download it.
- Implement a backend method to return preview information based on different file format in the backend.

Acceptance Test and Review

We implemented functionality that allows users to preview the content of scrolls before deciding whether to download them. User can select `2` in the main page to access the download page (also `5` for previewing the contents only).

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
VirtualScrollAccessSystem provides the following functions:
[0] Pick a lucky Scroll for Today
[1] View Scrolls
[2] Download Scrolls
[3] Upload Scrolls
[4] Search Scrolls
[5] Preview Scrolls
[6] Update Profile
[7] Edit Scroll
[8] Log Out
[9] Return to Admin Panel
Please select an operation:
2
```

The download page displays a list of scrolls, including information on whether a scroll is readable based on its file type, shown in the `File Type` column. To support this feature, we enhanced the Scroll database by adding an attribute that detects the text file type for each scroll entry. Currently, the system supports `.txt` files as readable formats, allowing users to preview these files. This functionality can be expanded to include more file types in future iterations.

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
scrollName        scrollID      uploadTime      uploader        File Type
-------------------------------------------------------------------------------------
PDF                  2          2024-10-17      admin           unreadable
Trig                 3          2024-10-18      admin           readable
Linear Regression    4          2024-10-18      admin           readable
-------------------------------------------------------------------------------------
Enter the scroll ID to preview or press [Q] to go back:
3
```

When a user selects a scroll with a readable format (e.g. plain text), the interface displays a preview window showing the first 30 words of the content. This allows users to review the content before deciding whether to download the entire scroll.

```
Scroll Preview:
Name: Trig
Uploader: admin
File Type: Text

Preview: trigonometry, the branch of mathematics concerned with specific functions of angles and
their application to calculations. There are six functions of an angle commonly used in trigonome
try. Their names and

Do you want to download this scroll? (Y/N)
```

For scrolls with unreadable formats (e.g., PDF), the interface notifies the user that the content cannot be previewed. Nevertheless, users still have the option to download these scrolls if they wish.

```
Scroll Preview:
Name: PDF
Uploader: admin
File Type: Binary

Preview: The preview of this document is unavailable, please download the scroll to view it.

Do you want to download this scroll? (Y/N)
```

On the backend, we developed a method that processes various file formats and returns the appropriate preview information. If the file type is identified as `1` (readable), the content is extracted and sent to the frontend for display. Otherwise, the backend returns a message indicating that the file type is unsupported for preview.

```
if (fileType == 1) {
    // Convert BLOB to String
    String content = new String(contentBlob);
    // Extract the first 30 words
    String[] words = content.split(regex:"\\s+");
    StringBuilder preview = new StringBuilder();

    for (int i = 0; i < Math.min(words.length, b:30); i++) {
        preview.append(words[i]).append(str:" ");
    }
    return preview.toString().trim();

} else {
    return "The preview of this document is unavailable, please
    download the scroll to view it.";
}
```

This implementation was thoroughly tested to confirm that users can access the scroll list, view content previews for supported file types, and choose to download scrolls based on the available information. The backend successfully detects and processes different file types, ensuring accurate and reliable content previews.

User Story

As a user, I want to view all available scrolls based on specific search keywords so I can find the scroll I want easily.

Sprint Backlog

To achieve user story, these backlogs are created and assigned to developers:

- Implement a filtering or searching functionality in the main UI for user to view all intended scrolls.
- Allow optional query parameters for filtering and sorting scroll information based on user input.
- Implement a filtering or searching functionality in the main UI for user to view all intended scrolls.
- Create a backend method to retrieve scrolls that satisfy a specific filtering requirement from the Scroll database.

✓ Acceptance Test and Review

We implemented a search functionality that allows users to search for scrolls based on provided keywords. Users can access the search menu by selecting option `4` from the main page.

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
VirtualScrollAccessSystem provides the following functions:
[0] Pick a lucky Scroll for Today
[1] View Scrolls
[2] Download Scrolls
[3] Upload Scrolls
[4] Search Scrolls
[5] Preview Scrolls
[6] Update Profile
[7] Edit Scroll
[8] Log Out
[9] Return to Admin Panel
Please select an operation:
4
```

The search menu offers three different criteria for users to filter scrolls:

- **Scroll ID:** Users can search by entering a specific scroll ID.
- **Uploader:** Users can filter scrolls based on the uploader's name.
- **Time Range:** Users can specify a date range to find scrolls uploaded within that period.

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
Search scrolls:
[1] Search by scroll ID
[2] Search by uploader
[3] Search by date range
[4] Return to previous menu

Select search method:
```

If the user searches using the scroll ID, all scrolls matching the entered ID will be displayed. In the example below, `2` was entered, and the scroll with that ID is returned successfully.

```
Scroll ID       Name       Uploader        Upload Time
------------------------------------------------------------
2               PDF        admin           2024-10-17
------------------------------------------------------------

Press [Enter] to continue searching, or enter [Q] to abort.
```

Users can also search based on the uploader's id. In this example, "admin" is entered as the search term, and all scrolls uploaded by "admin" are displayed.

```
Scroll ID       Name       Uploader        Upload Time
------------------------------------------------------------
2               PDF        admin           2024-10-17
3               Trig       admin           2024-10-18
4               Linear Regression admin           2024-10-18
------------------------------------------------------------

Press [Enter] to continue searching, or enter [Q] to abort.
```

The user can search for scrolls uploaded within a specific date range. For instance, searching between `2024-10-15` and `2024-10-17` returns all scrolls uploaded during that period.

```
Scroll ID       Name       Uploader        Upload Time
------------------------------------------------------------
2               PDF        admin           2024-10-17
------------------------------------------------------------

Press [Enter] to continue searching, or enter [Q] to abort.
```

If no scrolls match the specified criteria, the system notifies the user. For example, searching from `2024-01-01` to `2024-01-02` yields no results, as shown below.

```
No scrolls found.

Press [Enter] to continue searching, or enter [Q] to abort.
```

The system includes error handling for invalid input. For example, if a user enters a date range where the end date is before the start date, an error message is displayed, prompting the user to correct the input.

```
End date must be after start date.

Enter end date (YYYY-MM-DD) or Press [Q] to go back:
```

This comprehensive search functionality ensures that users can effectively locate scrolls using various filters, enhancing the efficiency of the application. The backend successfully processes and validates user inputs, providing accurate results or appropriate error messages as needed.

2.4.2 Admin Related

User Story

As an admin, I want to view the number of downloads/uploads in each scroll so I can manage the scrolls status.

Sprint Backlog

To achieve user story, these backlogs are created and assigned to developers:

- Add attributes upload/download counters for each time relevant methods called into Scroll.
- Create a backend method to extract the statistics of a scroll.
- Create a backend method to update upload/download counter by given scroll_id.
- Design an upload/download counter view for admin user.

Acceptance Test and Review

We developed a feature in the admin panel that allows administrators to view scroll statistics. Admins can access the statistics page by selecting option `4` from the admin menu.

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
Welcome to Admin Management
[1] View all users and their profiles
[2] Add a new user
[3] Delete a user
[4] View statistics (downloads/uploads)
[5] Back to Admin Panel
Please select an operation:
4
```

The statistics page provides an overview of all scrolls in the system, along with their respective download and upload counters. This information helps admins monitor the usage and activity of each scroll.

```
Scroll ID  Download times  Upload/Update times
-----------------------------------------------------
3          2               1
2          1               1
4          0               1
-----------------------------------------------------
Enter [Q] to exit current page
```

The download counter increments each time a scroll is downloaded. This is handled by the `updateDownloadCounter` method, which updates the count in real-time whenever a

download action is performed. The screenshot below shows the download counter for a scroll.

```
/**
 * Increment download counter by 1 based on the scroll id
 * @param scroll_id The id for the scroll used to update download_counter
 * @throws IllegalArgumentException If the {@code scroll_id} is null,
 empty, or if no download_counter is found with the provided {@code
 scroll_id}.
 * @throws IllegalStateException If there is an error accessing the
 database or an unexpected error occurs.
 */
public static void updateDownloadCounter(int scroll_id){
    String query = "UPDATE Scroll SET download_counter = download_counter +
    1 WHERE scroll_id = ?;";

    try (PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setInt(parameterIndex:1, scroll_id);
        int affectedRows = stmt.executeUpdate();
        if (affectedRows == 0) {
            throw new SQLException(reason:"No download_counter found with
            the provided ID, or no update made.");
        }
    } catch (SQLException e) {
        throw new RuntimeException(message:"Error updating download_counter
        for the provided ID");
    }
}
```

Similarly, the upload counter is updated whenever a scroll is uploaded or edited successfully. This ensures that the upload activity for each scroll is accurately tracked, giving admins a clear view of the number of uploads associated with each scroll.

```java
/**
 * Increment upload counter by 1 based on the scroll id
 * @param scroll_id The id for the scroll used to update upload_counter
 * @throws IllegalArgumentException If the {@code scroll_id} is null,
 empty, or if no upload_counter is found with the provided {@code
 scroll_id}.
 * @throws IllegalStateException If there is an error accessing the
 database or an unexpected error occurs.
 */
public static void updateUploadCounter(int scroll_id){
    String query = "UPDATE Scroll SET upload_counter = upload_counter + 1
    WHERE scroll_id = ?;";

    try (PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setInt(parameterIndex:1, scroll_id);
        int affectedRows = stmt.executeUpdate();
        if (affectedRows == 0) {
            throw new SQLException(reason:"No upload_counter found with the
            provided ID, or no update made.");
        }
    } catch (SQLException e) {
        throw new RuntimeException(message:"Error updating upload_counter
        for the provided ID");
    }
}
```

This implementation allows admins to efficiently monitor scroll activities, providing valuable insights into the system's usage patterns. The counters update in real-time, ensuring the statistics displayed are accurate and up-to-date.

2.4.3 New Feature

User Story

As a user, I want to view a random text scroll so I can see contents of the same scroll logged in on the same date.

Sprint Backlog

To achieve user story, these backlogs are created and assigned to developers:

- Add an attribute to detect the text file in Scroll database.
- Create a backend method to add a new daily scroll into the Daily_Scroll database.
- Create a daily random scroll UI and display its content.
- Create a new database to store everyday random scroll info.

Acceptance Test and Review

A new feature called "Get a Lucky Scroll of the Day" was developed, allowing users to view a randomly selected scroll each day. This feature is accessible by selecting option `0` from the main menu.

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
VirtualScrollAccessSystem provides the following functions:
[0] Pick a lucky Scroll for Today
[1] View Scrolls
[2] Download Scrolls
[3] Upload Scrolls
[4] Search Scrolls
[5] Preview Scrolls
[6] Update Profile
[7] Edit Scroll
[8] Log Out
[9] Return to Admin Panel
Please select an operation:
0
```

When the user selects this option, the application displays the lucky scroll for the day. The scroll is chosen randomly from the database, and it remains the same for the entire day. In the example below, today's lucky scroll is "Linear Regression."

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
Today's lucky scroll: Linear Regression
------------------------------
Content:

Linear regression an...
------------------------------
Press [Q] to go back to the previous page.
```

Only readable scrolls (e.g. plain text files) are eligible to be selected as the lucky scroll. If the database contains no readable scrolls, the application displays an error message indicating that no valid text scroll is available.

```
An unexpected error occurred while retrieving the daily scroll: No text scrolls available.
--------------------------
Welcome admin
You are login as a admin!
--------------------------
VirtualScrollAccessSystem provides the following functions:
[0] Pick a lucky Scroll for Today
[1] View Scrolls
[2] Download Scrolls
[3] Upload Scrolls
[4] Search Scrolls
[5] Preview Scrolls
[6] Update Profile
[7] Edit Scroll
[8] Log Out
[9] Return to Admin Panel
Please select an operation:
```

The lucky scroll is refreshed each day. A new lucky scroll will be selected.

```
--------------------------
Welcome admin
You are login as a admin!
--------------------------
Today's lucky scroll: Trig
------------------------------
Content:

trigonometry, the br...
------------------------------
Press [Q] to go back to the previous page.
```

This feature adds an element of surprise and engagement for users by offering a new scroll each day, while ensuring that only appropriate (readable) scrolls are selected. The application handles the selection logic effectively, providing informative feedback when no valid scrolls are available.

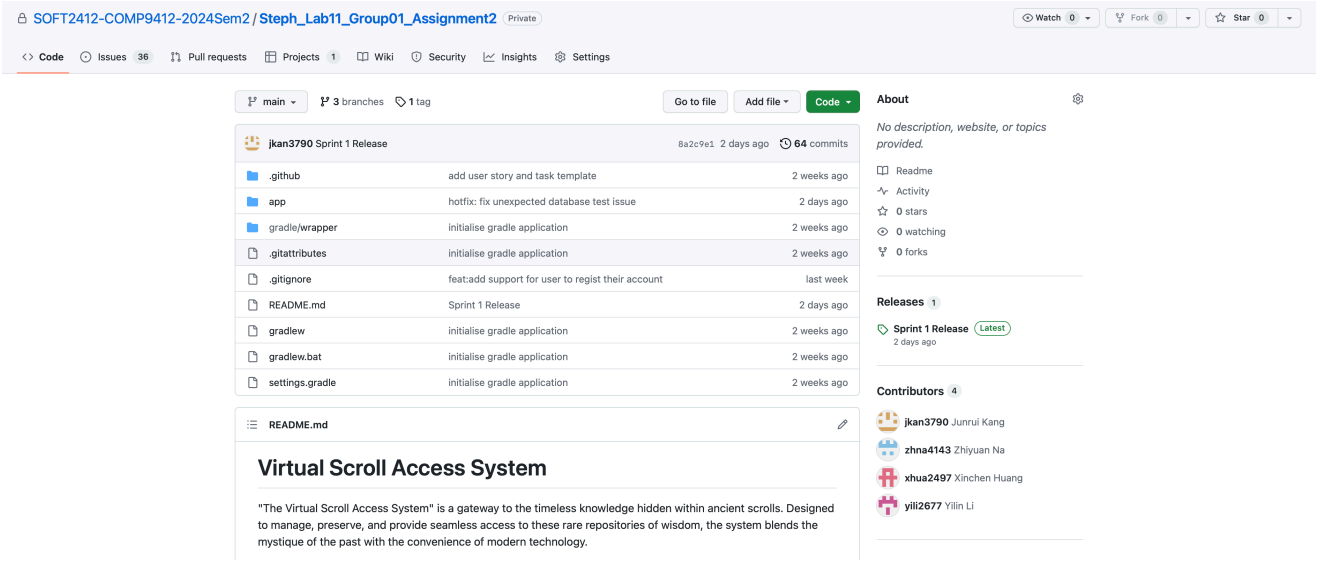
3 Agile Tools

Agile tools are essential for streamlining and automating various aspects of the development process, ensuring efficiency, collaboration, and quality. In this project, we utilise agile tools to manage code, automate builds, and run tests, supporting continuous integration and delivery (CI/CD).

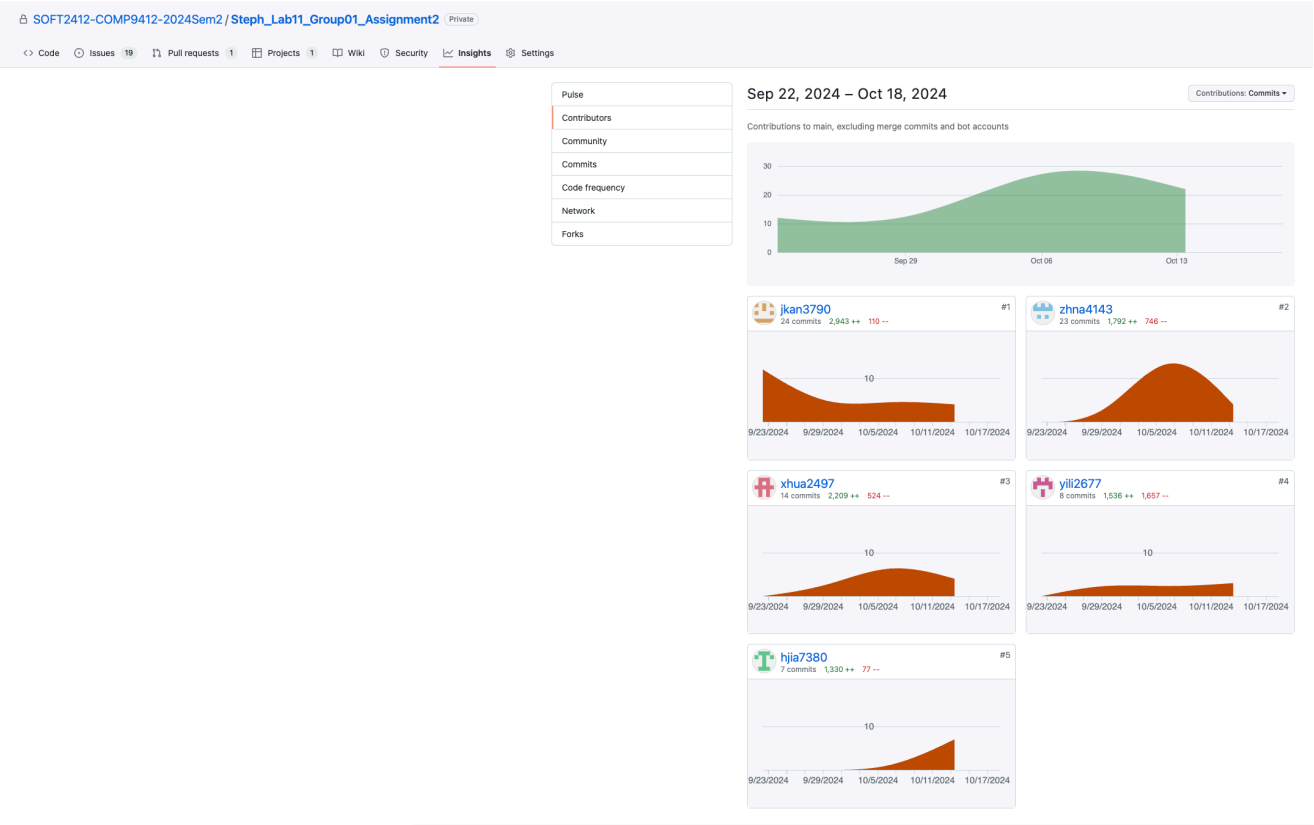
3.1 Code Management: GitHub

> **Reference**
>
> Please refer to Sprint 1 report (section 3.1) to see the basic GitHub usage.

In this project, we use GitHub as our primary tool for code management. The GitHub repository serves to store the code and maintain the revision history of each file, allowing anyone with the necessary permissions to make modifications and fixes.



3.1.1 Github Contribution

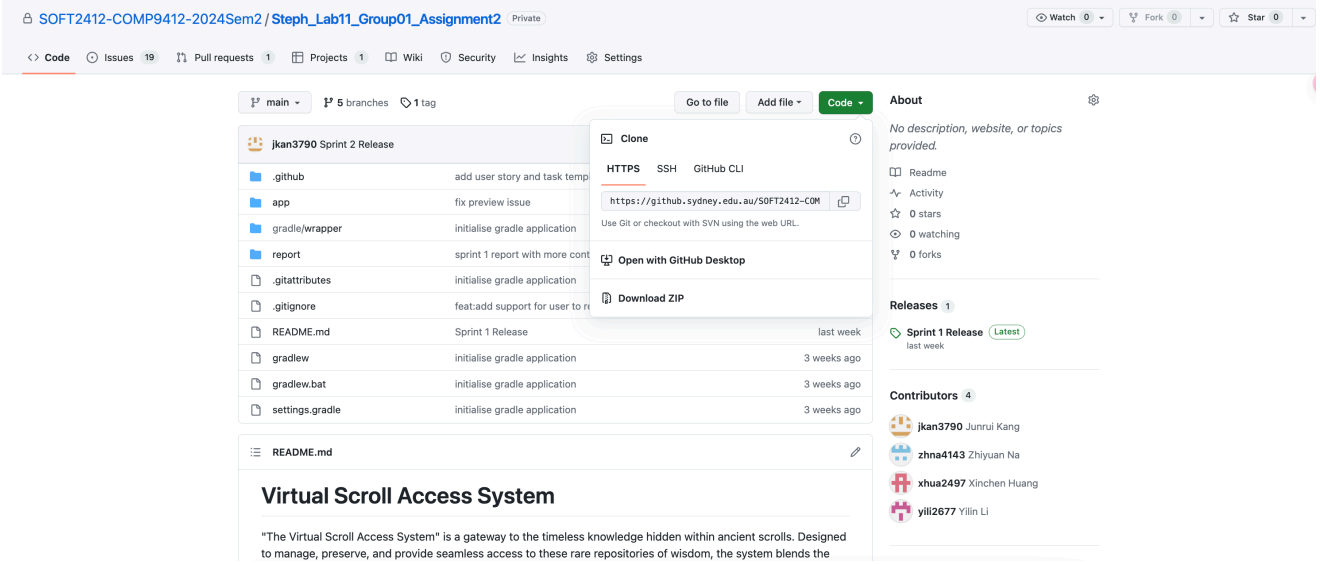


This figure obviously generates the Github contribution ordered by the commits for five developers. It demonstrates that every developer starting from sprint 0 up to current (the ending of sprint 2) all contributes into this repository (`SOFT2412-COMP9412-2024Sem2/Steph_Lab11_Group01_Assignment2`). However, considering the issues happened on `2.2.5` , the current recorded contribution is quite confusion.

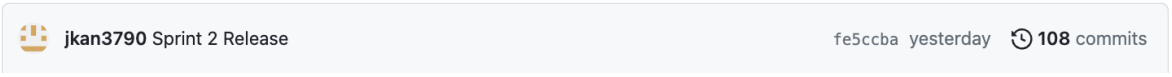
3.1.2 GitHub Setup and Git Commands

3.1.2.1 Repository Setup

- **Repository Creation**: The GitHub repository created by the team is utilised for centralised version control and collaboration. As the image generated, this repository is named `Steph_Lab11_Group01_Assignment2` and is hosted at `SOFT2412-COMP9412-2024Sem2` (strictly adhere to formatting criteria).

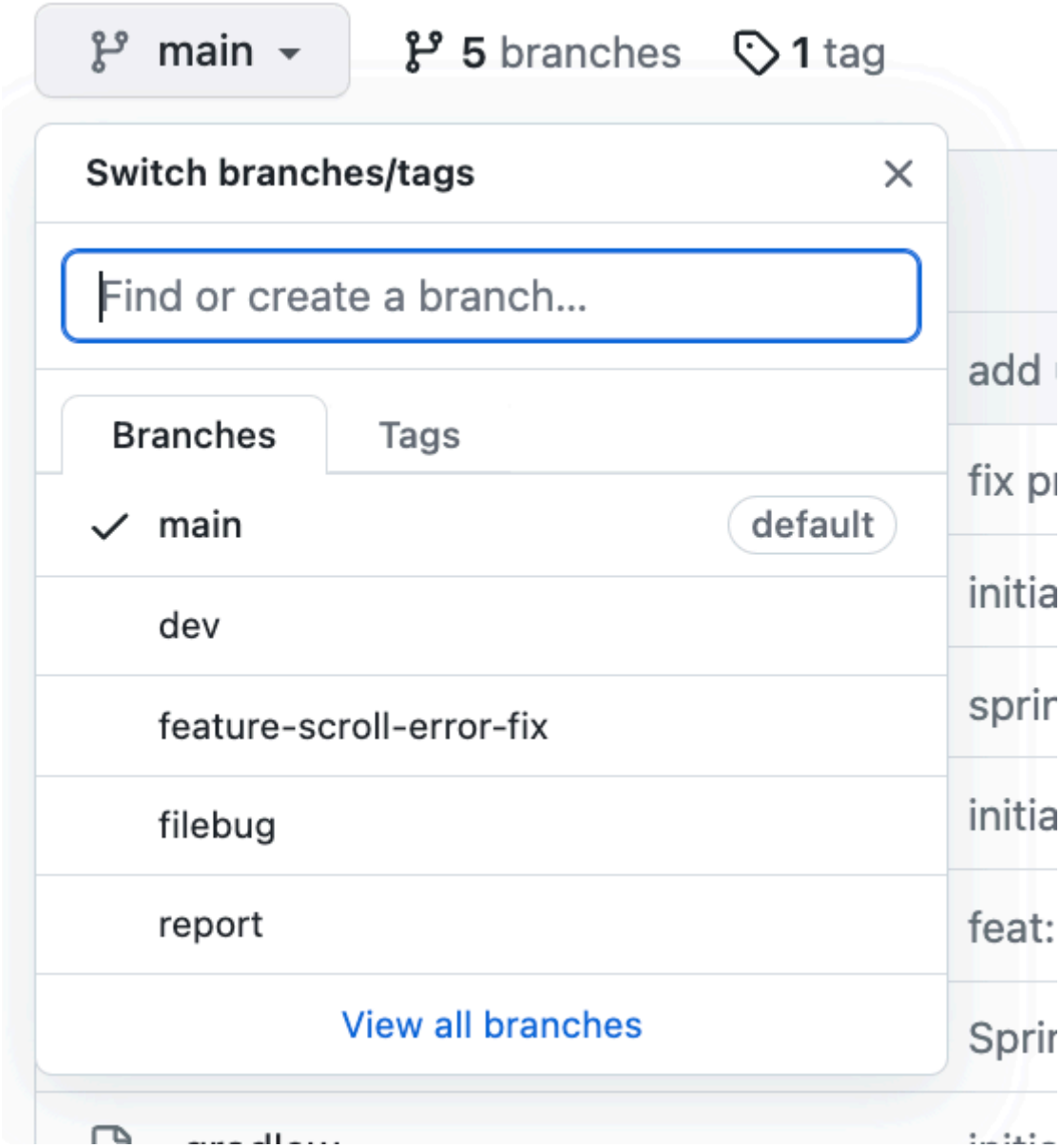


- **Branch Strategy**: The repository has to follow a GitHub branching strategy to ensure smooth collaboration. And the team uses:
  - `main` branch: for summarising each sprint ending works (e.g., commit `Sprint 2 release` into `main` done by `jkan3790`)

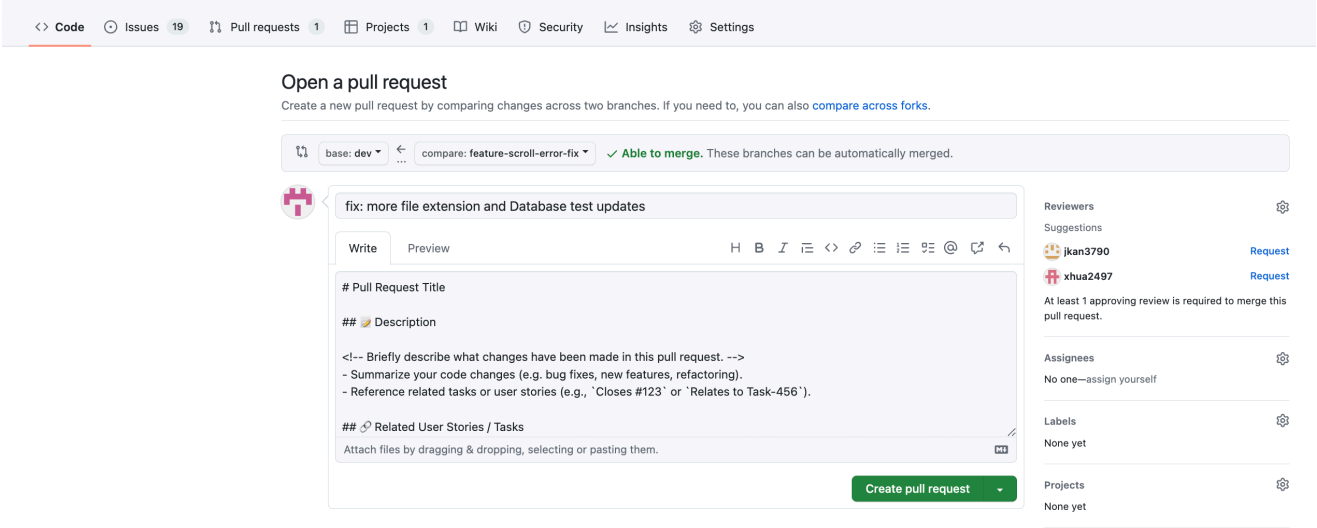
    

  - `dev` branch: for most finished working as each developer finished their `In progress` tasks
  - `feature-x` branch: a sub-branch on `dev`. For every developer's ongoing integration working
  - `report` branch: for sprint report editing

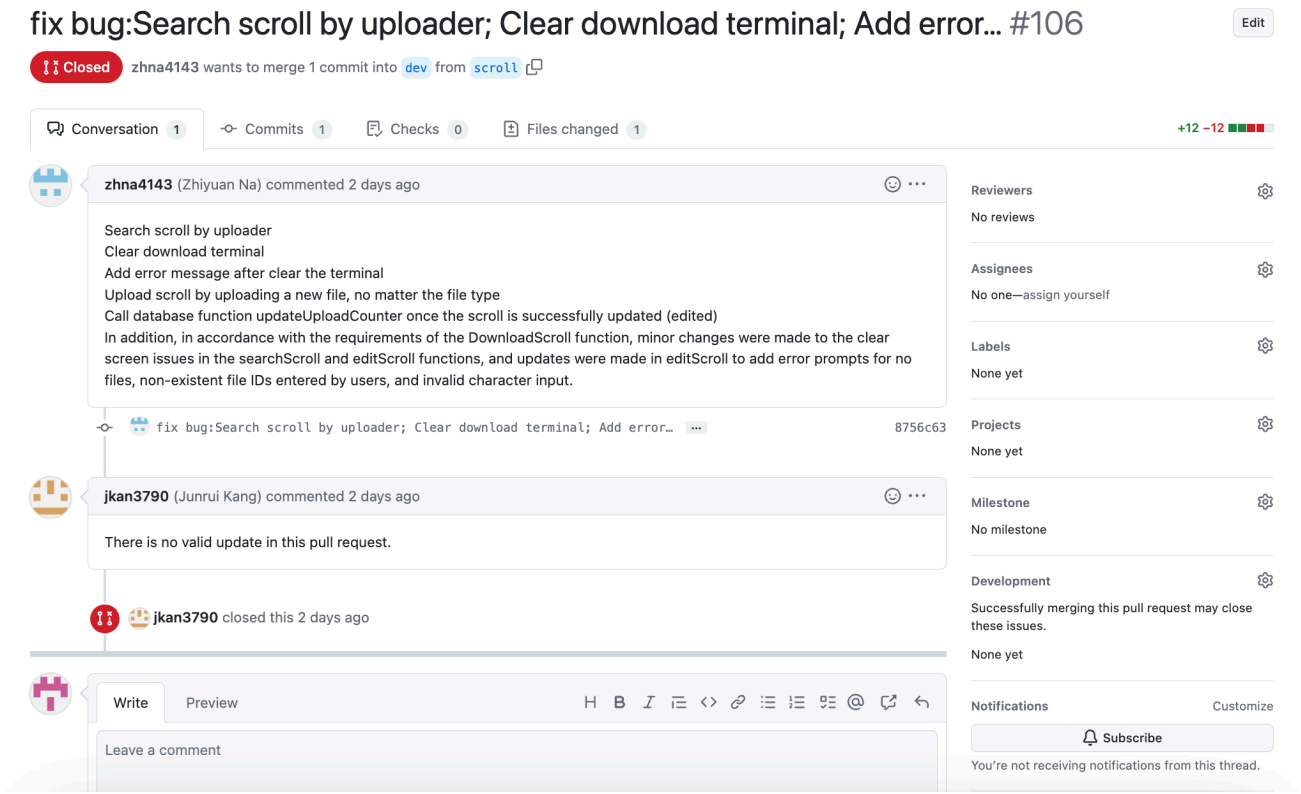
Based on this strategy, the team must act in accordance with and clear division of labor and cooperation. But as the image shown, there were still some unclear branch exists (working for next sprint). This should be mentioned on next sprint to figure out.



- **Access Control**: The team allocated the repository with appropriate access control, avoiding most covered issues happens on the branch merged. As the given figure develops an example for the pull request for `feature-x` branch merging into `dev` branch. Under the reviewers section, the pull request is set `at least 1 approving review is required to merge this pull request`. This enhances the correctness on working and makes the repository more clearly. And all the irregular requests (modify on other developer workspace, error branch, ambiguous work, etc.) will be rejected by reviewers.



Here is an ambiguous work pull request was rejected by `jkan3790`. As viewing the file changed on this pull request, the developer `zhna4143` only added the extra whitespaces on `Interface.java`. The useless work cannot be approved where this is not an available work to merge into the `dev` contribution. Therefore, it should be rejected.



fix bug:Search scroll by uploader; Clear download terminal; Add error... #106

Edit

Closed zhna4143 wants to merge 1 commit into dev from scroll

Conversation 1 · Commits 1 · Checks 0 · Files changed 1 · +12 −12

Changes from all commits · File filter · Conversations · Jump to · 0 / 1 files viewed · Review changes

24 app/src/main/java/VirtualScrollAccessSystem/Interface.java · Viewed

```
@@ -728,7 +728,7 @@ public void downloadScroll() {
728                 System.out.println("\nScroll Preview:");
729                 System.out.println("Name: " + d_scrollName);
730                 System.out.println("Uploader: " + uploader);
731 -
732                 String fileTypeString;
733                 if (fileType == 1) {
734                     fileTypeString = "Text";
@@ -807,26 +807,26 @@ public void searchScrolls() {
807             System.out.println("[4] Return to previous menu");
808             System.out.println();
809             System.out.print("Select search method: ");
810 -
811             String choice = scan.nextLine().trim();
812 -
813             if (choice.equalsIgnoreCase("Q")) {
814                 ClearTerminal.clearTerminal();
815                 userPanel();
816                 return;
817             }
818 -
819             if (!choice.equals("1") && !choice.equals("2") && !choice.equals("3") && !choice.equals("4")) {
820                 System.out.println("\nInvalid input. Please enter a number between 1 and 4.");
821                 scan.nextLine();
822                 ClearTerminal.clearTerminal();
823                 continue;
824             }
825 -
826             int newChoice = Integer.parseInt(choice);
827 -
828             ArrayList<HashMap<String, Object>> res = new ArrayList<>();
829 -
```

```
728                 System.out.println("\nScroll Preview:");
729                 System.out.println("Name: " + d_scrollName);
730                 System.out.println("Uploader: " + uploader);
731 +
732                 String fileTypeString;
733                 if (fileType == 1) {
734                     fileTypeString = "Text";
807             System.out.println("[4] Return to previous menu");
808             System.out.println();
809             System.out.print("Select search method: ");
810 +
811             String choice = scan.nextLine().trim();
812 +
813             if (choice.equalsIgnoreCase("Q")) {
814                 ClearTerminal.clearTerminal();
815                 userPanel();
816                 return;
817             }
818 +
819             if (!choice.equals("1") && !choice.equals("2") && !choice.equals("3") && !choice.equals("4")) {
820                 System.out.println("\nInvalid input. Please enter a number between 1 and 4.");
821                 scan.nextLine();
822                 ClearTerminal.clearTerminal();
823                 continue;
824             }
825 +
826             int newChoice = Integer.parseInt(choice);
827 +
828             ArrayList<HashMap<String, Object>> res = new ArrayList<>();
829 +
```

3.1.2.2 Git commands

There are a few main git commands used to modify, create and pull actions in a GitHub repository. Here are the main roles employed in this project for each command:

- `git add` – move changes from the working directory to the staging area.

```
y2l@aidmxx report % git add images/sprint-2-report/burndown-chart.png images/sprint-2-report/burndown-table.png images/sprint-2-report/story-point-estimated-vs-voting.png images/spr
int-2-report/user-story-relate-backlogs.png images/sprint-2-report/velocity-chart.png sprint2-report.md
```

  This is an example when the developer wants to add the number of images and modified report updates onto the remote repository.

- `git commit` – save staged changes to local repository and creates a new entry in the commit history.

```
y2l@aidmxx report % git commit -m "add: 2.3 insert"
[report 716839b] add: 2.3 insert
 6 files changed, 154 insertions(+)
 create mode 100644 report/images/sprint-2-report/burndown-chart.png
 create mode 100644 report/images/sprint-2-report/burndown-table.png
 create mode 100644 report/images/sprint-2-report/story-point-estimated-vs-voting.png
 create mode 100644 report/images/sprint-2-report/user-story-relate-backlogs.png
 create mode 100644 report/images/sprint-2-report/velocity-chart.png
```

  This is an example when the developer writes the commit with an addition updates on the report 2.3 section. And the changes are moved into the staged status which is ready to upload on the remote GitHub repository.

- `git push` – update local commits (or branches) into remote repository.

```
y2l@aidmxx report % git push
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 10 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 678.68 KiB | 21.21 MiB/s, done.
Total 11 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To github.sydney.edu.au:SOFT2412-COMP9412-2024Sem2/Steph_Lab11_Group01_Assignment2.git
   9b6489f..716839b  report -> report
```

  This is an example when the developer push the commit for the previous report edition. After the command executed, the inserted objects size and items should be automatically calculated and update on the remote GitHub repository `report`.

- `git pull` – bring the local branch up to date with the remote branch by fetching new (similar to `git fetch`).
- `git checkout` – switch between branches or checkout specific commits.

```
y2l@aidmxx report % git checkout dev
A       report/images/sprint-2-report/git-add.png
A       report/images/sprint-2-report/git-commit.png
A       report/images/sprint-2-report/git-push.png
A       report/images/sprint-2-report/git-status.png
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.
```

This is an example when the developer wants to switch from current branch into `dev` branches. `A` stands for "Added" and means that the files have been staged for the next commit.

- `git rebase` – reapply commits from the current branch onto another branch.
- `git merge` – combine changes from one branch to another.
- `git status` – display the current state about uncommitted/untracked changes, and the staging status of files.

```
y2l@aidmxx report % git status
On branch report
Your branch is up to date with 'origin/report'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   images/sprint-2-report/burndown-chart.png
        new file:   images/sprint-2-report/burndown-table.png
        new file:   images/sprint-2-report/story-point-estimated-vs-voting.png
        new file:   images/sprint-2-report/user-story-relate-backlogs.png
        new file:   images/sprint-2-report/velocity-chart.png

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   sprint2-report.md
```

This is an example when the developer checks out in the current state that all untracked modifications and changes compared with the remote repository.

- `git stash` - temporarily saves the local modifications (changes in tracked files, staged files, or both) without committing them to allow the branch switching or pull updates.

```
y2l@aidmxx report % git stash
Saved working directory and index state WIP on report: 716839b add: 2.3 insert
```

This is an example when the developer temporarily saves changes and may want to switch branches or pull updates. `WIP on report` stands for "Work In Progress", and

references to the most recent commit before stashing (which is the commit with message `add: 2.3 insert`).

3.1.3 Git Issues

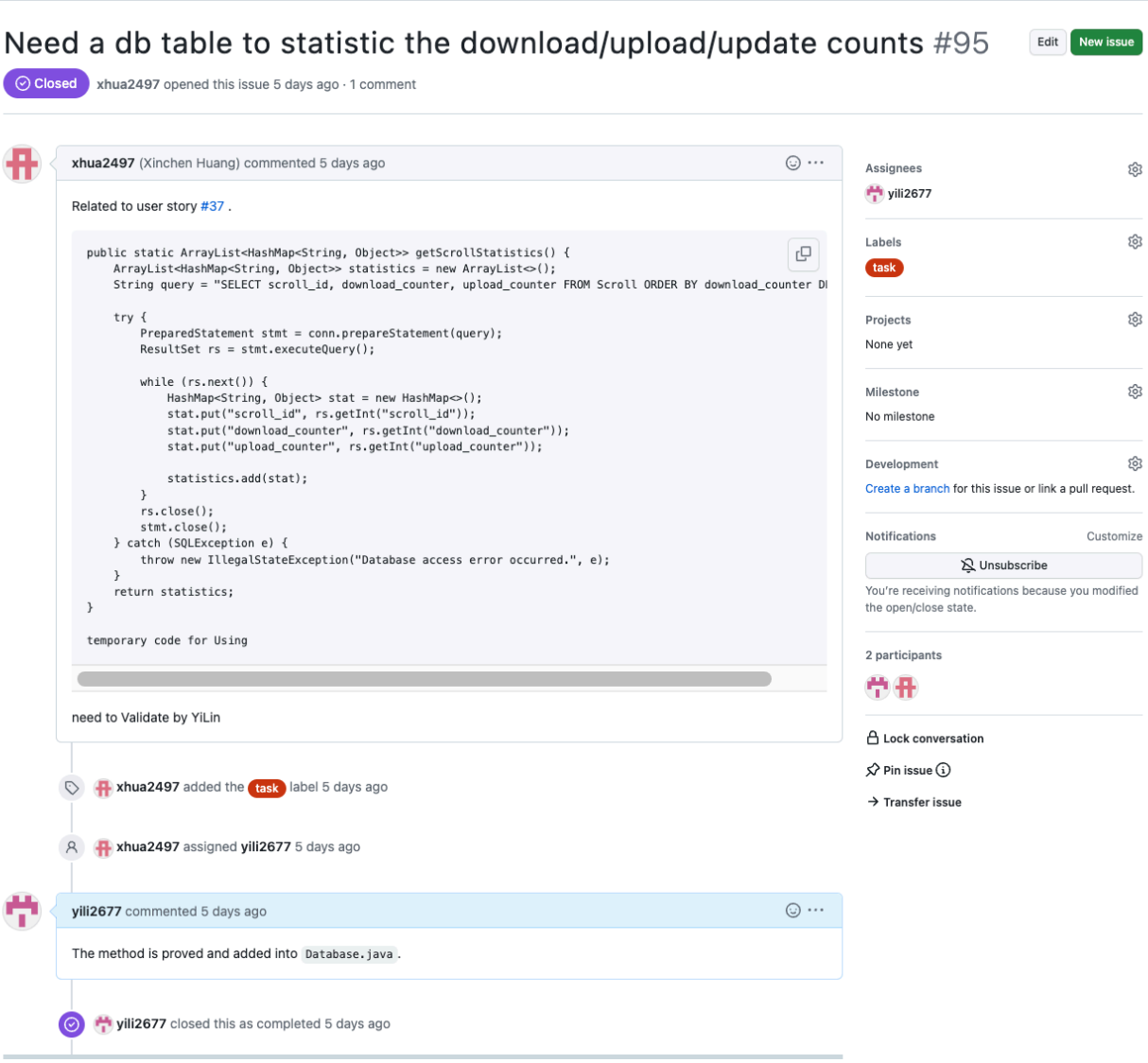
Using GitHub Issues in group repositories can help improve team collaboration, organisation, and communication. It makes the team project's concerns apparent, allowing members to rapidly comprehend the problem and provide appropriate remedies, eliminating a lot of unnecessary trouble (such as duplicity). The following part discusses in detail the challenges encountered by team members while compiling code:

1. Need a db table to statistic the download/upload/update counts

This problem is related to the user story on admin interface has available viewing scroll statistics. As the `Database.java` missed the SQLite method to return those values, a methodology is implemented by `xhua2497` and assigned to `yili2677` for checking validation:

```
public static ArrayList<HashMap<String, Object>> getScrollStatistics() {
  ArrayList<HashMap<String, Object>> statistics = new ArrayList<>();
  String query = "SELECT scroll_id, download_counter, upload_counter FROM
Scroll ORDER BY download_counter DESC;";

  try {
      PreparedStatement stmt = conn.prepareStatement(query);
      ResultSet rs = stmt.executeQuery();

      while (rs.next()) {
          HashMap<String, Object> stat = new HashMap<>();
          stat.put("scroll_id", rs.getInt("scroll_id"));
          stat.put("download_counter", rs.getInt("download_counter"));
          stat.put("upload_counter", rs.getInt("upload_counter"));

          statistics.add(stat);
      }
      rs.close();
      stmt.close();
  } catch (SQLException e) {
      throw new IllegalStateException("Database access error occurred.",
e);
  }
  return statistics;
}

temporary code for Using
```

Fixed: The method is proved by `yili2677` and added into Database.java.



2. Need a function to search the scroll by name and timestamp

This problem is related to the user story that when user wants to view all available scrolls with given information.

Fixed: Implement getScrollInfoByName() and searchScrollByTimeRange() methods in database to solve this issue.

Need a function to search the scroll by name and timestamp #70

Edit | New issue

Closed | xhua2497 opened this issue 2 weeks ago · 1 comment · Fixed by #92

xhua2497 (Xinchen Huang) commented 2 weeks ago

#15

xhua2497 added the task label 2 weeks ago

xhua2497 assigned yili2677 2 weeks ago

jkan3790 added enhancement and removed task labels last week

yili2677 added the task label last week

jkan3790 removed the enhancement label last week

yili2677 added the enhancement label last week

yili2677 commented last week • edited

Implement `getScrollInfoByName()` and `searchScrollByTimeRange()` methods in database to solve this issue

yili2677 mentioned this issue last week

Enhance searching and previewing features to access scroll database and a daily scroll support access #92

8 tasks

Merged

jkan3790 linked a pull request last week that will close this issue

Enhance searching and previewing features to access scroll database and a daily scroll support access #92

8 tasks

Merged

Assignees

yili2677

Labels

enhancement task

Projects

Scrum Project Board

Status: Done +1 more

Milestone

No milestone

Development

Successfully merging a pull request may close this issue.

Enhance searching and previewing features t...
SOFT2412-COMP9412-2024Sem2/Steph_Lab11_Group01_Assignment2

Notifications Customize

Unsubscribe

You're receiving notifications because you were assigned.

3 participants

Lock conversation

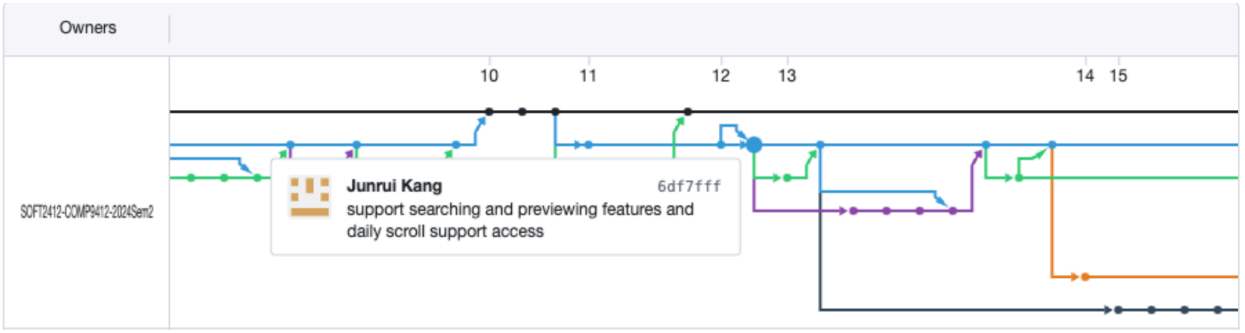
Pin issue

Transfer issue

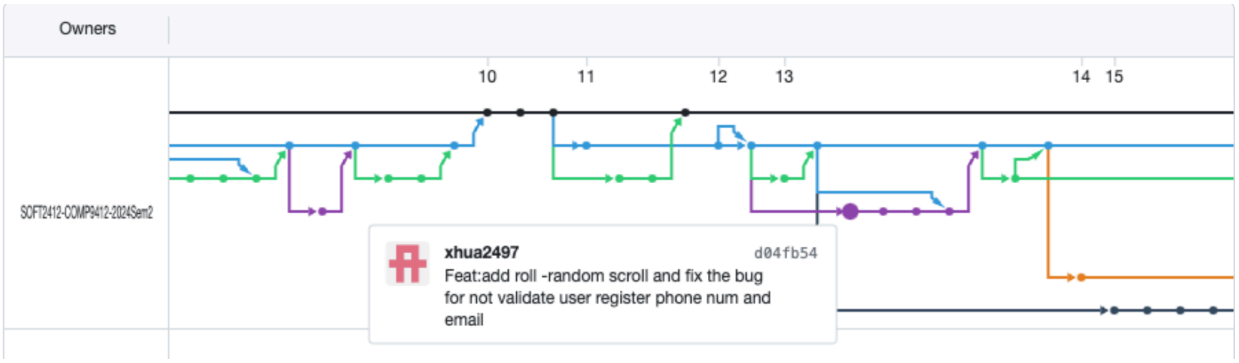
3.1.4 GitHub Network Graph

The team contribution timeline can be intelligibly represented by the Network Graph on GitHub repository in the visual way. This allows the developers to see the history of development, like when the branches diverged and merged, allows to track the repository evolution. Here is the branch starting diverged by each developer:

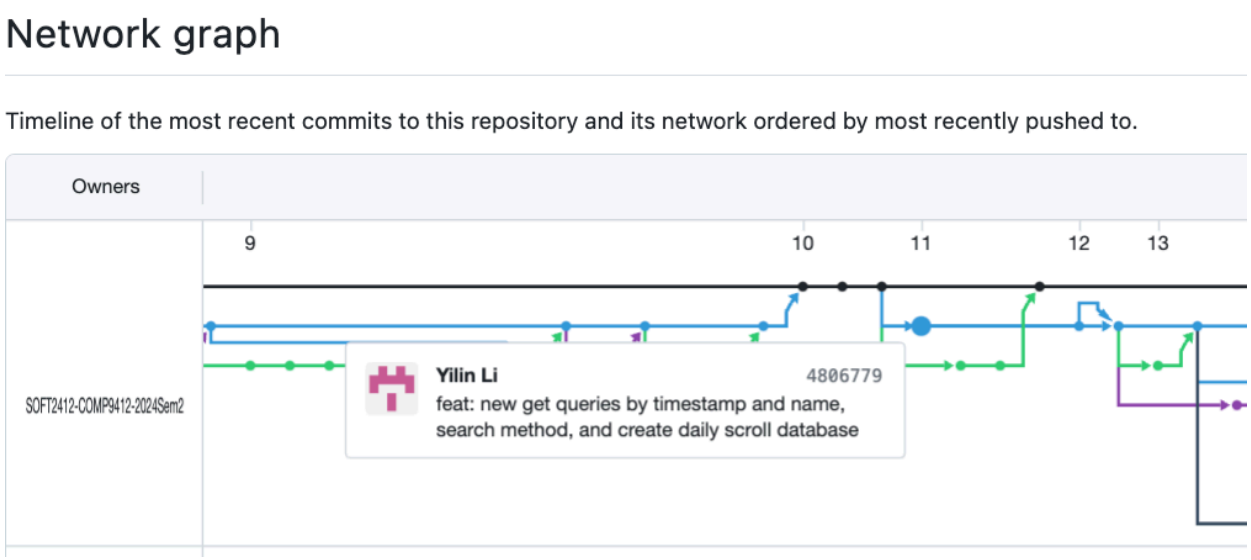
- `jkan3790`:



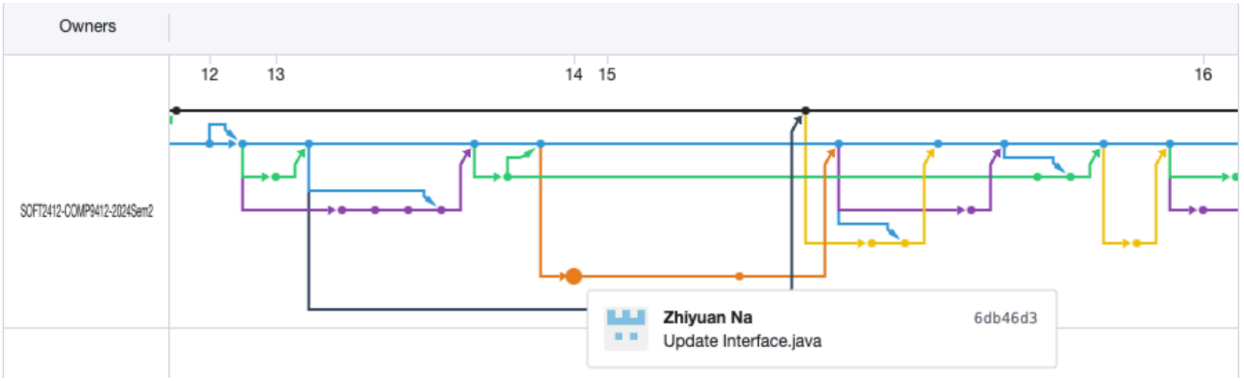
- `xhua2497`:



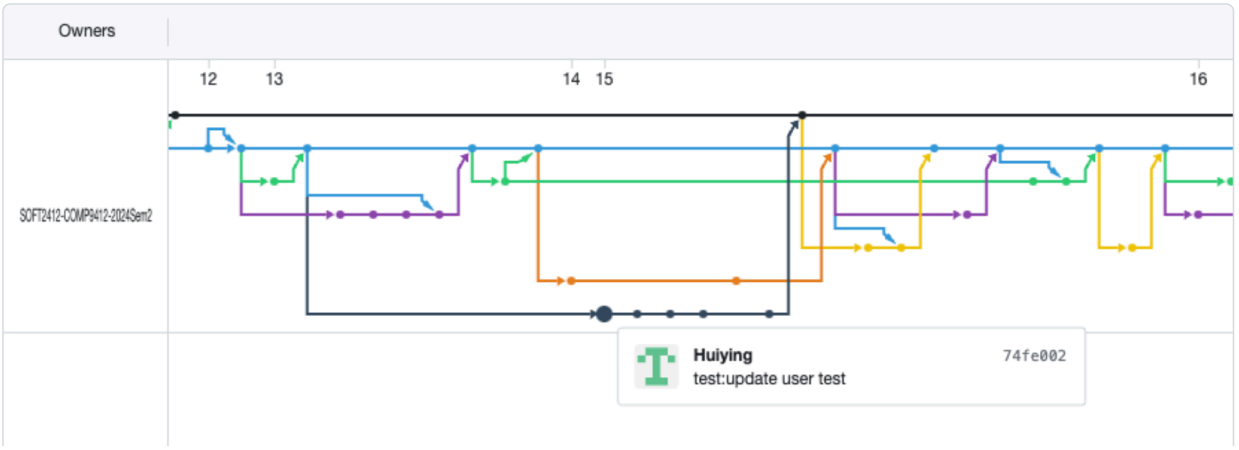
- `yili2677`:



- `zhna4143`:



- `hjia7380`:



These clearly demonstrate the first starting branch on sprint 2 for each developer. Considering the merged pull request is done by other reviewers and is hard to recognise. Then it would not identify in this section.

3.1.5 Pull Requests

The pull request permits developers to propose changes to the `dev` repository and increments their contribution to the project editing. As the previous mentioned, the scrum master provided a formal pull request format for developers to employ. And the list of pull requests for sprint are illustrated below (only shown the approved correct requests in here):

1. `dev` <- `feature-scroll-frist-enhancement` edited by `yili2677` (approved by `jkan3790`)

Enhance searching and previewing features to access scroll database and a daily scroll support access #92

Edit

Merged jkan3790 merged 2 commits into `dev` from `feature-scroll-first-enhancement` last week

Conversation 0 · Commits 2 · Checks 0 · Files changed 3 · +692 −46

yili2677 commented last week

Pull Request Title

Description

This pull request adds backend capabilities for the Scroll and Daily Scroll systems, such as searching scrolls by name and timestamp, retrieving filtered scrolls, extracting scroll statistics, and managing upload/download counters. In addition, a new database for random scroll data is built, and a text file detection property is added to the Scroll database. These enhancements enhance the general functioning and administration of scroll-related data.

Related User Stories / Tasks

- Closes Need a function to search the scroll by name and timestamp #70 Create a backend method to add a new daily scroll into the Daily_Scroll database #91 Create a backend method to retrieve scrolls that satisfy a specific filtering requirement from the Scroll database #56 Create a backend method to extract scroll ID used to preview #86 Create a backend method to extract the statistics of a scroll #38 Create a backend method to extract upload/download counter by given scroll_id #89 Create a new database to store everyday random scroll info #84 Add attributes upload/download counters for each time relevant methods called into Scroll #87 Implement an upload/download counter for a scroll each time a user upload/download in the Scroll database #39 Add an attribute to detect the text file in Scroll database #85
- Relates to User Stories- As a user, I want to view a random text scroll, so I can see contents of the same scroll logged in on the same date. #81 As a user, I want to view all available scrolls, so I can implement this in any possible ways (list of title names, timestamp, uploader). #15 As an admin, I want to view the number of downloads/uploads in each scroll, so I can manage the scrolls status. #7

Acceptance Criteria

- [x] I have implemented all acceptance criteria for the user story/task.
- [x] I have reviewed the changes with the team in our stand-up meeting.
- [x] I have ensured the changes align with the current sprint goals.
- [x] I have checked the code formatting and ensured it follows project guidelines.
- [x] I have merged the latest main branch locally and resolved all conflicts.
- [x] I have written and run appropriate tests (unit/integration) for this change.

Testing Details

- [x] Unit tests passed.
- [] Integration tests passed.
- [x] Manual testing:
  - Test scenario 1
  - Test scenario 2

Impact on Sprint

- Allow front end developers to access the Scroll and Daily_Scroll database.

Screenshots (Optional)

- None required.

Potential Risks / Focus Areas

- Potential performance impacts in handling large datasets.

Additional Context

- Relevant documentation updates have been included.

yili2677 added 2 commits last week

`feat: new get queries by timestamp and name, search method, and creat…` 4806779

`feat & test: time query method rewrite + new features test update` 75713aa

This was linked to issues last week

- Create a backend method to extract the statistics of a scroll #38 — Closed
- Implement an upload/download counter for a scroll each time a user upload/download in the Scroll database #39 — Closed
- Create a backend method to retrieve scrolls that satisfy a specific filtering requirement from the Scroll database #56 — Closed
- Need a function to search the scroll by name and timestamp #70 — Closed
- Create a new database to store everyday random scroll info #84 — Closed
- Add an attribute to detect the text file in Scroll database #85 — Closed
- Create a backend method to extract scroll ID used to preview #86 — Closed
- Add attributes upload/download counters for each time relevant methods called into Scroll #87 — Closed
- Create a backend method to extract upload/download counter by given scroll_id #89 — Closed
- Create a backend method to add a new daily scroll into the Daily_Scroll database #91 — Closed

jkan3790 approved these changes last week View reviewed changes

Reviewers: jkan3790

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues.

- Create a new database to store everyday ran...
- Need a function to search the scroll by name...
- Create a backend method to retrieve scrolls t...
- Implement an upload/download counter for a...
- Create a backend method to extract the stati...

See more

Notifications Customize

Unsubscribe

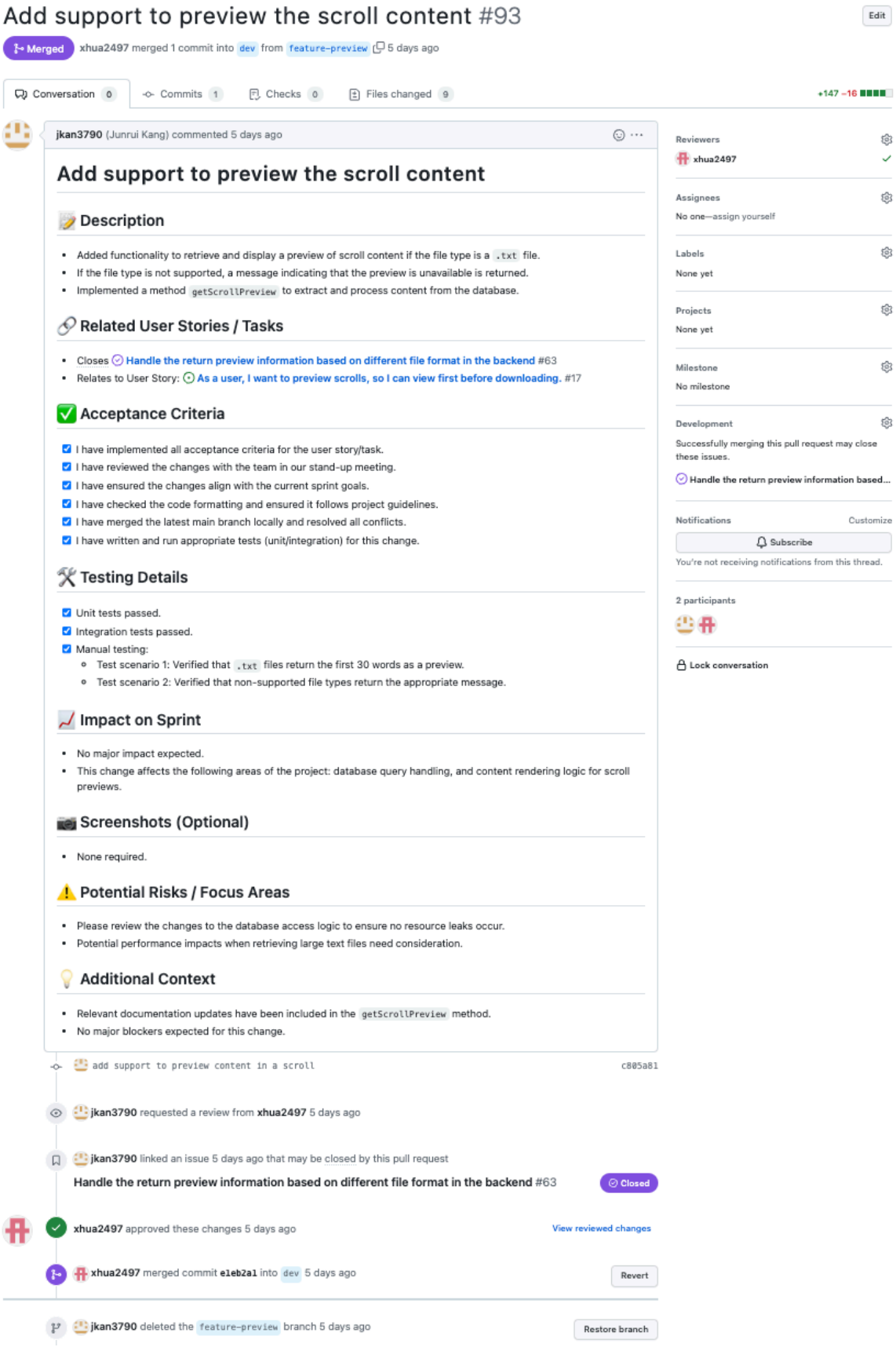
You're receiving notifications because you authored the thread.

2 participants

Lock conversation


jkan3790 merged commit 6df7fff into dev last week
Revert
jkan3790 deleted the feature-scroll-first-enhancement branch last week
Restore branch

2. `dev` <- `feature-preview` edited by `jkan3790` (approved by `xhua2497`)



Add support to preview the scroll content #93

Edit

Merged xhua2497 merged 1 commit into dev from feature-preview 5 days ago

Conversation 0 · Commits 1 · Checks 0 · Files changed 9 · +147 −16

jkan3790 (Junrui Kang) commented 5 days ago

Add support to preview the scroll content

Description

- Added functionality to retrieve and display a preview of scroll content if the file type is a `.txt` file.
- If the file type is not supported, a message indicating that the preview is unavailable is returned.
- Implemented a method `getScrollPreview` to extract and process content from the database.

Related User Stories / Tasks

- Closes Handle the return preview information based on different file format in the backend #63
- Relates to User Story: As a user, I want to preview scrolls, so I can view first before downloading. #17

Acceptance Criteria

- [x] I have implemented all acceptance criteria for the user story/task.
- [x] I have reviewed the changes with the team in our stand-up meeting.
- [x] I have ensured the changes align with the current sprint goals.
- [x] I have checked the code formatting and ensured it follows project guidelines.
- [x] I have merged the latest main branch locally and resolved all conflicts.
- [x] I have written and run appropriate tests (unit/integration) for this change.

Testing Details

- [x] Unit tests passed.
- [x] Integration tests passed.
- [x] Manual testing:
  - Test scenario 1: Verified that `.txt` files return the first 30 words as a preview.
  - Test scenario 2: Verified that non-supported file types return the appropriate message.

Impact on Sprint

- No major impact expected.
- This change affects the following areas of the project: database query handling, and content rendering logic for scroll previews.

Screenshots (Optional)

- None required.

Potential Risks / Focus Areas

- Please review the changes to the database access logic to ensure no resource leaks occur.
- Potential performance impacts when retrieving large text files need consideration.

Additional Context

- Relevant documentation updates have been included in the `getScrollPreview` method.
- No major blockers expected for this change.

Reviewers: xhua2497

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues. Handle the return preview information based...

Notifications: Customize · Subscribe · You're not receiving notifications from this thread.

2 participants

Lock conversation

add support to preview content in a scroll c805a81

jkan3790 requested a review from xhua2497 5 days ago

jkan3790 linked an issue 5 days ago that may be closed by this pull request

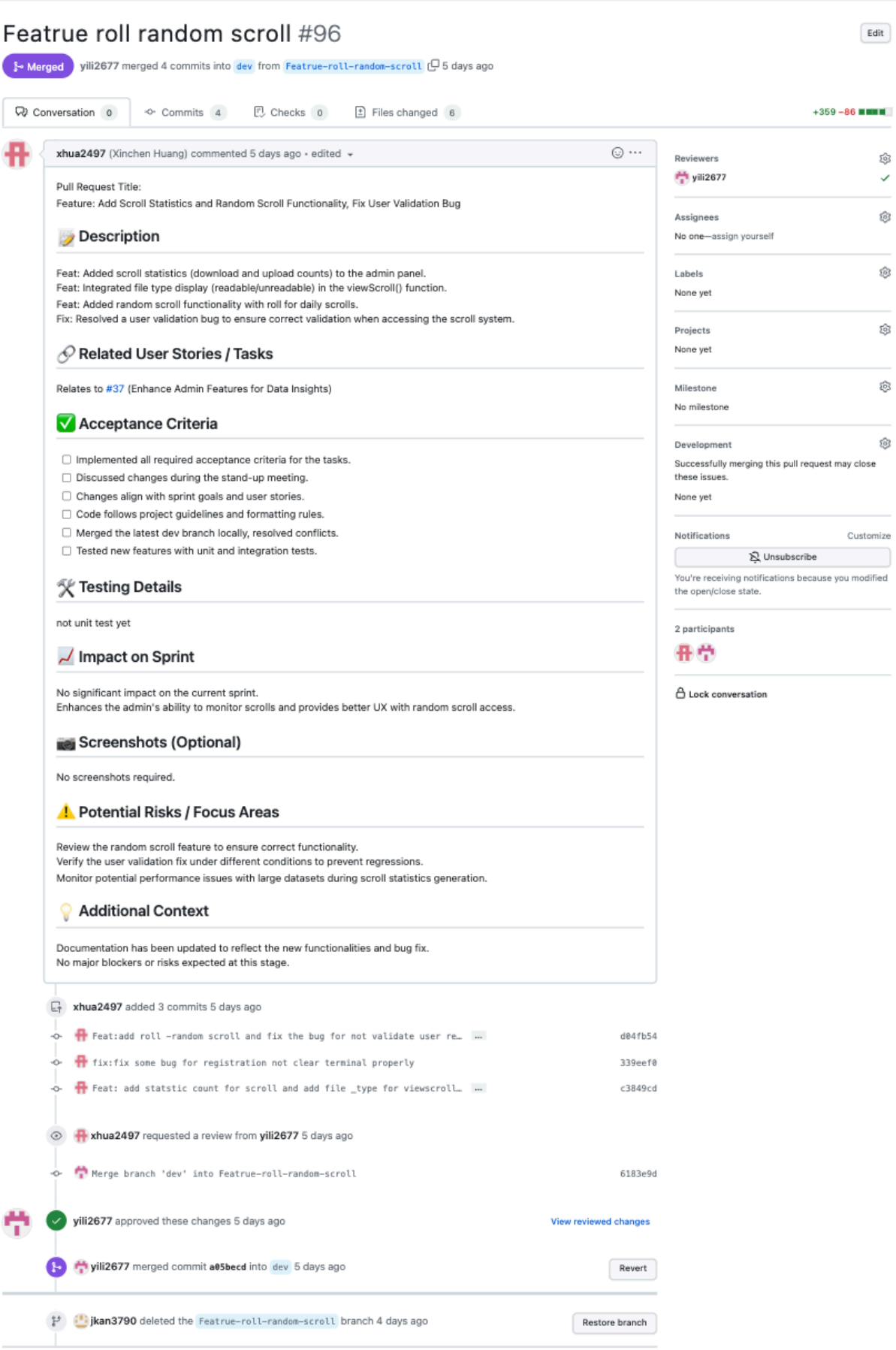
Handle the return preview information based on different file format in the backend #63 Closed

xhua2497 approved these changes 5 days ago View reviewed changes

xhua2497 merged commit e1eb2a1 into dev 5 days ago Revert

jkan3790 deleted the feature-preview branch 5 days ago Restore branch

3. `dev` <- `feature-roll-random-roll` edited by `xhua2497` (approved by `yili2677`)



Featrue roll random scroll #96

Edit

Merged yili2677 merged 4 commits into dev from Featrue-roll-random-scroll 5 days ago

Conversation 0 · Commits 4 · Checks 0 · Files changed 6 · +359 −86

xhua2497 (Xinchen Huang) commented 5 days ago • edited

Pull Request Title:
Feature: Add Scroll Statistics and Random Scroll Functionality, Fix User Validation Bug

Description

Feat: Added scroll statistics (download and upload counts) to the admin panel.
Feat: Integrated file type display (readable/unreadable) in the viewScroll() function.
Feat: Added random scroll functionality with roll for daily scrolls.
Fix: Resolved a user validation bug to ensure correct validation when accessing the scroll system.

Related User Stories / Tasks

Relates to #37 (Enhance Admin Features for Data Insights)

Acceptance Criteria

- [ ] Implemented all required acceptance criteria for the tasks.
- [ ] Discussed changes during the stand-up meeting.
- [ ] Changes align with sprint goals and user stories.
- [ ] Code follows project guidelines and formatting rules.
- [ ] Merged the latest dev branch locally, resolved conflicts.
- [ ] Tested new features with unit and integration tests.

Testing Details

not unit test yet

Impact on Sprint

No significant impact on the current sprint.
Enhances the admin's ability to monitor scrolls and provides better UX with random scroll access.

Screenshots (Optional)

No screenshots required.

Potential Risks / Focus Areas

Review the random scroll feature to ensure correct functionality.
Verify the user validation fix under different conditions to prevent regressions.
Monitor potential performance issues with large datasets during scroll statistics generation.

Additional Context

Documentation has been updated to reflect the new functionalities and bug fix.
No major blockers or risks expected at this stage.

Reviewers: yili2677
Assignees: No one—assign yourself
Labels: None yet
Projects: None yet
Milestone: No milestone
Development: Successfully merging this pull request may close these issues. None yet
Notifications: Customize · Unsubscribe
You're receiving notifications because you modified the open/close state.
2 participants
Lock conversation

xhua2497 added 3 commits 5 days ago

- Feat:add roll -random scroll and fix the bug for not validate user re… d04fb54
- fix:fix some bug for registration not clear terminal properly 339eef0
- Feat: add statstic count for scroll and add file _type for viewscroll… c3849cd

xhua2497 requested a review from yili2677 5 days ago

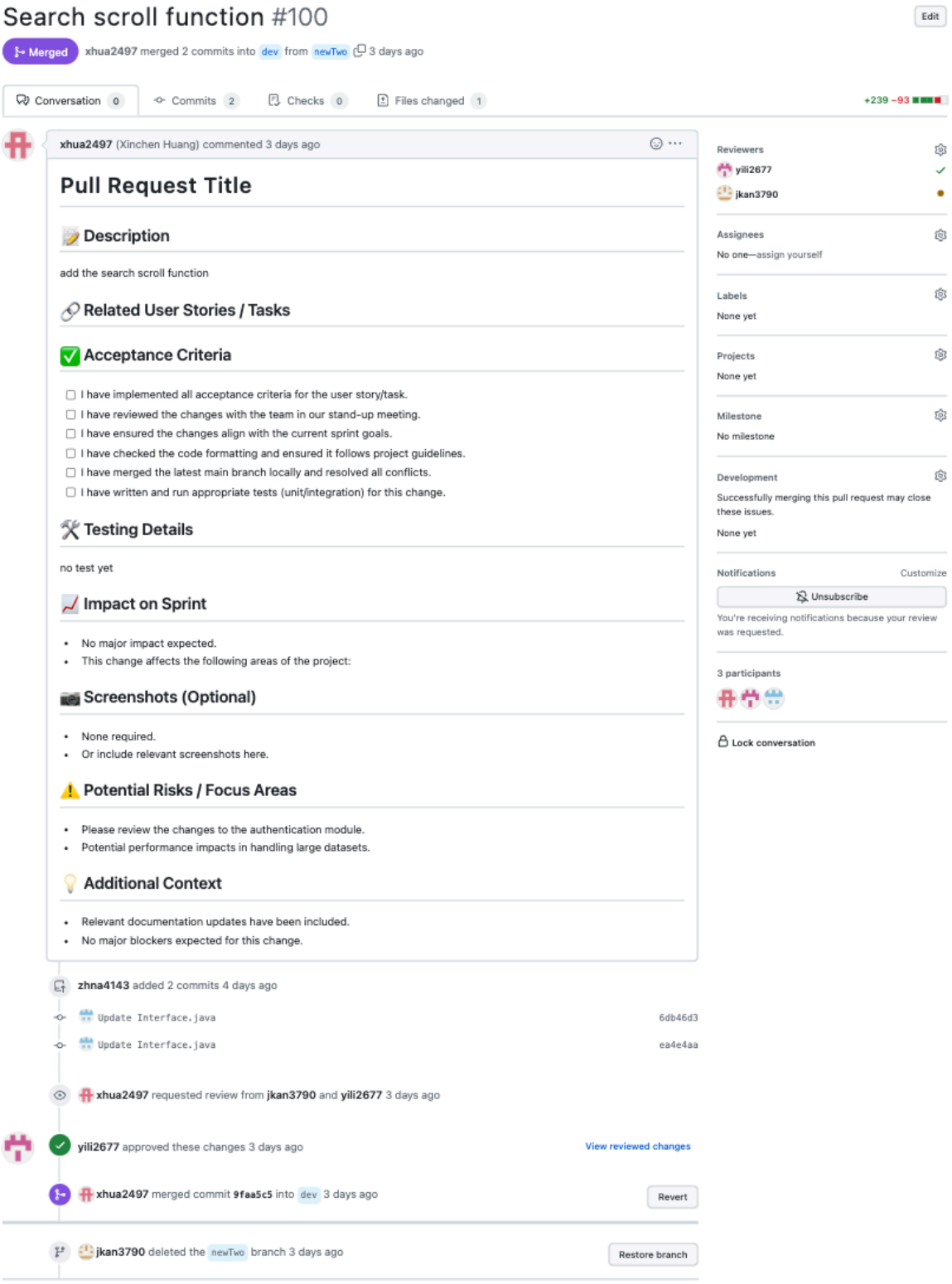
- Merge branch 'dev' into Featrue-roll-random-scroll 6183e9d

yili2677 approved these changes 5 days ago — View reviewed changes

yili2677 merged commit a05becd into dev 5 days ago — Revert

jkan3790 deleted the Featrue-roll-random-scroll branch 4 days ago — Restore branch

4. `dev` <- `newTwo` edited by `xhua2497` (approved by `yili2677`)



Search scroll function #100

Edit

Merged xhua2497 merged 2 commits into dev from newTwo 3 days ago

Conversation 0 Commits 2 Checks 0 Files changed 1 +239 −93

xhua2497 (Xinchen Huang) commented 3 days ago

Pull Request Title

Description

add the search scroll function

Related User Stories / Tasks

Acceptance Criteria

- [ ] I have implemented all acceptance criteria for the user story/task.
- [ ] I have reviewed the changes with the team in our stand-up meeting.
- [ ] I have ensured the changes align with the current sprint goals.
- [ ] I have checked the code formatting and ensured it follows project guidelines.
- [ ] I have merged the latest main branch locally and resolved all conflicts.
- [ ] I have written and run appropriate tests (unit/integration) for this change.

Testing Details

no test yet

Impact on Sprint

- No major impact expected.
- This change affects the following areas of the project:

Screenshots (Optional)

- None required.
- Or include relevant screenshots here.

Potential Risks / Focus Areas

- Please review the changes to the authentication module.
- Potential performance impacts in handling large datasets.

Additional Context

- Relevant documentation updates have been included.
- No major blockers expected for this change.

Reviewers: yili2677, jkan3790

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues. None yet

Notifications: Customize — Unsubscribe — You're receiving notifications because your review was requested.

3 participants

Lock conversation

zhna4143 added 2 commits 4 days ago

Update Interface.java 6db46d3

Update Interface.java ea4e4aa

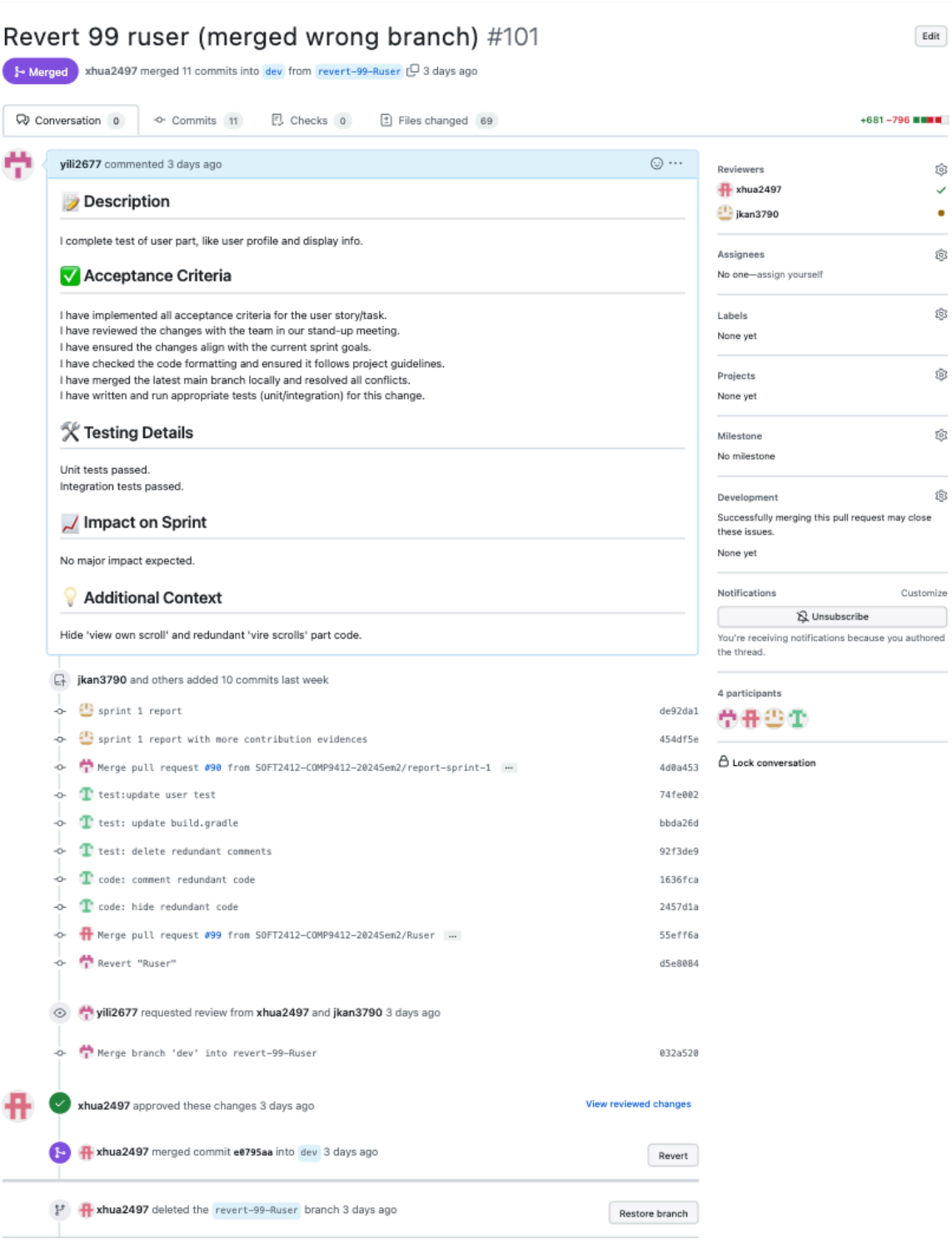
xhua2497 requested review from jkan3790 and yili2677 3 days ago

yili2677 approved these changes 3 days ago View reviewed changes

xhua2497 merged commit 9faa5c5 into dev 3 days ago Revert

jkan3790 deleted the newTwo branch 3 days ago Restore branch

5. `dev` <- `revert-99-Ruser` edited by `yili2677` (approved by `xhua2497`) revert pull request for `hjia7380` branch



Revert 99 ruser (merged wrong branch) #101

Edit

Merged xhua2497 merged 11 commits into dev from revert-99-Ruser 3 days ago

Conversation 0 · Commits 11 · Checks 0 · Files changed 69 · +681 −796

yili2677 commented 3 days ago

Description

I complete test of user part, like user profile and display info.

Acceptance Criteria

I have implemented all acceptance criteria for the user story/task.
I have reviewed the changes with the team in our stand-up meeting.
I have ensured the changes align with the current sprint goals.
I have checked the code formatting and ensured it follows project guidelines.
I have merged the latest main branch locally and resolved all conflicts.
I have written and run appropriate tests (unit/integration) for this change.

Testing Details

Unit tests passed.
Integration tests passed.

Impact on Sprint

No major impact expected.

Additional Context

Hide 'view own scroll' and redundant 'vire scrolls' part code.

jkan3790 and others added 10 commits last week

- sprint 1 report — de92da1
- sprint 1 report with more contribution evidences — 454df5e
- Merge pull request #90 from SOFT2412-COMP9412-2024Sem2/report-sprint-1 — 4d0a453
- test:update user test — 74fe002
- test: update build.gradle — bbda26d
- test: delete redundant comments — 92f3de9
- code: comment redundant code — 1636fca
- code: hide redundant code — 2457d1a
- Merge pull request #99 from SOFT2412-COMP9412-2024Sem2/Ruser — 55eff6a
- Revert "Ruser" — d5e8084

yili2677 requested review from xhua2497 and jkan3790 3 days ago

- Merge branch 'dev' into revert-99-Ruser — 032a520

xhua2497 approved these changes 3 days ago — View reviewed changes

xhua2497 merged commit e0795aa into dev 3 days ago — Revert

xhua2497 deleted the revert-99-Ruser branch 3 days ago — Restore branch

Reviewers: xhua2497 ✓, jkan3790 ●

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

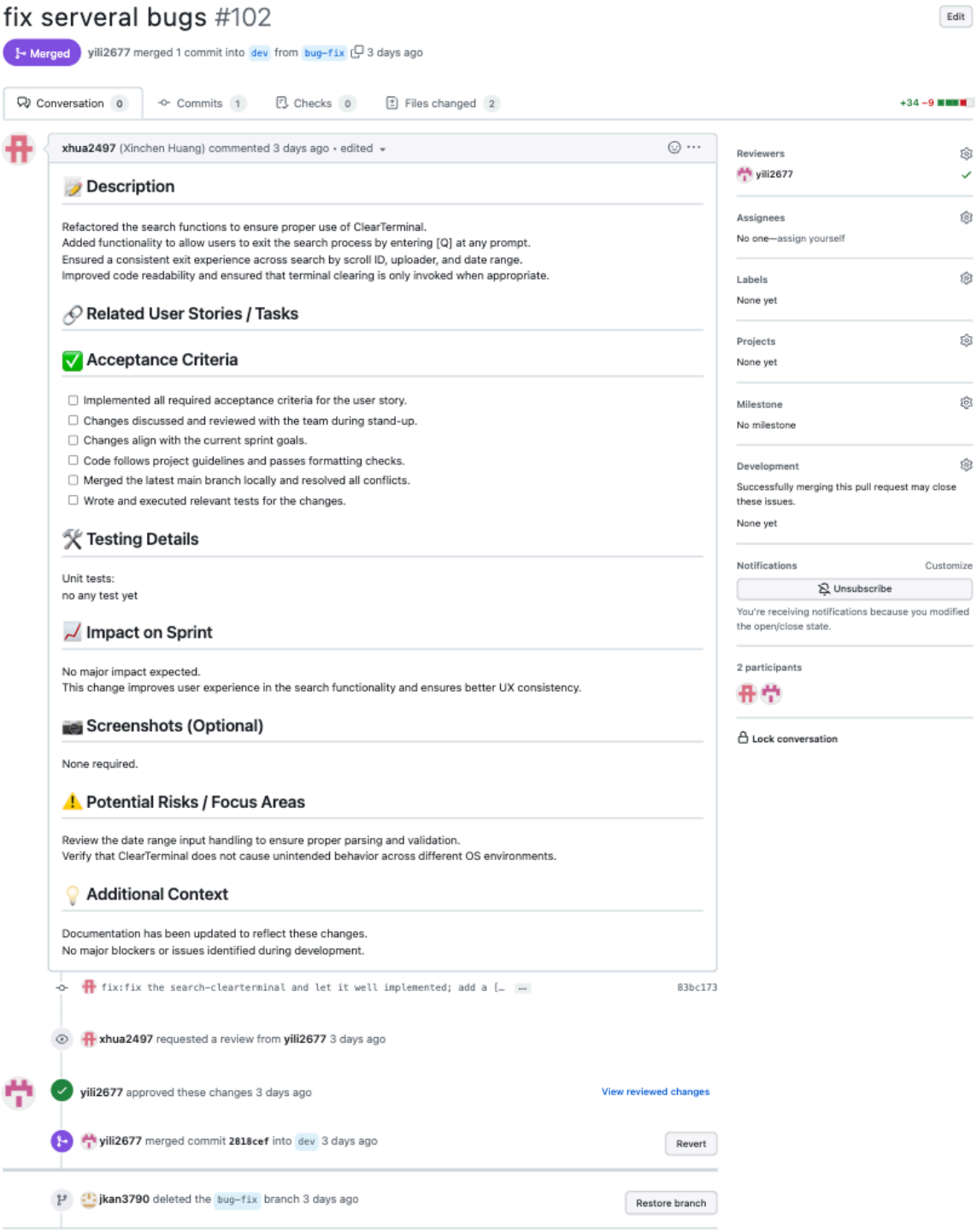
Development: Successfully merging this pull request may close these issues. None yet

Notifications — Customize: Unsubscribe. You're receiving notifications because you authored the thread.

4 participants

Lock conversation

6. `dev` <- `bug-fix` edited by `xhua2497` (approved by `yili2677`)



fix serveral bugs #102

Edit

Merged yili2677 merged 1 commit into dev from bug-fix 3 days ago

Conversation 0 · Commits 1 · Checks 0 · Files changed 2 · +34 −9

xhua2497 (Xinchen Huang) commented 3 days ago • edited

Description

Refactored the search functions to ensure proper use of ClearTerminal.
Added functionality to allow users to exit the search process by entering [Q] at any prompt.
Ensured a consistent exit experience across search by scroll ID, uploader, and date range.
Improved code readability and ensured that terminal clearing is only invoked when appropriate.

Related User Stories / Tasks

Acceptance Criteria

- [ ] Implemented all required acceptance criteria for the user story.
- [ ] Changes discussed and reviewed with the team during stand-up.
- [ ] Changes align with the current sprint goals.
- [ ] Code follows project guidelines and passes formatting checks.
- [ ] Merged the latest main branch locally and resolved all conflicts.
- [ ] Wrote and executed relevant tests for the changes.

Testing Details

Unit tests:
no any test yet

Impact on Sprint

No major impact expected.
This change improves user experience in the search functionality and ensures better UX consistency.

Screenshots (Optional)

None required.

Potential Risks / Focus Areas

Review the date range input handling to ensure proper parsing and validation.
Verify that ClearTerminal does not cause unintended behavior across different OS environments.

Additional Context

Documentation has been updated to reflect these changes.
No major blockers or issues identified during development.

fix:fix the search-clearterminal and let it well implemented; add a [… 83bc173

xhua2497 requested a review from yili2677 3 days ago

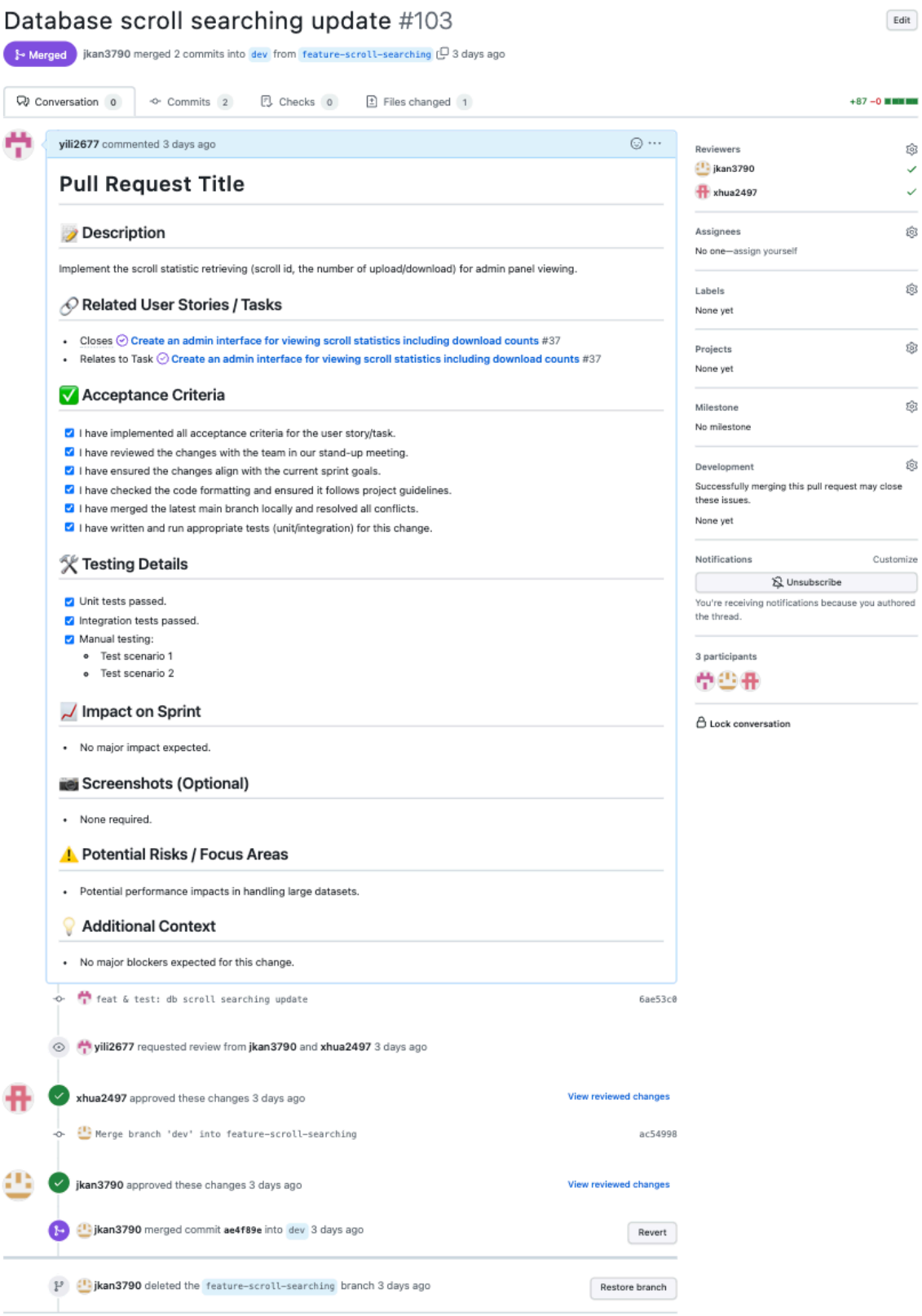
yili2677 approved these changes 3 days ago — View reviewed changes

yili2677 merged commit 2818cef into dev 3 days ago — Revert

jkan3790 deleted the bug-fix branch 3 days ago — Restore branch

Reviewers: yili2677
Assignees: No one—assign yourself
Labels: None yet
Projects: None yet
Milestone: No milestone
Development: Successfully merging this pull request may close these issues. None yet
Notifications: Customize — Unsubscribe
You're receiving notifications because you modified the open/close state.
2 participants
Lock conversation

7. `dev` <- `feature-scroll-searching` edited by `yili2677` (approved by `jkan3790` and `xhua2497`)



Database scroll searching update #103

Edit

Merged jkan3790 merged 2 commits into dev from feature-scroll-searching 3 days ago

Conversation 0 · Commits 2 · Checks 0 · Files changed 1 · +87 −0

yili2677 commented 3 days ago

Pull Request Title

Description

Implement the scroll statistic retrieving (scroll id, the number of upload/download) for admin panel viewing.

Related User Stories / Tasks

- Closes Create an admin interface for viewing scroll statistics including download counts #37
- Relates to Task Create an admin interface for viewing scroll statistics including download counts #37

Acceptance Criteria

- [x] I have implemented all acceptance criteria for the user story/task.
- [x] I have reviewed the changes with the team in our stand-up meeting.
- [x] I have ensured the changes align with the current sprint goals.
- [x] I have checked the code formatting and ensured it follows project guidelines.
- [x] I have merged the latest main branch locally and resolved all conflicts.
- [x] I have written and run appropriate tests (unit/integration) for this change.

Testing Details

- [x] Unit tests passed.
- [x] Integration tests passed.
- [x] Manual testing:
  - Test scenario 1
  - Test scenario 2

Impact on Sprint

- No major impact expected.

Screenshots (Optional)

- None required.

Potential Risks / Focus Areas

- Potential performance impacts in handling large datasets.

Additional Context

- No major blockers expected for this change.

Reviewers: jkan3790 ✓, xhua2497 ✓

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues. None yet

Notifications: Customize — Unsubscribe — You're receiving notifications because you authored the thread.

3 participants

Lock conversation

feat & test: db scroll searching update 6ae53c0

yili2677 requested review from jkan3790 and xhua2497 3 days ago

xhua2497 approved these changes 3 days ago — View reviewed changes

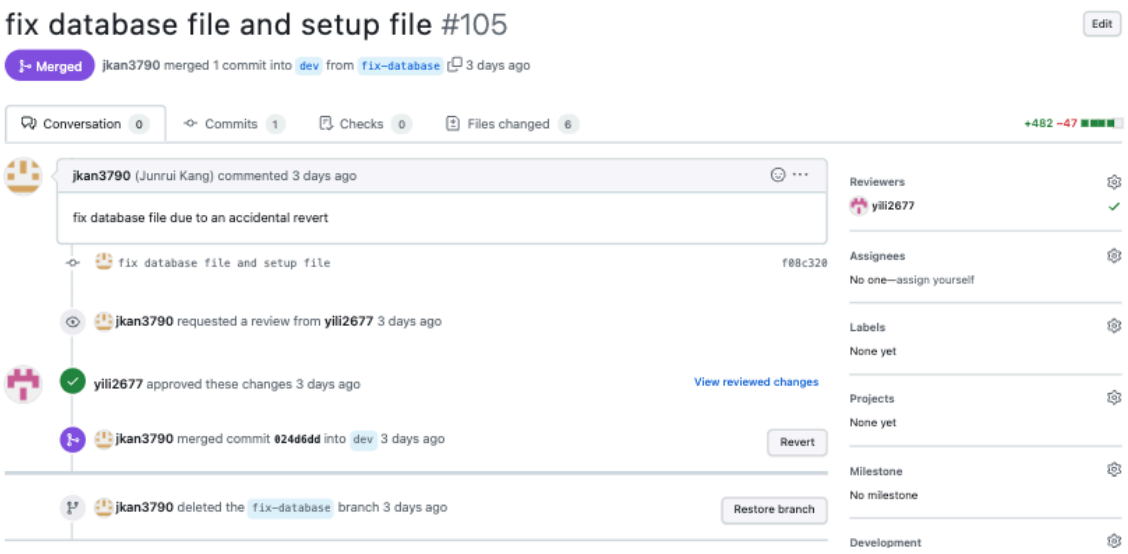
Merge branch 'dev' into feature-scroll-searching ac54998

jkan3790 approved these changes 3 days ago — View reviewed changes

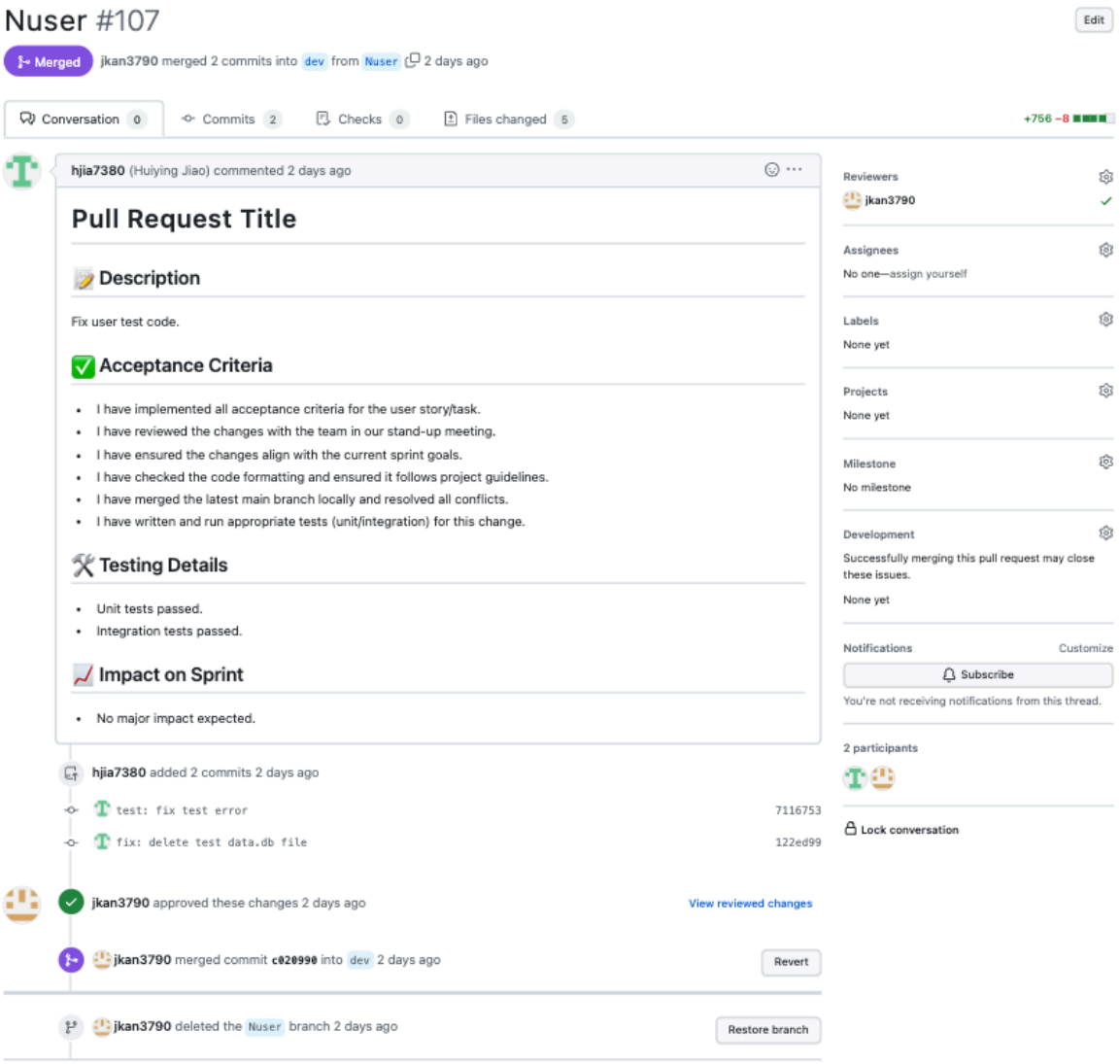
jkan3790 merged commit ae4f89e into dev 3 days ago — Revert

jkan3790 deleted the feature-scroll-searching branch 3 days ago — Restore branch

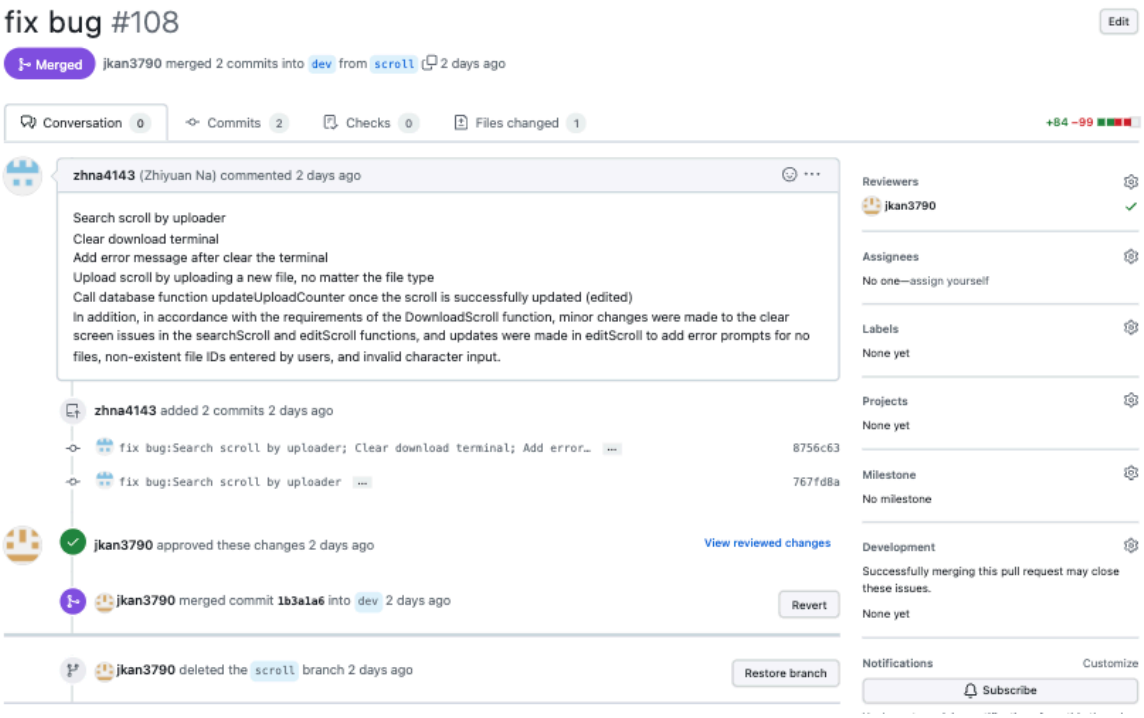
8. `dev` <- `fix-database` edited by `jkan3790` (approved by `yili2677`)



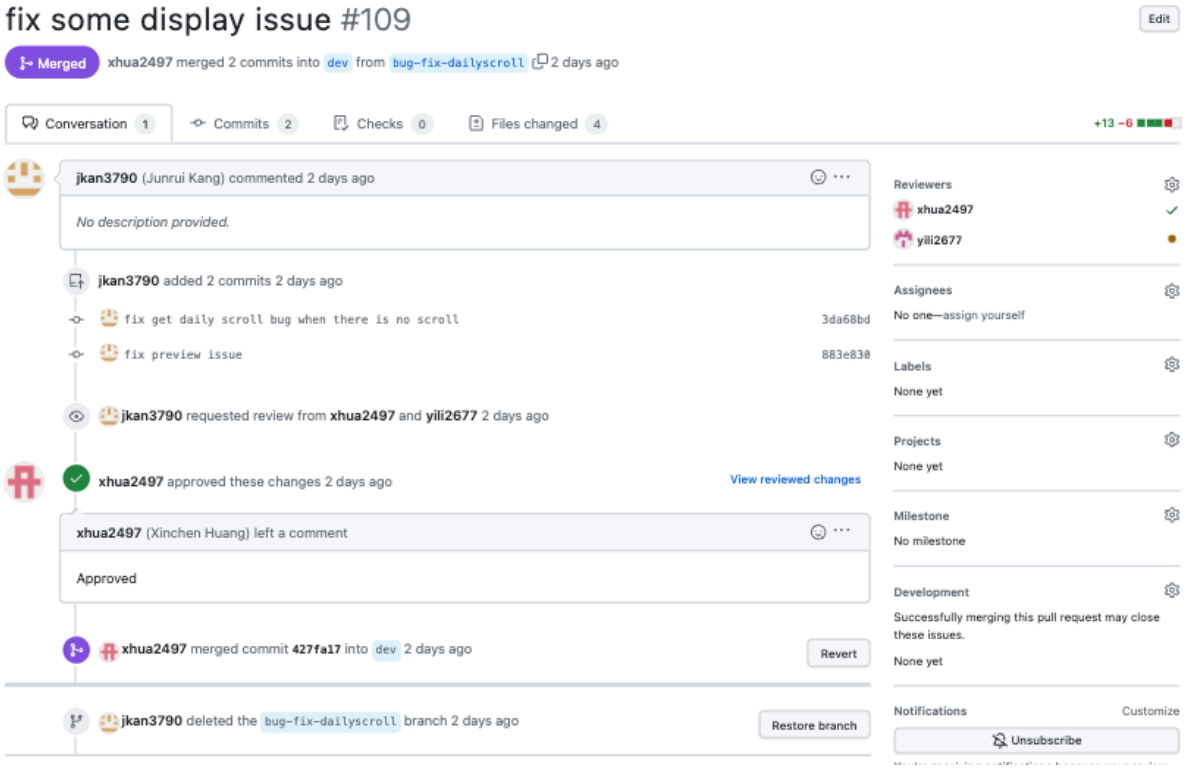
9. `dev` <- `Nuser` edited by `hjia7380` (approved by `jkan3790`)



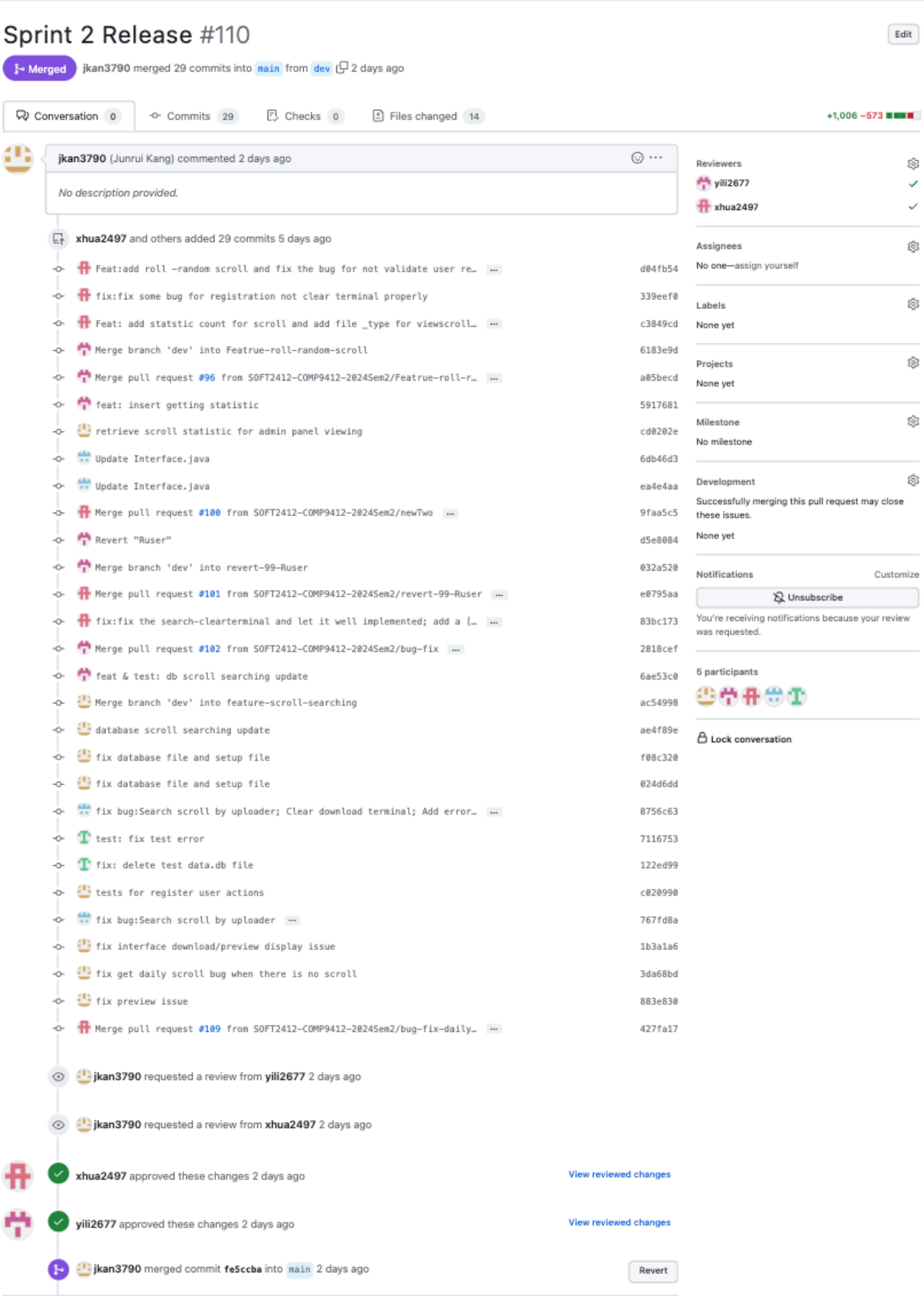
10. `dev` <- `scroll` edited by `zhna4143` (approved by `jkan3790`)



11. `dev` <- `bug-fix-dailyscroll` edited by `jkan3790` (approved by `xhua2497`)



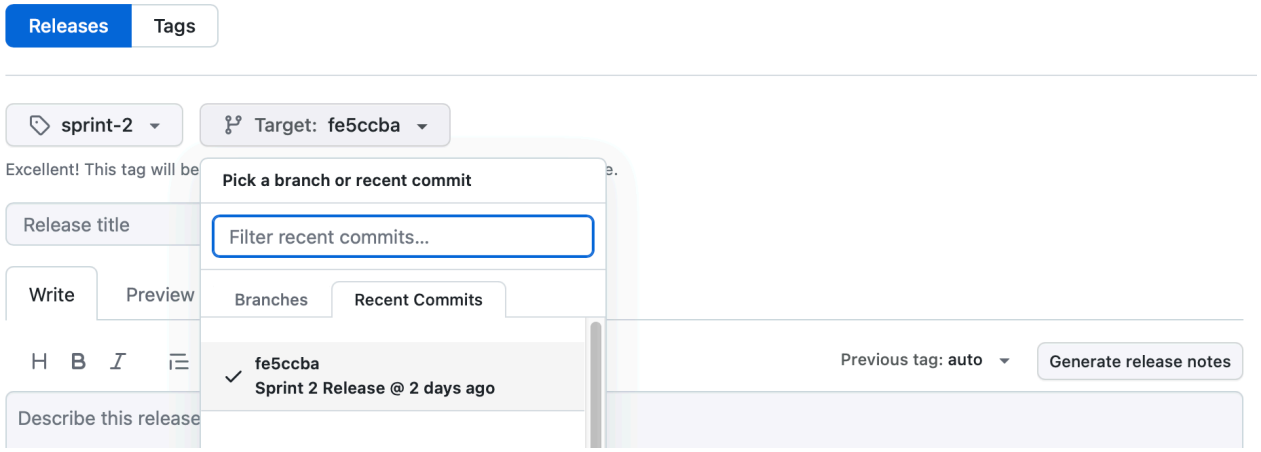
And the final coding project `dev` merged into `main` branch to summarise the sprint 2 work:
12. `main` <- `dev` edited by `jkan3790` (approved by `xhua2497` and `yili2677`)



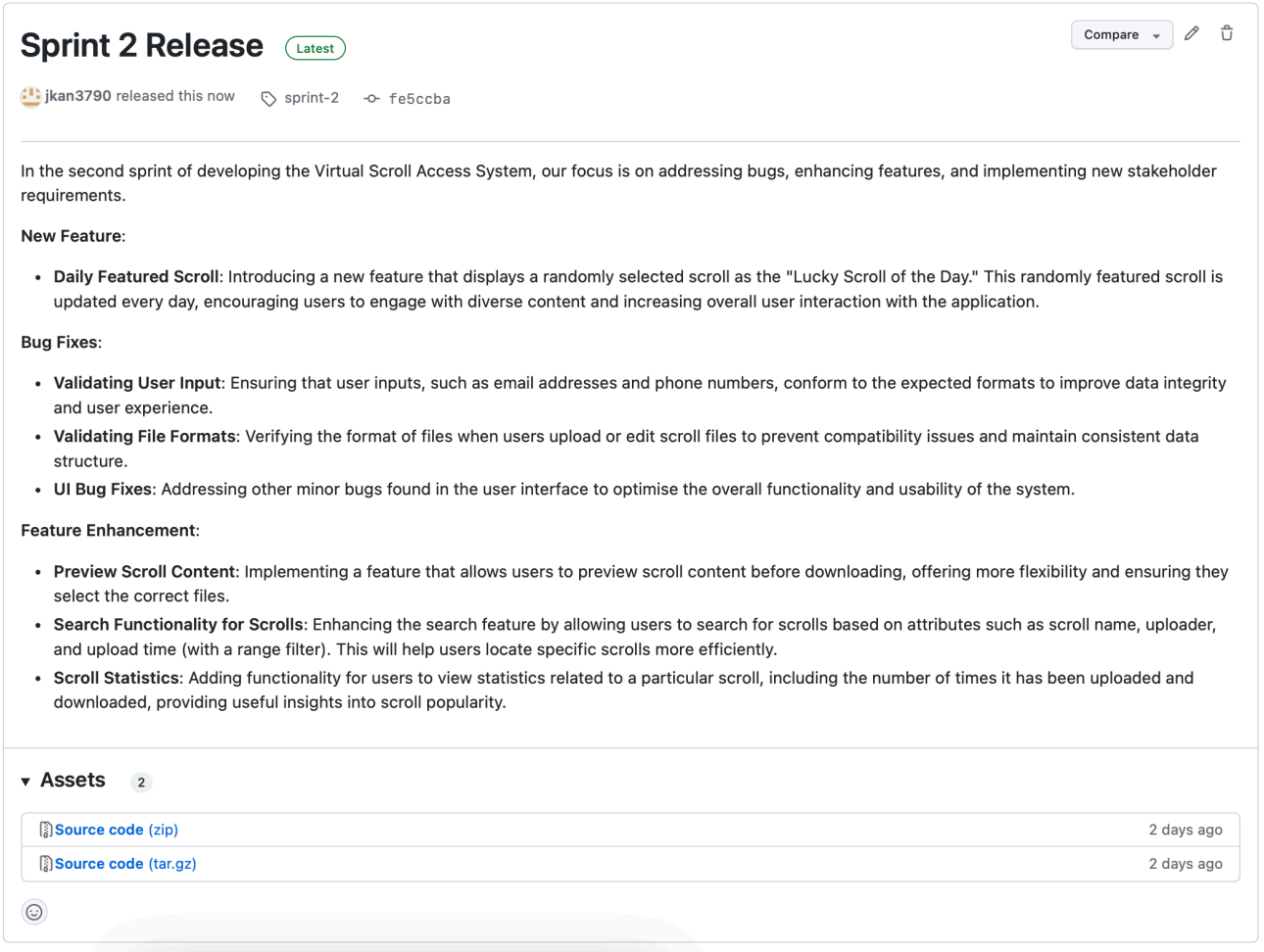
3.1.6 GitHub Release and Tags

GitHub release and tagging is used to manage and mark the release version of each sprint. By tagging these versions, the team provides the client with a clear way to visualise and track changes between sprints.

We create a new release on Wednesday night (the day before the sprint review meeting with the stakeholder) in the GitHub release page and tag it with the corresponding sprint number.



The release will contain summary of what we did in this sprint as a reference to our stakeholder.



3.2 CI/CD: Gradle

Gradle served as the build automation tool for this project, managing dependencies, compiling code, running tests, and generating results.

3.2.1 Gradle Build File

> **Reference**
>
> Please refer to Sprint 1 report (section 3.2) to see details about the gradle build file used in this project. The following part remains the same as Sprint 1.

- **Plugins**:
  - `application` plugin: This plugin supports building and running Java applications directly from the command line. It simplifies the packaging process by specifying the main class (`VirtualScrollAccessSystem.App`), allowing for the easy creation of runnable applications and automating tasks such as executing the main class.
  - `jacoco` plugin: This plugin integrates JaCoCo, a code coverage tool that measures the coverage of JUnit tests. It generates coverage reports, helping ensure that the application code is thoroughly tested.

```
plugins {
    // Apply the application plugin to add support for building a CLI
    application in Java.
    id 'application'
    id 'jacoco'
}
```

- **Repositories**:
  - `mavenCentral()`: Gradle pulls all required dependencies from the Maven Central repository, facilitating the download and management of external libraries.

```
repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}
```

- **Dependencies**:
  - `testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'` : Adds JUnit Jupiter (JUnit 5) as a dependency for unit testing. JUnit Jupiter provides a structured framework for writing and executing tests.
  - `implementation 'com.google.guava:guava:30.1.1-jre'` : Integrates Google Guava, a library offering utilities for collections, caching, and string manipulation, making it easier to handle complex data structures and utility functions within the application.
  - `implementation 'org.xerial:sqlite-jdbc:3.46.1.3'` : Adds support for using SQLite as a database through the SQLite JDBC driver.
  - `implementation 'org.mindrot:jbcrypt:0.4'` : Incorporates the BCrypt library for password hashing, enhancing the security of user credentials.

```
dependencies {
    // Use JUnit Jupiter for testing.
    testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'

    // This dependency is used by the application.
    implementation 'com.google.guava:guava:30.1.1-jre'

    // SQLite JDBC dependency
    implementation 'org.xerial:sqlite-jdbc:3.46.1.3'

    // Bcrypt dependency
    implementation 'org.mindrot:jbcrypt:0.4'
}
```

- **Application Configuration**:
  - `application.mainClass = 'VirtualScrollAccessSystem.App'` : Defines the main entry point as the `App` class within the `VirtualScrollAccessSystem` package, simplifying the execution of the application using Gradle.

```
application {
    // Define the main class for the application.
    mainClass = 'VirtualScrollAccessSystem.App'
}
```

- **Testing**:
  - `test (JUnit test)` task: The test task utilises the JUnit platform (`useJUnitPlatform()`) for running unit tests with JUnit 5. Automated testing offers detailed feedback on test outcomes, ensuring the application behaves as expected.
  - `junitXml.required.set(true)` and `html.required.set(true)` : These configurations ensure that XML and HTML reports are generated for test results.

The XML report is suitable for CI platforms like Jenkins, while the HTML report is convenient for developers to view in a browser.

```
tasks.named('test') {
    // Use JUnit Platform for unit tests.
    useJUnitPlatform()
}

jacoco {
    toolVersion = "0.8.8"  // Or the latest version
}

test {
    useJUnitPlatform()
    testLogging {
        events "passed", "skipped", "failed"
    }
    reports {
        junitXml.required.set(true)
        html.required.set(true)
    }
    maxParallelForks = 1
}
```

- **Code Coverage**:
  - `jacoco` and `jacocoTestReport`: The `jacoco` task utilises JaCoCo for measuring code coverage, while the `jacocoTestReport` task generates reports in XML and HTML formats. This helps track the percentage of code covered by tests, highlighting areas that need further testing.

```
jacocoTestReport {
    dependsOn test // Ensure the tests are run before generating the report
    reports {
        xml.required.set(false) // Disable the XML report if not needed
        csv.required.set(false) // Disable the CSV report if not needed
        html.required.set(true) // Enable the HTML report
    }
}
```

- **Custom Tasks**:
  - `run { standardInput = System.in }`: Configures the application to accept user input from the command line during execution, enabling real-time interaction with users.

```
tasks.named('run') {
    standardInput = System.in
}
```

3.2.2 Gradle Commands

The main Gradle commands utilised were `gradle init`, `gradle build`, `gradle run`, `gradle run --console=plain`, and `gradle test`.

- **`gradle init`**: This command initialises the Gradle project, creating the necessary directory structure, configuration files (`build.gradle`), and other essential files for the project setup. It serves as the first step to prepare the environment for development.
- **`gradle build`**: This command compiles the source code and builds the project into a JAR file. It also verifies that all dependencies are available and that there are no compilation errors. Running `gradle build` ensures that the application is properly assembled and packaged before further steps are taken. The command was executed as the second step after `gradle init` and coding part to validate that the code was stable and ready for testing.
- **`gradle test`**: Following the build, `gradle test` was executed to run all unit and integration tests defined in the project. This step is essential for verifying that the code functions as expected and meets the quality and functional requirements. This command was run immediately after `gradle build` to catch any issues in the code before moving to execution.
- **`gradle run`**: After the code passed all tests, the application was executed using the `gradle run` command. This command starts the application by running the main class specified in the `build.gradle` file. It allows for validating that the build works as expected when deployed and executed in the local environment.

The sequence of running these commands typically followed this order:

1. `gradle init` command sets up the project environment (done in the Sprint 0).
2. `gradle build` command compiles the application.

```
kang@macmini virtual-scroll-access-system % gradle build -x test

BUILD SUCCESSFUL in 259ms
6 actionable tasks: 6 up-to-date
kang@macmini virtual-scroll-access-system %
```

   As shown in the provided screenshot, the build for the Sprint 2 release code was successful, indicating that all code compiled correctly. Normally, `gradle build` command will run all unit tests found in the test directory, in this case, for demonstration, `gradle build -x test` is used to skip the testing step.

3. `gradle test` command runs tests.

```
kang@macmini virtual-scroll-access-system % gradle test

> Task :app:test

DatabaseTest > testDeleteUserWithInvalidUserID() PASSED
```

```
BUILD SUCCESSFUL in 6s
5 actionable tasks: 2 executed, 3 up-to-date
kang@macmini virtual-scroll-access-system %
```

   The command outputs the results of the tests, indicating which tests passed and which failed. All tests pass in this screenshot.

4. `gradle run` command executes the application.

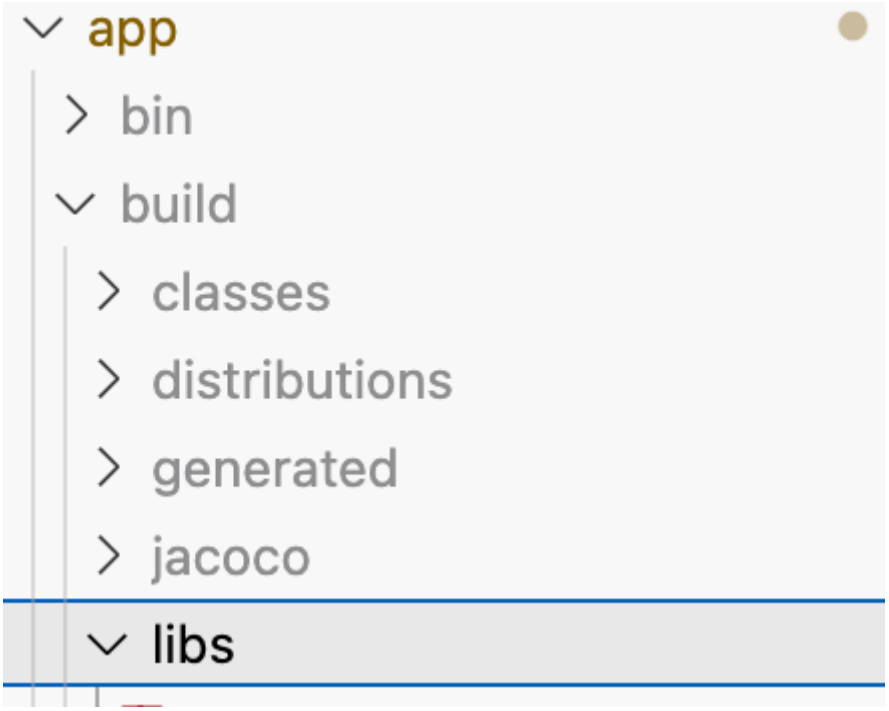
```
kang@macmini virtual-scroll-access-system % gradle run
<==========----> 75% EXECUTING [1s]
> :app:run
```

   This command executes the application. It runs the main class defined in the project configuration, which is `App` in this project. The VSAS application will be executed then.

3.2.3 Gradle Builded JAR File

After executing the `gradle build` command, a JAR (Java Archive) file is generated as part of the build process.

This file is a packaged representation of the compiled application, containing all the necessary classes, resources, and metadata required for execution. All Java class files created from the source code are included, allowing the application to run without needing to recompile in the client's devices.



The builded JAR file can be executed using `java -jar <jar_file.jar>` command.

```
kang@macmini virtual-scroll-access-system % java -jar '/Users/kang/Documents/adv-comp/soft2412/virtual-scroll-access-system/app/build/libs/app.jar'
```

```
Welcome to the library!
Do you want to log in to an existing account[1], register new account[2], visit as a guest[3], or exit[4]?
```

This confirms that the JAR is functioning as intended after being built and packaged by Gradle.

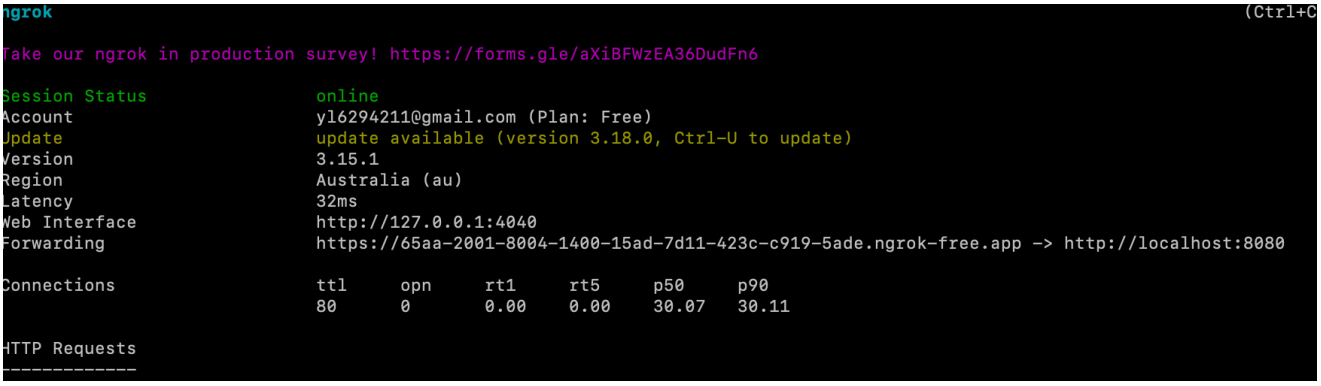
3.3 CI/CD: Jenkins

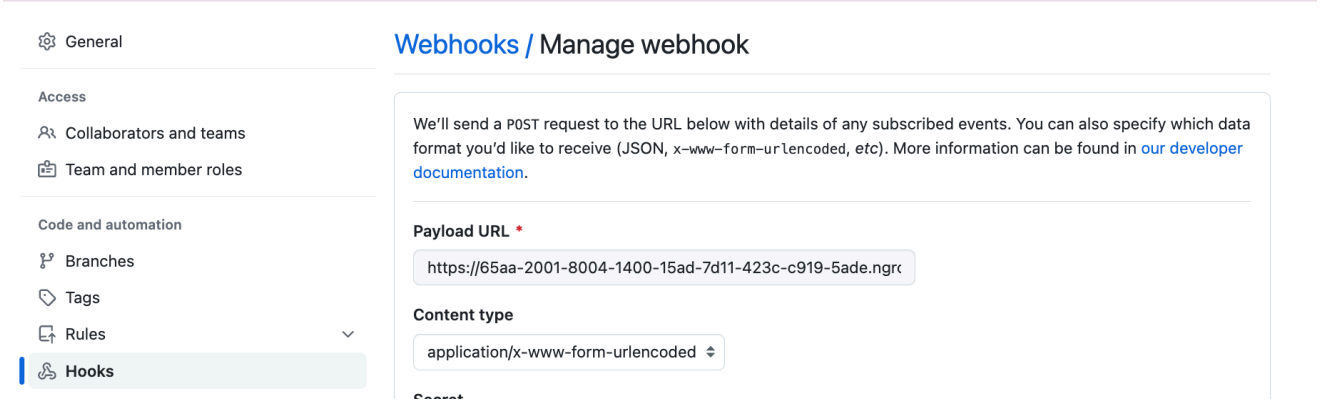
> **Reference**
>
> Please refer to Sprint 1 report (section 3.3) to see the Jenkins usage.

The Jenkins configuration in this program is developed following Edstem's tutorial, enabling web-hook communication between the local Jenkins instance and the GitHub repository using `ngrok http 8080`. Once connected, Jenkins automatically triggers builds and runs automated tests whenever code is contributed to the shared GitHub repository.

3.3.1 Jenkins Integrated With GitHub

The Jenkins and GitHub Integration is built through the GitHub WebHooks. As the time when the `ngrok http 8080` is executing on the developer own PC, then the `Payload URL` should be generated on the terminal. As the time the integration is hooked, the developer should ensure that `ngrok http 8080` is always executing in the backend on the developer's PC. Otherwise, the link would be terminated and the Jenkins Project cannot be hooked as the previous `Payload URL`, where the `ngrok http 8080` is automatically changing as each time opening. As well as this hook has been employed at the beginning of the scrum project (Sprint 0), therefore the data is unable to lose dynamically.



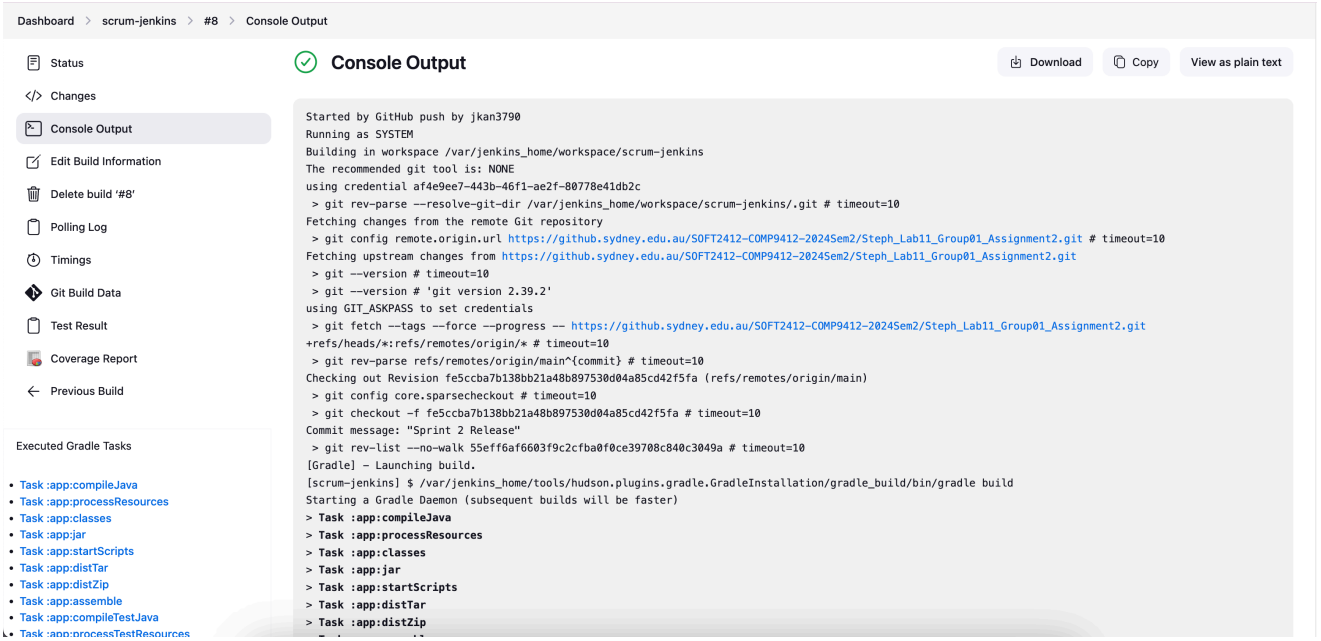


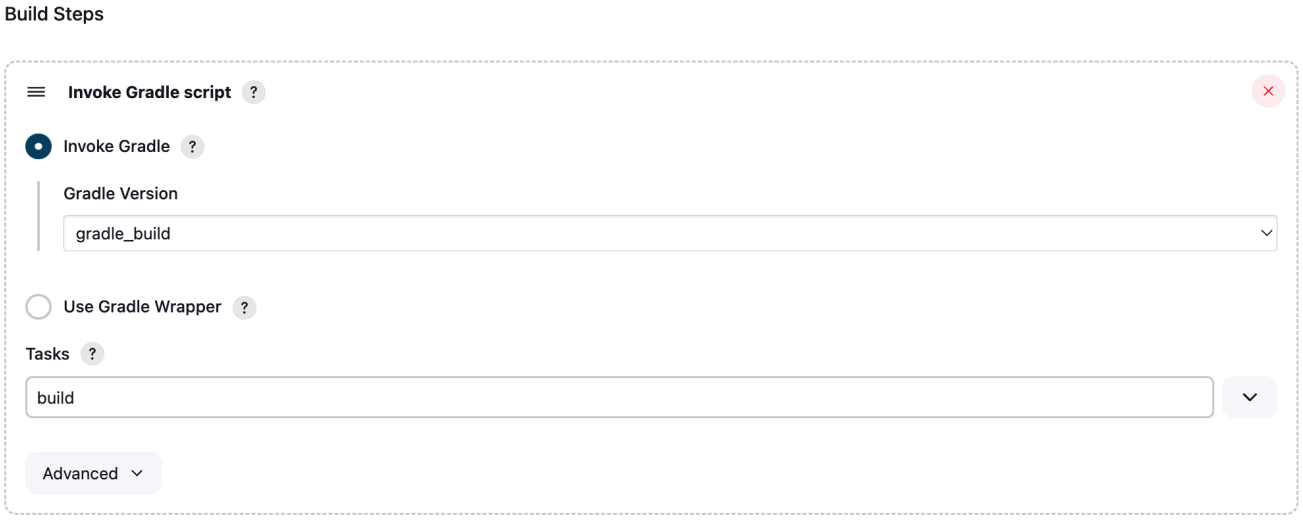
The following displays the how to receive changes once the developer `jkan3790` successfully pushed the summary of the sprint 2 work to a GitHub repository. When Jenkins receives a message from the GitHub webhook, it utilises git to fetch the most recent commit

from the repository (SEND TRIGGER TO JENKINS FROM GitHub) and checks the source code using a sequence of tasks.

The console output indicates that Jenkins connected to the repository and downloaded the most recent updates. The job runs smoothly and without mishap. This means that the code was successfully retrieved from GitHub, built, and is available for further processing.

Gradle (`gradle_build`) is also set up for usage with Jenkins, ensuring that Jenkins automatically downloads and install the required Gradle version. This step enables Jenkins to automate the execution of Gradle test tasks for all unit and integration tests defined in the project.

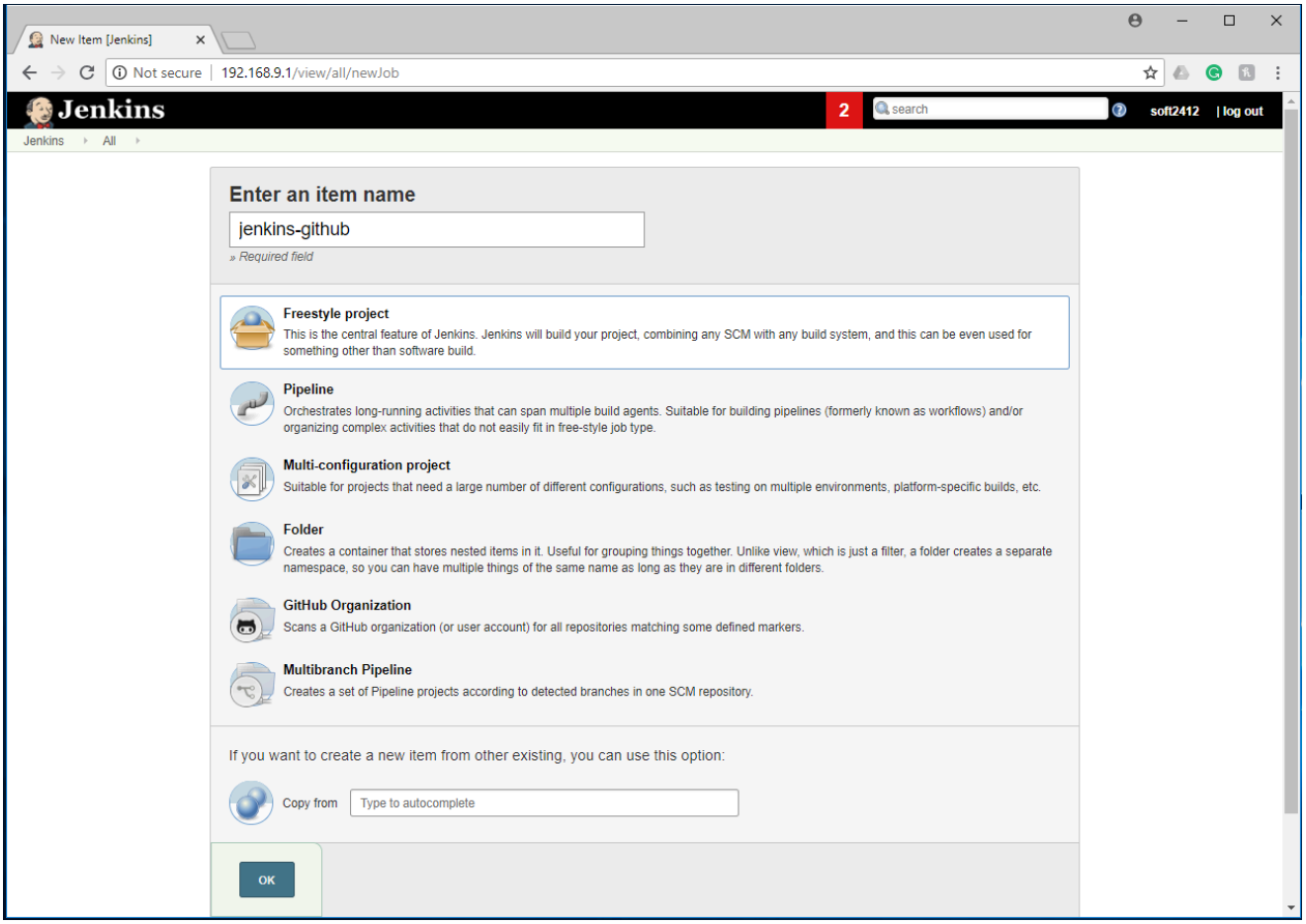




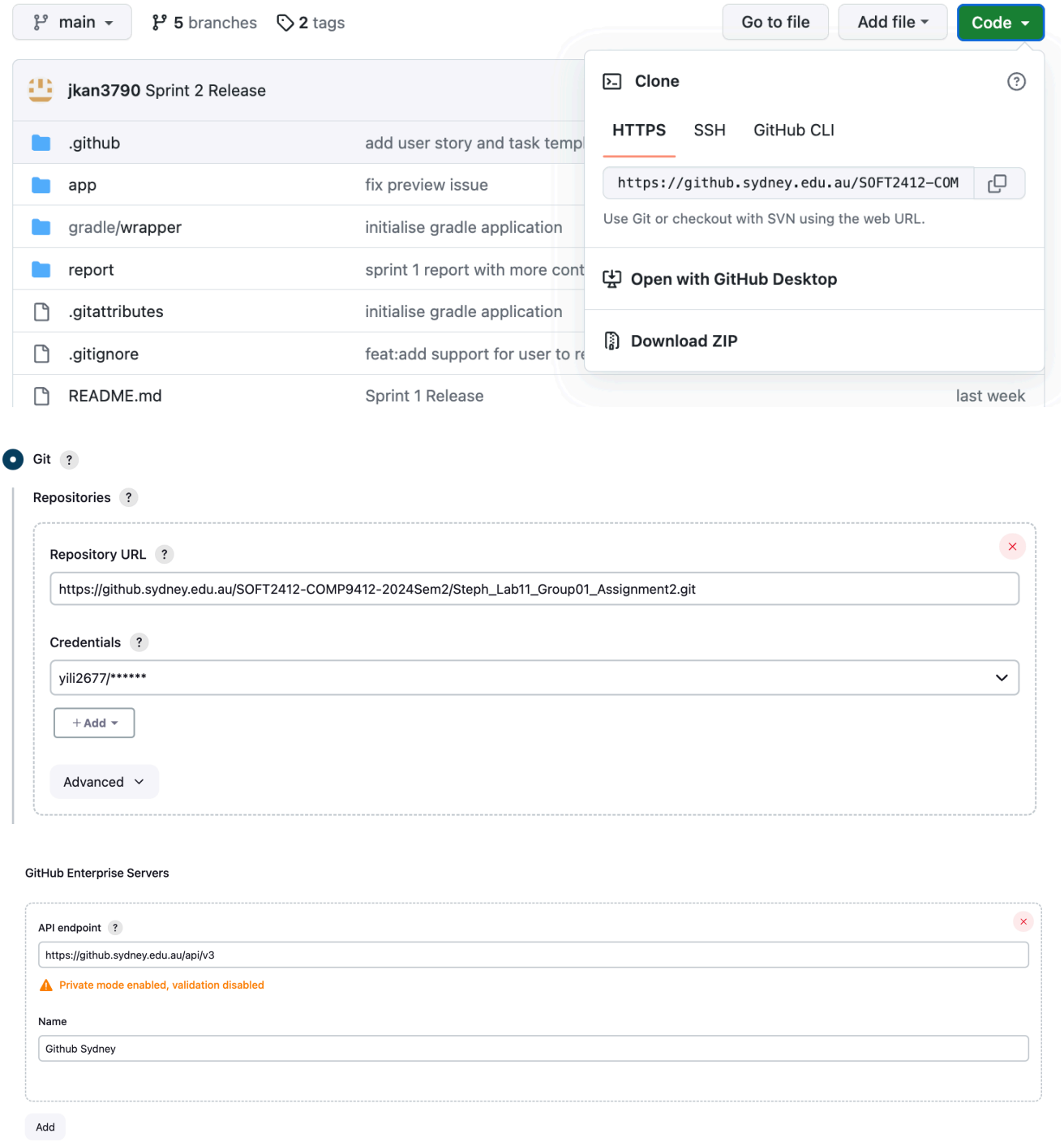
3.3.2 Jenkins Build and Test Coverage

Jenkins Build

The continuous integration (CI) plugins built on the Jenkins facilitate the development and reduce the rate of complexity existing in the team work. Where the Jenkins employed in this project is utilised a freestyle project, which is the central feature of Jenkins extremely suitable for the software design.



During the connection between Jenkins and GitHub, the repository should be noticed that using the `HTTPS` as the figure shown (`https://github.sydney.edu.au/SOFT2412-COMP9412-2024Sem2/Steph_Lab11_Group01_Assignment2.git` for current sprint 2 using), and then connected with the developer's Global credentials (username and password working with the GitHub repository) and API with the `usyd GitHub`. In here, the developer `yili2677` is the Jenkins manager for this whole scrum project.

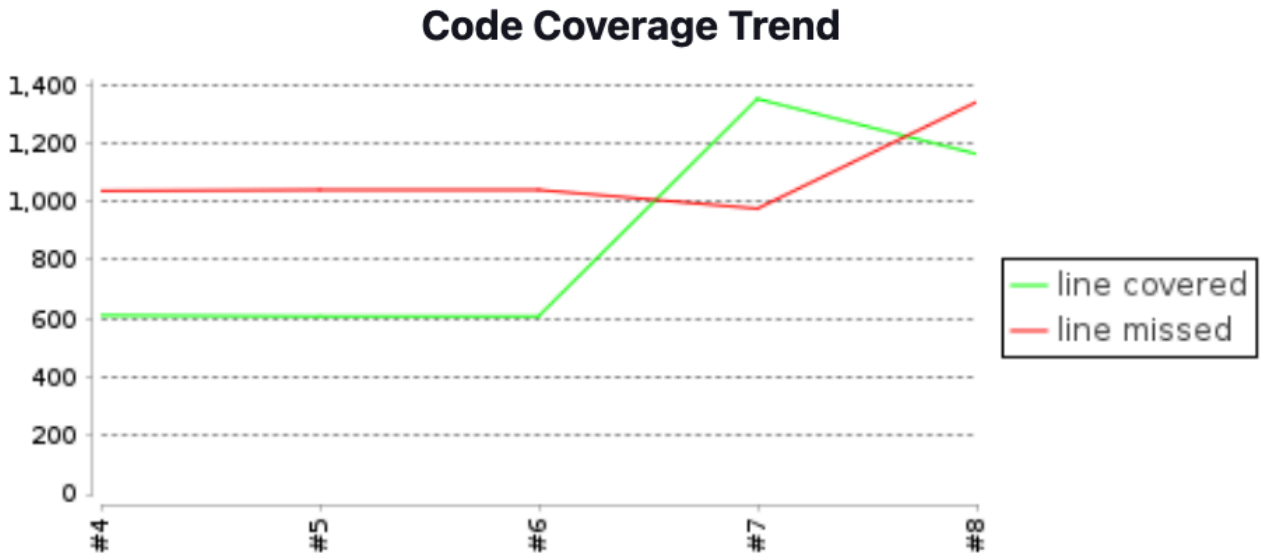


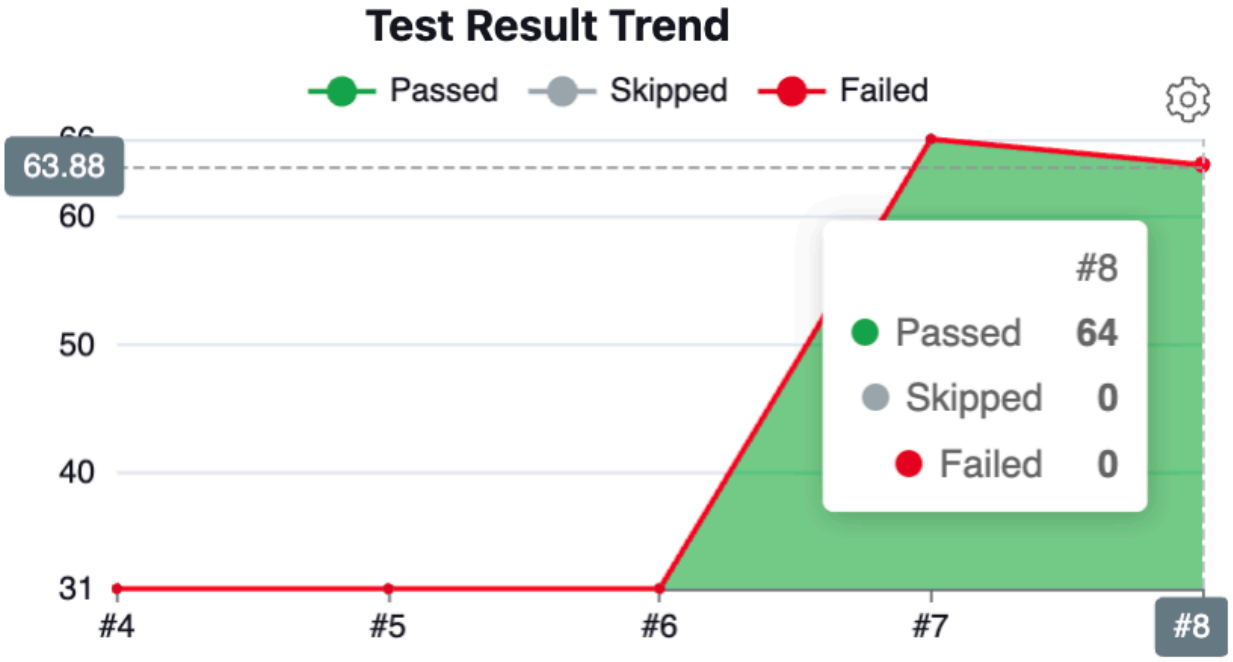
Excepted the normal plugins for the general software design using (Gits and Gradle), there are several extra plugins have been applied in this scrum project (as well as sprint 2):

Publish JUnit test result report: this plugin

The JUnit test result report is automatically generated with every `git push` to the `main` branch, taking advantage of the CI's ability to automate and consolidate changes, as opposed to the usual cumbersome `gradle test` (which only generates a report with the run command and requires a lookup in the file to see it). Jenkins visualization of trends can easily help teams quickly browse the information. This can play a big advantage in meeting.png)

Comparing to the sprint 1, the code coverage trend and test result trend both decreased. This is caused by the issue demonstrated on the contribution confusion (linked to 2.2.5) and partial of the testcases in `Database.java` not included in this push. This issue will be fixed on the next sprint.





Record JaCoCo coverage report

Similarly, this plugin also handles the role of CI ability and automatically generates a JaCoCo report on Jenkins dashboard. This saves a lot of times for searching the file directory and looking up. As the below generated, the sprint 2 test coverage with `INSTRUCTION = 50%` and `BRANCH = 25%`. This has a large range not covered tested as the client limitation (both coverage at least 75%). Thus, on sprint 3 should focus on the testcase writing.

Test Result (no failures)

Jacoco - Overall Coverage Summary

INSTRUCTION	50%
BRANCH	25%
COMPLEXITY	37%
LINE	46%
METHOD	68%
CLASS	69%

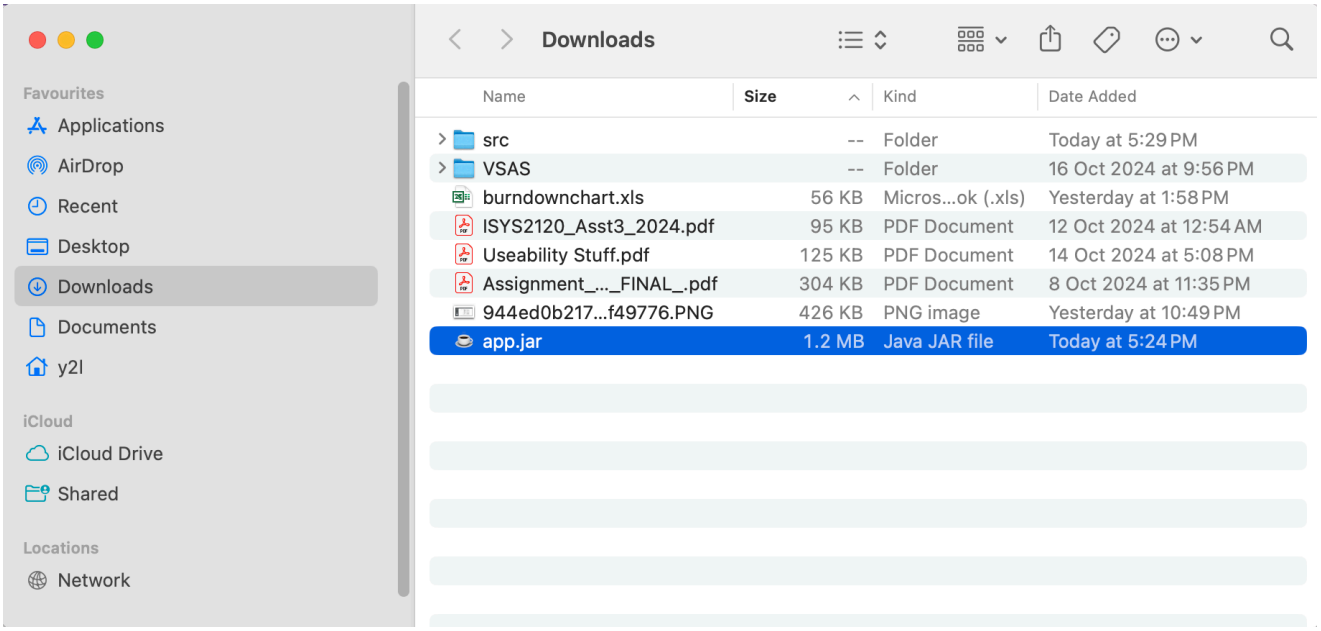
Archive the Artefacts

Jenkins generates the 'app.jars' plugin with each code push, which includes the libraries required for the plugin's functionality. It includes reusable code components that the Jenkins plugin can use without rewriting. This also allows the developer to download and run the file in their own terminal without having to use code.

3.3.3 CI Jar Running In Terminal

As the `app.jar` automatically generated by Jenkins after pushing code, this file can be able to execute with the appropriate command after downloading into local PC. Here is the steps for the sprint 2 `app.jar` running:

1. Firstly, the `app.jar` file generated by Jenkins for the last successful artifacts (`Sprint 2 release` commit pushed [link to 3.1.5]) should be downloaded into the local directory. As the developer testing shown, this file has been downloaded into `Downloads` folder.



2. Then, the developer turns on the terminal and moves the current directory to `Downloads` then execute the command:

```
jar tf app.jar
```

This is used to list all the contents in this downloaded file. As the terminal shown, the main class file `VirtualScrollAccessSystem/` is located in this jar file, which means it is able to run without any origin code based.

```
[y2l@aidmxx Downloads %  jar tf app.jar
META-INF/
META-INF/MANIFEST.MF
VirtualScrollAccessSystem/
VirtualScrollAccessSystem/Setup.class
VirtualScrollAccessSystem/RegisteredUser.class
VirtualScrollAccessSystem/User.class
VirtualScrollAccessSystem/Interface.class
VirtualScrollAccessSystem/App.class
VirtualScrollAccessSystem/Database.class
VirtualScrollAccessSystem/util/
VirtualScrollAccessSystem/util/DBUtil.class
VirtualScrollAccessSystem/AdminUser.class
VirtualScrollAccessSystem/UserProfileManager.class
VirtualScrollAccessSystem/ExitProgramException.class
VirtualScrollAccessSystem/Scroll.class
VirtualScrollAccessSystem/ClearTerminal.class
data.db
```

3. Finally, the jar running command directly:

```
java -cp app.jar VirtualScrollAccessSystem.App
```

If the `App.class` in the original Java code folder contains a `main` method, this command should kick the action as the below showing, which currently runs the project as same as sprint 2 uploaded code.

```
y2l@aidmxx Downloads % java -cp app.jar VirtualScrollAccessSystem.App
Using existing resource folder: /Users/y2l/Downloads/src/main/resources
java.lang.ClassNotFoundException: org.sqlite.JDBC
        at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:641)
        at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.java:188)
        at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:525)
        at java.base/java.lang.Class.forName0(Native Method)
        at java.base/java.lang.Class.forName(Class.java:375)
        at VirtualScrollAccessSystem.util.DBUtil.connect(DBUtil.java:59)
        at VirtualScrollAccessSystem.Setup.initDatabase(Setup.java:30)
        at VirtualScrollAccessSystem.Database.<init>(Database.java:46)
        at VirtualScrollAccessSystem.App.main(App.java:57)
java.lang.NullPointerException: Cannot invoke "java.sql.Connection.prepareStatement(String)" because "conn" is null
        at VirtualScrollAccessSystem.Setup.createUserTable(Setup.java:52)
        at VirtualScrollAccessSystem.Database.<init>(Database.java:47)
        at VirtualScrollAccessSystem.App.main(App.java:57)
java.lang.NullPointerException: Cannot invoke "java.sql.Connection.prepareStatement(String)" because "conn" is null
        at VirtualScrollAccessSystem.Setup.insertDefaultAdminUser(Setup.java:86)
        at VirtualScrollAccessSystem.Setup.createUserTable(Setup.java:58)
        at VirtualScrollAccessSystem.Database.<init>(Database.java:47)
        at VirtualScrollAccessSystem.App.main(App.java:57)
java.lang.NullPointerException: Cannot invoke "java.sql.Connection.prepareStatement(String)" because "conn" is null
        at VirtualScrollAccessSystem.Setup.createScrollTable(Setup.java:132)
        at VirtualScrollAccessSystem.Database.<init>(Database.java:48)
        at VirtualScrollAccessSystem.App.main(App.java:57)
java.lang.NullPointerException: Cannot invoke "java.sql.Connection.prepareStatement(String)" because "conn" is null
        at VirtualScrollAccessSystem.Setup.createScrollCategoryTable(Setup.java:154)
        at VirtualScrollAccessSystem.Database.<init>(Database.java:49)
        at VirtualScrollAccessSystem.App.main(App.java:57)
java.lang.NullPointerException: Cannot invoke "java.sql.Connection.prepareStatement(String)" because "conn" is null
        at VirtualScrollAccessSystem.Setup.createCategoryTable(Setup.java:176)
        at VirtualScrollAccessSystem.Database.<init>(Database.java:50)
        at VirtualScrollAccessSystem.App.main(App.java:57)
java.lang.NullPointerException: Cannot invoke "java.sql.Connection.prepareStatement(String)" because "conn" is null
        at VirtualScrollAccessSystem.Setup.createDailyScrollTable(Setup.java:198)
        at VirtualScrollAccessSystem.Database.<init>(Database.java:51)
        at VirtualScrollAccessSystem.App.main(App.java:57)



Welcome to the library!
Do you want to log in to an existing account[1], register new account[2], visit as a guest[3], or exit[4]?
1
```

3.4 Testing: JUnit and Jacoco

3.4.1 Overall Test Coverage

app

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
VirtualScrollAccessSystem		30%		23%	292	374	1,268	1,776	58	103	4	11
VirtualScrollAccessSystem.util		64%		50%	9	15	16	39	2	8	0	1
Total	4,102 of 5,971	31%	418 of 552	24%	301	389	1,284	1,815	60	111	4	12

As part of our ongoing efforts to ensure code quality and reliability, we have evaluated the test coverage for the project. Currently, the overall test coverage is as follows:

- **Instruction Coverage**: 31%
- **Branch Coverage**: 23%

Instruction Coverage measures the percentage of executable lines of code that have been tested. A coverage of 31% indicates that a little less than one-third of the code has been executed during testing, suggesting that there are many untested paths in the codebase.

Branch Coverage examines the percentage of decision points (like `if` statements) that have been tested. With a branch coverage of 23%, this implies that only a small fraction of the conditional paths in the code have been exercised.

The current code coverage metrics indicate that our project's coverage is low. Several factors have contributed to this situation:

1. **Time Pressure**: The development timeline for the project has been compressed, leading to constraints on how thoroughly we could test the code.
2. **Reliance on Manual Testing**: Instead of implementing formal test cases, the team primarily conducted manual testing for frontend functionalities.

The breakdown of testing coverage for different classes are shown in the screenshot below.

VirtualScrollAccessSystem

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Interface		3%		0%	166	167	732	733	31	32	0	1
Database		48%		47%	74	104	281	535	12	30	0	1
AdminUser		0%		0%	32	32	176	176	8	8	1	1
App		0%		0%	5	5	17	17	2	2	1	1
RegisteredUser		85%		84%	5	29	34	145	0	13	0	1
Setup		85%		50%	2	10	14	67	1	9	0	1
UserProfileManager		95%		90%	3	19	6	82	0	2	0	1
User		80%		50%	2	4	4	14	1	3	0	1
ExitProgramException		0%		n/a	1	1	2	2	1	1	1	1
ClearTerminal		66%		n/a	1	2	1	4	1	2	0	1
Scroll		0%		n/a	1	1	1	1	1	1	1	1
Total	4,062 of 5,859	30%	411 of 538	23%	292	374	1,268	1,776	58	103	4	11

3.4.2 Database Tests

> **Note**
>
> These tests are written by Yilin Li (yili2677) and Junrui Kang (jkan3790)

app > VirtualScrollAccessSystem > Database

Database

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
searchScrollByUploader(String)		0%		0%	4	4	25	25	1	1
searchScrollByName(String)		0%		0%	4	4	24	24	1	1
searchScrollByTimeRange(Date, Date)		0%		0%	2	2	25	25	1	1
getScrollInfoByName(String)		0%		0%	2	2	20	20	1	1
getScrollStatistics()		0%		0%	2	2	17	17	1	1
addDailyScroll(Date, int)		0%		0%	2	2	15	15	1	1
getScrollIDOfTheDay(Date)		0%		0%	2	2	14	14	1	1
getAllTextScrolls()		0%		0%	2	2	12	12	1	1
isUserScroll(String, int)		0%		n/a	1	1	13	13	1	1
updateUploadCounter(int)		0%		0%	2	2	10	10	1	1
updateDownloadCounter(int)		0%		0%	2	2	10	10	1	1
updateScroll(File, int)		66%		50%	4	5	9	24	0	1
updateScrollContent(int, String)		0%		0%	2	2	8	8	1	1
updatePassword(String, String)		66%		60%	4	6	6	20	0	1
updateUserName(String, String)		64%		60%	4	6	6	19	0	1
updatePhoneNumber(String, String)		64%		60%	4	6	6	19	0	1
updateEmail(String, String)		64%		60%	4	6	6	19	0	1
addScroll(String, File, String)		80%		55%	8	10	6	30	0	1
getScrollPreview(int)		74%		83%	1	4	7	22	0	1
deleteScroll(int)		46%		50%	1	2	7	14	0	1
getAllScrollByID(String)		80%		66%	2	4	5	24	0	1
addUser(String, String, String, String, String, String)		78%		50%	9	10	4	22	0	1
deleteUserByUserID(String)		63%		66%	2	4	5	16	0	1
getAllScrollInfo()		85%		100%	0	2	4	23	0	1
getScrollInfoExceptContentByScrollID(int)		83%		50%	1	2	5	22	0	1
getAllUserInfo()		82%		100%	0	2	4	21	0	1
getScrollContent(int)		63%		50%	1	2	5	12	0	1
getUserInfoByUserID(String)		89%		66%	2	4	3	26	0	1
Database(String)		100%		n/a	0	1	0	8	0	1
static {...}		100%		n/a	0	1	0	1	0	1
Total	1,012 of 1,952	48%	77 of 148	47%	74	104	281	535	12	30

Currently, the test coverage for `Database` class is as follows:

- **Instruction Coverage**: 48%
- **Branch Coverage**: 47%

During the development process of Sprint 2, we encountered significant challenges related to version control that resulted in the accidental loss of tests. Specifically, when a team member's branch was merged into the main branch, the reversion of certain commits inadvertently erased the test cases that had been developed. These tests will be recovered in the next sprint aiming for a higher test coverage.

However, for the already existing tests, we have done the following tests (in general):

- **When inserting and updating a data entry:**

    - **Positive Test Cases:**
        - Normal insertion or update with all required parameters. This test ensures that the data is correctly inserted or updated when all required fields are provided.
    - **Negative Test Cases:**
        - **Primary Key/Unique Constraint Violation:**
            - If the database enforces a primary key (or unique) constraint (e.g., `userid`), a test case should be designed where a duplicate entry is attempted, such as inserting a user with an existing `userid`. This validates the system's response to violations of uniqueness constraints.
        - **Missing Parameters:**
            - If the method requires multiple parameters, test the method by providing fewer parameters than required to verify if the system correctly identifies and handles missing input. For instance, inserting a user without the required email field.
- **When deleting a data entry:**
    - **Positive Test Cases:**
        - Normal deletion with a valid primary key. This ensures that the entry is successfully deleted when the correct identifier is provided.
    - **Negative Test Cases:**
        - **Invalid Primary Key:**
            - Attempt to delete an entry using a non-existent primary key to check if the system gracefully handles cases where the specified identifier does not match any record in the database.
        - **Empty or Null Identifier:**
            - Test deletion with an empty or `null` identifier to see if the system correctly identifies the invalid input and prevents the operation.

The following are all tests for `Database` class:

Class: Test Class	Method tested	Testcase type +: postive -: negative ~: edge case	Explanation
BeforeEach		setup()	set up the environment and testing database
Admin Database: DatabaseTest	AddUser	+ testAddUserSuccessfully()	Check the User database can insert a new user.
		- testAddUserWithDuplicateUserID()	Check the database can handle primary key (user id) repetition, error message will be thrown.
	GetUserInfoByUserID	+ testGetUserInfoByUserIDSuccessfully()	Check the database can retreive user information with a given valid user id.
		- testGetUserInfoByInvalidUserID()	Check the database can handle the case when the provided user id is invalid.
	GetAllUserInfo	+ testGetAllUserInfo()	Check the ability to retrieve all user information from the database.
	DeleteUser	+ testDeleteUserSuccessfully()	Check the ability to delete a user from the database with a valid user id.
		- testDeleteUserWithInvalidUserID()	Check the case that the user id is invalid when deleting a user.
	UpdateUserName	+ testUpdateUserNameSuccessfully()	Check the database can update user's fullname given a user id.
		- testUpdateUserNameWithInvalidUserID()	Check the case that the user id is invalid when updating name.
	UpdatePassword	+ testUpdatePasswordSuccessfully()	Check the database can update user's password given a user id.
		- testUpdatePasswordWithInvalidUserID()	Check the case that the user id is invalid when updating password.
	UpdatePhoneNumber	+ testUpdatePhoneNumberSuccessfully()	Check the database can update user's phone number given a user id.
	UpdateEmail	+ testUpdateEmailSuccessfully()	Check the database can update user's email given a user id.
		- testUpdateEmailWithInvalidUserID()	Check the case that the user id is invalid when updating email.
AfterEach		teardown()	method to auto delete the inserted data and close connection

Class: Test Class	**Method tested**	**Testcase type** **+: postive** **-: negative** **~: edge case**	**Explanation**
BeforeEach		setup()	set up the environment and testing database
Scroll Database: DatabaseTest	AddScroll	+ testAddScrollSuccessfully()	To check the Scroll database successfully insert a new data
		- testAddScrollWithDuplicateName()	To check the Scroll database cannot insert the error data
	GetScrollPreview	+ testPreviewScrollTxt() + testPreviewScrollTxtLongFile()	To check the preview method successfully return the readable file content within 20 characters
		- testPreviewScrollPdf()	To check the preview method cannot return the unreadable file content
	GetScrollContent	+ testGetScrollContent()	To check the get query successfully return the scroll content by the given scroll ID
	GetAllScrollInfo	+ testGetAllScrollInfo()	To check the get query successfully return all scrolls info
	DeleteScroll	+ testDeleteScroll()	To check the Scroll database successfully delete a scroll based on the given scroll ID
	UpdateScroll	+ testUpdateScroll()	To check the Scroll database successfully update a scroll based on the given scroll ID
	GetScrollInfoExceptContentByScrollID	+ testGetScrollInfoExceptContentByScrollID()	To check the get query successfully return required scrolls info except the content with the given scroll ID
	GetAllScrollByID	+ testGetAllScrollByID()	To check the get query successfully return required scrolls info with the given scroll ID
AfterEach		teardown()	method to auto delete the inserted data and close connection

3.4.3 Registered User Tests

> **Note**
>
> These tests are written by Huiying Jiao (hjia7380)

RegisteredUser

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
registration()		81%		85%	3	11	11	58	0	1
setNewPassword(String)		67%		100%	0	2	6	14	0	1
updateEmail(String)		72%		n/a	0	1	4	12	0	1
updatePhoneNumber(String)		72%		n/a	0	1	4	12	0	1
updateFullName(String)		72%		n/a	0	1	4	12	0	1
checkUserIDExists(String)		82%		75%	1	3	4	12	0	1
containsIllegalCharacters(String)		92%		75%	1	3	1	4	0	1
static {...}		100%		n/a	0	1	0	2	0	1
displayCurrentUserInfo()		100%		n/a	0	1	0	9	0	1
verifyOldPassword(String)		100%		100%	0	2	0	5	0	1
RegisteredUser(String)		100%		n/a	0	1	0	3	0	1
isValidEmail(String)		100%		n/a	0	1	0	1	0	1
isValidPhoneNumber(String)		100%		n/a	0	1	0	1	0	1
Total	69 of 467	85%	5 of 32	84%	5	29	34	145	0	13

Currently, the test coverage for `RegisteredUser` class is as follows:

- **Instruction Coverage**: 85%
- **Branch Coverage**: 84%

In these testcases, the test script simulates the user input, and compare the output from the program with the expected output.

Registration Tests include a positive test for successful registration when all fields are filled out correctly and several negative tests that assess user exits at various stages of the registration process, the handling of an invalid email format (not in `__@__.__`), and the ability to exit at different input fields.

Check User ID Exists Tests feature a positive case confirming the detection of an existing user ID, alongside a negative case testing the system's response to a non-existent user ID.

Display Current User Info Tests consist of a positive test that verifies the accurate display of user information.

Update Email, Phone Number, and Full Name Tests include positive tests to validate successful updates with valid input and negative tests to check the system's responses to illegal arguments during these updates.

Verify Old Password Tests assess the successful verification of the old password and its handling of incorrect input.

Set New Password Tests include a positive test for successfully setting a new password and a negative test to verify the system's response when the new password matches the old one.

The following are all tests for `RegisteredUser` class:

Class: Test Class	Method tested	Testcase type +: postive -: negative ~: edge case	Explanation
BeforeAll		setupDatabase()	Sets up an in-memory SQLite database and creates the User table for testing.
BeforeEach		resetDatabase()	Clears the User table and initializes a RegisteredUser instance before each test.
Register User: RegisterUserTest	registration	+ testRegistration()	Checks the successful registration of a user
		- testRegistrationExit()	Tests user canceling registration at the start
		- testRegistrationExitAtPhoneNumber()	Tests user canceling registration at the phone number stage.
		- testRegistrationExitAtEmail()	Tests user canceling registration at the email stage.
		- testRegistrationExitAtFullName()	Tests user canceling registration at the full name stage.
		- testRegistrationInvalidEmail()	Tests behavior when an invalid email is entered.
		- testRegistrationExitAtUserName()	Tests user canceling registration at the username stage.
		- testRegistrationExitAtUserID()	Tests user canceling registration at the user ID stage.
	checkUserIDExists	+ testCheckUserIDExists()	Checks if an existing user ID is correctly identified.
		- testCheckUserIDNotExists()	Checks if a non-existing user ID is correctly identified.
	displayCurrentUserInfo	+ testDisplayCurrentUserInfo()	Checks if displaying user information works properly.
		+ testUpdateEmail()	Tests the functionality of updating the user's email.

	updateEmail		
		- testUpdateEmail_illegalArgument()	Tests handling of invalid email update.
	updatePhoneNumber	+ testUpdatePhoneNumber()	Tests updating the user's phone number.
		- testUpdatePhoneNumber_illegalArgument()	Tests handling of invalid phone number update.
	updateFullName	+ testUpdateUserName()	Tests updating the user's full name.
		- testUpdateUserName_illegalArgument()	Tests handling of invalid full name update.
	verifyOldPassword	+ testVerifyOldPassword()	Tests verification of the correct old password.
		- testVerifyOldPassword_incorrect()	Tests behavior when an incorrect old password is provided.
	setNewPassword	- testSetNewPassword_sameAsOld()	Tests behavior when setting a new password that matches the old one.
		+ testSetNewPassword_success()	Tests updating the password to a new, different one.
AfterEach		teardown()	Closes the in-memory database connection after all tests.

3.4.4 User Profile Tests

> **Note**
>
> These tests are written by Huiying Jiao (hjia7380)

UserProfileManager

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
showUpdateOptions()		94%		90%	3	18	6	78	0	1
UserProfileManager(RegisteredUser, Interface)		100%		n/a	0	1	0	4	0	1
Total	10 of 204	95%	3 of 30	90%	3	19	6	82	0	2

Currently, the test coverage for `RegisteredUser` class is as follows:

- **Instruction Coverage**: 85%
- **Branch Coverage**: 84%

In these testcases, the test script simulates the user input, and compare the return value from the method with the expected return value.

The tests cover various scenarios for updating user information, including email, phone number, name, and password.

For each update type (email, phone number, and name), there are positive test cases that confirm the update method is called when changes are made, and negative test cases that verify the method is not called if the update is canceled.

For password updates, the tests are more comprehensive, including positive validation of the update process when the correct old password is provided and a new one is set. The negative cases include handling incorrect old passwords, canceling the update before verification, entering invalid new passwords, and canceling the process after a valid old password is entered.

The following are all tests for `UserProfileManager` class:

Class: Test Class	Method tested	Testcase type +: postive -: negative ~: edge case	Explanation
BeforeEach		setup()	set up the environment for user profile
User Profile Manager: UserProfileManagerTest	ShowUpdateOptions	+ testUpdateEmail()	Simulates updating the email and verifies that the updateEmail method is called.
		- testUpdateEmail_cancel()	Simulates canceling the email update and verifies that the updateEmail method is not called.
		+ testUpdatePhoneNumber()	Simulates updating the phone number and verifies that the updatePhoneNumber method is called.
		- testUpdatePhoneNumber_cancel()	Simulates canceling the phone number update and verifies that the updatePhoneNumber method is not called.
		+ testUpdateUserName()	Simulates updating the full name and verifies that the updateFullName method is called.
		- testUpdateUserName_cancel()	Simulates canceling the full name update and verifies that the updateFullName method is not called.
		+ testUpdatePassword()	Simulates updating the password and verifies that the verifyOldPassword and setNewPassword methods are called.
		- testUpdatePassword_cancel()	Simulates canceling the password update and verifies that the verifyOldPassword method is not called.
		- testCancelUpdatePassword()	Simulates canceling the password update process and verifies that no update methods are called.
		- testUpdatePassword_incorrectOldPassword()	Simulates entering an incorrect old password and verifies that the correct error message is displayed.
		- testUpdatePassword_newPasswordCancel()	Simulates canceling the new password input after entering the correct old password and verifies that the update process is stopped.

			stopped.
		- testUpdatePassword_tryDifferentPassword()	Simulates entering a disallowed new password and verifies that the user is prompted to try a different password.

Appendix

Meeting Record

Time	Duration	Description
2024-10-10	10:00–10:30	Sprint 1 Retrospective meeting, discussing what went well, discovered b release, and plans for tackling the new feature in sprint 2.
2024-10-11	20:00–21:00	Sprint 2 Sprint planning meeting, voting for the story points, and allocati developers. No recording.
2024-10-13	20:00–20:30	Scrum meeting, demonstrating individual progress. Recording: https://ur sydney.zoom.us/rec/share/uu5DU5E9Ty2GR-53Q1alZ8yEF1NPUO2zE8DMdCqVibQocPSPQsPtvEJ9mMYv35pZ.3Y Password: 2*V1RstH
2024-10-14	20:00–21:00	Scrum meeting, demonstrating individual progress.
2024-10-15	20:00–21:30	Scrum meeting, demonstrating individual progress and preparing for spr Recording: https://uni-sydney.zoom.us/rec/share/ijL64WFLM1ZRA37TIk y15iTAKrVXIAL1IlsPChNkqtWxqE9mYVVo1qa0.mZuOkUmcvrxnRHt3 P
2024-10-17	09:00–09:30	Sprint Review meeting with tutor.

Burndown Chart Creation

The `burndown.xls` provided on Canvas is used to generate the burndown chart and velocity chart for sprint 2